HYBRID CONTROL MOMENT GYROSCOPE ANGULAR MOMENTUM DESATURATION USING MAGNETORQUERS FOR CUBESATS IN LOW EARTH ORBIT

BY
KRITTIN JINDAMAI

AN HONORS THESIS PRESENTED TO THE
HERBERT WERTHEIM COLLEGE OF ENGINEERING
OF THE UNIVERSITY OF FLORIDA IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF BACHELOR OF SCIENCE IN AEROSPACE
ENGINEERING WITH SUMMA CUM LAUDE HONORS

UNIVERSITY OF FLORIDA

2027

This page is intentionally left blank

Acknowledgements:

I would like to express my gratitude to Dr. Norman Fitz-Coy for providing me with guidance and support needed to accomplish this Thesis. Additionally, I would like to thank Dr. _____ and Dr. _____ who serve as the advisory undergraduate honors thesis committee.

Abstract of Thesis Presented to the Herbert Wertheim College of Engineering of the University of Florida in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Aerospace Engineering with Summa Cum Laude Honors

HYBRID CONTROL MOMENT GYROSCOPE ANGULAR MOMENTUM DESATURATION USING MAGNETORQUERS FOR CUBESATS IN LOW EARTH ORBIT

With increasing number of CubeSats research on formation flying, docking, and rendezvous, they also increasingly require both rapid retargeting and precise attitude control. However, the limited volume available on small satellites often restricts the use of multiple actuators. Patankar proposed a Hybrid Control Moment Gyroscope (HCMG) to address this limitation by combining the high-torque slewing capability of a control moment gyroscope (CMG) with the fine-pointing capability of a reaction wheel (RW). However, the proposed HCMG system was validated in the absence of external disturbance torques.

This thesis investigates the use of magnetorquers (MTQs) for active momentum desaturation of an HCMG system under simulated environmental disturbances. A 6DoF attitude and orbit simulation was developed in MATLAB for a 12U, Earth-observing CubeSat operating in low Earth orbit. The simulation includes disturbances such as aerodynamic drag, J2 gravitational perturbations, and gravity gradient torque. MTQ was implemented using the cross-product desaturation law and compared with a baseline case in which desaturation was disabled.

The results show that, without MTQ, excess angular momentum accumulates and flywheel speeds gradually move away from their nominal operating point. When MTQ desaturation is enabled, the momentum does not accumulate, and the flywheel speeds are driven back toward their nominal values. The effect on pointing accuracy is negligible because the momentum buildup occurs relatively slowly. The cross-product desaturation law also has an inherent geometric limitation: it can only remove the component of excess angular momentum perpendicular to the local geomagnetic field. As a result, the effectiveness of momentum dumping depends on orbit geometry and magnetic-field direction, rather than on actuator capability alone. Overall, the results indicate that magnetorquer-based desaturation can provide a practical, low-power method for managing momentum in HCMG-equipped CubeSats and extending their operational lifetime without requiring continuous actuator operation.

Honors Thesis Committee:

Associate Professor Norman G. Fitz-Coy (Honors Thesis Advisor), Department of Mechanical and Aerospace Engineering

_____ Prof. _____

_____ Prof. _____

Table of Contents:

List of Tables:

List of Figures:

Nomenclature:

Bold symbols denote vectors and matrices and italic symbols denote scalars. A right superscript indicates the reference frame in which a quantity is resolved.

List of Abbreviations:

CMG	Control moment gyroscope
COM	Centre of mass
COP	Centre of pressure
DGCMG	Dual gimbal control moment gyroscope
DOF	Degrees of freedom
ECEF	Earth-centered, Earth-fixed
ECI	Earth-centered inertial
GISL	Generalized inverse steering law
GSR	Generalized singularity robust inverse
HCMG	Hybrid control moment gyroscope
LEO	Low Earth orbit
LG	Local gradient
LVLH	Local vertical, local horizontal
MTQ	Magnetorquer
PQW	Perifocal coordinate frame
RAAN	Right ascension of the ascending node
RCS	Reaction control system
RW	Reaction wheel
SGCMG	Single gimbal control moment gyroscope
SR	Singularity robust inverse
SRP	Solar radiation pressure
VSCMG	Variable speed control moment gyroscope
WGS-84	World Geodetic System 1984

Latin Symbols:

$\boldsymbol{A}$	Gyroscopic Jacobian of the HCMG cluster; its columns are the torque axes
$\boldsymbol{A_i}$	Torque axis of HCMG i expressed in the body frame

A_{face}	Area of flat plate i in the aerodynamic model	m^2
A_{coil}	Cross-sectional area of a magnetorquer coil	m^2
a	Orbital semi-major axis	m
a_{aero}	Acceleration due to aerodynamic drag	m/s^2
a_{J2}	Acceleration due to Earth oblateness	m/s^2
$\boldsymbol{B}$	Flywheel spin-axis distribution matrix of the cluster	
$\boldsymbol{B_i}$	Flywheel spin axis of HCMG i expressed in the body frame	
b	Geomagnetic field vector	T
$\boldsymbol{C}$	Gimbal-axis matrix of the cluster	
$\boldsymbol{C_i}$	Gimbal axis of HCMG i expressed in the body frame	
C_D	Flat plate drag coefficient	
c	Speed of light in vacuum	m/s
$\boldsymbol{d}$	Null-motion vector	rad/s
$\boldsymbol{d_{RW}}$	Normalized null vector chosen to increase the reaction wheel singularity parameter	
$\boldsymbol{E}$	Modulation matrix of the generalized singularity-robust inverse	
e	Orbital eccentricity	
e_i	Off-diagonal element of E	
e_o	Amplitude of the GSR modulation	
$\boldsymbol{F_{aero}}$	Aerodynamic force	N
$\boldsymbol{F_{srp}}$	Solar radiation pressure force	N
f	True anomaly	rad
H	Scale height of an atmospheric layer	m
h_{alt}	Geodetic altitude above the reference ellipsoid	m
h_o	Base altitude of an atmospheric layer	m
$\boldsymbol{H_c}$	Total centroidal angular momentum of the spacecraft and cluster	Nms
$\boldsymbol{h}$	Angular momentum of the HCMG cluster	Nms
$\boldsymbol{h_{ex}}$	Excess flywheel angular momentum above nominal	Nms
$\boldsymbol{h_{nom}}$	Nominal flywheel angular momentum	Nms
h_w	Angular momentum of a single flywheel	Nms
I	Magnetorquer coil current	A

I_w	Flywheel moment of inertia about its spin axis	kgm²
I_g	Gimbal assembly moment of inertia about the gimbal axis	kgm²
$\boldsymbol{I_{CMG_i}}$	Inertia tensor of HCMG i expressed in its gimbal frame	kgm²
i	Orbital inclination	rad
$\boldsymbol{J_C}$	Centroidal inertia tensor of the spacecraft including the cluster	kgm²
$\boldsymbol{\bar{J}_C}$	Centroidal inertia tensor of the non-moving parts of the spacecraft	kgm²
J_2	Second zonal harmonic coefficient of the Earth gravity field	
K_p, K_d	Proportional and derivative gains of the eigenaxis controller	
k	Cross-product desaturation gain	s^{-1}
m_s	Singularity measure of the cluster	
m_{sat}	Spacecraft mass	kg
m_{cmg}	Mass of one HCMG assembly	kg
$\boldsymbol{m}$	Magnetic dipole moment	Am²
$\boldsymbol{m_{cmd}}$	Commanded magnetic dipole moment	Am²
m_{sat}	Unit vector along the geomagnetic dipole axis	
N_{cmg}	Number of HCMGs in the cluster	
N_{face}	Number of flat plates in the aerodynamic model	
N_{turn}	Number of turns in a magnetorquer coil	
$\boldsymbol{N}$	Basis of the null space of the gyroscopic Jacobian	
$\boldsymbol{\hat{n}_i}$	Outward unit normal of face i	
$\boldsymbol{P}$	Projection of each flywheel spin axis onto the singular direction	Nms
P_{srp}	Solar radiation pressure at one astronomical unit	N/m²
$\boldsymbol{Q}$	Singularity classification matrix	Nms
$\boldsymbol{q}$	Spacecraft attitude quaternion, scalar last	
$\boldsymbol{q_d}$	Desired attitude quaternion	
$\boldsymbol{q_e}$	Attitude error quaternion	
$\boldsymbol{R_B^A}$	Rotation matrix transforming components from frame B to frame A	
$\boldsymbol{R_{COM}}$	Offset from the geometric center to the center of mass	m

$\boldsymbol{R_{CMG_i}}$	Position of HCMG i relative to the spacecraft center of mass	m
R_E	Earth equatorial radius	m
$\boldsymbol{r}$	Spacecraft position vector	m
$\boldsymbol{r_{cp_i}}$	Centre of pressure of face i relative to the center of mass	m
S	Mean solar flux at one astronomical unit	W/m²
S_{CMG}	Singularity parameter of the control moment gyroscope mode	
S_{RW}	Singularity parameter of the reaction wheel mode	
$\boldsymbol{s}$	Singular direction of the cluster	
$\hat{\boldsymbol{s}}$	Unit vector from the spacecraft to the Sun	
T	Orbital period	s
t	Time	s
$\boldsymbol{v}$	Spacecraft velocity vector	m/s

Greek Symbols:

α	Mode switch parameter	
β	Null-motion gain	rad/s
β_o	Maximum value of the null-motion gain	rad/s
γ	Tilt angle of the geomagnetic dipole from the rotation axis	rad
$\boldsymbol{\delta}$	Gimbal angle vector of the cluster	rad
δ_i	Gimbal angle of HCMG i	rad
$\widehat{\boldsymbol{\delta}}_{\boldsymbol{i}}$	Unit vector along the gimbal axis of HCMG i	
$\boldsymbol{\delta^S}$	Gimbal angles at a singular configuration	rad
$\boldsymbol{\Delta\delta}$	Gimbal angle perturbation from a singular configuration	rad
$\boldsymbol{\varepsilon}$	Vector part of the attitude quaternion	
$\boldsymbol{\varepsilon_e}$	Vector part of the attitude error quaternion	
η	Scalar part of the attitude quaternion	
η_e	Scalar part of the attitude error quaternion	
θ	Skew angle of the HCMG cluster arrangement	rad
$\boldsymbol{\kappa}$	Scaling vector of the null space basis	
λ	Singularity parameter of the SR and GSR inverses	
λ_o	Maximum value of the singularity parameter	

μ	Earth standard gravitational parameter	m^3/s^2
μ_o	Permeability of free space	H/m
μ_m	Magnetic dipole moment of the Earth	Am^2
μ_1, μ_2	Decay constants of the singularity parameters	
$\boldsymbol{\varsigma}$	Position of a mass element relative to the center of mass	m
ρ	Atmospheric density	kg/m^3
ρ_o	Reference density at the base of an atmospheric layer	kg/m^3
ε	Surface reflectivity coefficient	
ϵ_i	Sign parameter of the singular direction, equal to +1 or −1	
$\boldsymbol{\tau_{gyro}}$	Gyroscopic torque produced by the cluster	Nm
$\boldsymbol{\tau_{D_w}}$	Flywheel motor direct torque	Nm
$\boldsymbol{\tau_{D_g}}$	Gimbal motor direct torque	Nm
$\boldsymbol{\tau_{C_{sat}}}$	Gyroscopic coupling due to spacecraft asymmetry	Nm
$\boldsymbol{\tau_{C_w}}$	Coupling between flywheel momentum and spacecraft rotation	Nm
$\boldsymbol{\tau_{C_g}}$	Coupling between gimbal momentum and spacecraft rotation	Nm
$\boldsymbol{\tau_{gg}}$	Gravity gradient torque	Nm
$\boldsymbol{\tau_{aero}}$	Aerodynamic torque	Nm
$\boldsymbol{\tau_{srp}}$	Solar radiation pressure torque	Nm
$\boldsymbol{\tau_{MTQ}}$	Magnetorquer torque	Nm
$\hat{\boldsymbol{\tau}}_{\boldsymbol{i}}$	Unit torque axis of HCMG i	
ϕ	Phase shift of the GSR modulation	rad
$\boldsymbol{\Omega}$	Flywheel speed vector of the cluster	rad/s
Ω_i	Flywheel speed of HCMG i	rad/s
$\widehat{\boldsymbol{\Omega}}$	Diagonal matrix of flywheel speeds	rad/s
Ω_{nom}	Nominal flywheel speed	rad/s
Ω_{RAAN}	Right ascension of the ascending node	rad
$\boldsymbol{\omega_d}$	Desired angular velocity	rad/s
$\boldsymbol{\omega_e}$	Angular velocity error	rad/s
ω_p	Argument of perigee	rad
$\omega_\oplus$	Earth rotation rate	rad/s

1) Introduction:

Over the past few years CubeSats or nanosatellites began to rise in popularity in both research and in industry. CubeSats are categorized based on the standardized unit (U). A single unit, 1U, has a dimension of 10 by 10 by 10 cm. A CubeSat can be made into any size from 1U up to 27U. Due to its small size, CubeSat lowers the overall mission budget and cuts the development time. These make it more accessible for researchers to build and test new space technologies within a shorter period of time. With the rise of more advanced technologies, cameras and sensors can be made smaller, making them suitable for CubeSats. Industries benefit from these as more advanced satellites such as earth observational satellites and Telecommunication satellites can be made much smaller than their traditional sizes.

Nowadays, many researchers put their focus on constellation or formation flying of system of CubeSats. Each satellite is given various tasks, whether it is to observe the Earth, measure certain data, communicate with the Earth, or communicate with each other; these tasks all require satellite's capabilities to rapidly retarget or precisely point at a certain attitude. Satellites commonly utilize actuators such as reach wheels (RW), control moment gyroscopes (CMG), or reaction control systems (RCS) to perform these tasks. However, these actuators usually have tradeoffs such as RW may perform precision pointing but may not provide enough torque for rapid targeting (R2), while CMG may provide high torque, but cannot perform precision pointing (P2).

Due to limited volume, most CubeSats do not have enough volume available for more than one type of actuator. In 2015 Kunal Patankar proposed a hybrid control moment gyroscope (HCMG) for CubeSats that combines both CMG and RW to allow for both rapid retargeting and precision pointing [1]. Patankar proposed steering algorithm that safely kept HCMG from singularities when operating as both CMG and RW. However, the actuator could potentially fail and will be unable to provide enough torque if the satellite's excess angular momentum that builds up overtime is not being properly managed and rejected.

There are many factors that lead to angular momentum building up. Attitude maneuvering or slewing itself stores momentum in RW and CMG which leads to building up of momentum in the satellite. The major cause, however, comes from environmental disturbances in space. Disturbances such as aerodynamic drag, solar radiation pressure, and gravity gradient can generate torque that disturbs satellite's attitude. Actuators must then put more effort into controlling the satellite back to its nominal attitude, causing angular momentum to constantly build up over time.

Various momentum management and unloading exist and are categorized as passive and active methods. Passive methods utilize the environmental disturbance to strategically unload

excess momentum. Active methods involve the use of actuators to generate torque in a direction that unloads the momentum. The actuators that are widely used for this purpose include reaction control systems (RCS), and magnetorquers (MTQ). RCS, however, are not commonly used for desaturation in CubeSats due to its size and the possibility of plane change when the thruster fires. MTQ are torque rods that interact with Earth's magnetic field to generate torque in the desired direction. When controlled using the cross-product desaturation law, MTQ can generate torque in a direction that desaturates excess angular momentum in the system. Patankar proposed a momentum redistribution method specifically for an HCMG system treating desaturation torque as a disturbance torque to drive the actuator back to its nominal speed. This method, however, was proposed and tested under the assumption of no disturbance torques on the system.

This thesis was made to explore the effectiveness of magnetorquer as a desaturation method for excess angular momentum in an HCMG system under simulated in-orbit conditions. A 12U CubeSat in a low Earth orbit is modeled in a full nonlinear attitude and orbit simulation developed entirely in MATLAB. The simulation includes aerodynamic drag, J2 perturbation, and gravity gradient as disturbances. Additionally, the geomagnetic field is represented by a tilted dipole model whose axis is rotated from the Earth-fixed frame into inertial components as a function of simulation time, which is used for the computations of cross-product law.

The thesis includes 9 chapters. Chapter 2 introduces common actuators used for satellite attitude control. Different types of RW and CMGs are discussed in this chapter. Chapter 3 formulates the dynamics of an HCMG. Chapter 4 discusses different types of HCMG singularities both internal and external, hyperbolic and elliptical. Chapter 5 discusses different types of HCMG configuration and their respective singularity surfaces. Chapter 6 introduces steering algorithms for singularity escape and avoidance. Chapter 7 discusses the common disturbances that satellite experiences. Chapter 8 discusses different methods of angular momentum management and explains why MTQ was chosen as the method for this thesis. Lastly, Chapter 9 outlines the simulation setup and discusses the simulation results.

2) Common Actuators:

2.1) Reaction Wheel:

A reaction wheel (RW) is a momentum exchange using an electrically driven variable speed flywheel. Reaction wheel usually operates at a nominal flywheel speed. As the wheel spins it stores angular momentum, h_w along the spin axis. The angular momentum is based on flywheel's moment of inertia I_w and its angular velocity Ω, which can be expressed as

$$h_w = I_w \Omega \tag{2-1}$$

The moment of inertia is assumed to be constant, while Ω can vary with time. As Ω changes, h_w changes which induces a flywheel torque τ_w along the spin axis as shown

$$\tau_w = \dot{h}_w = I_w \dot{\Omega} \tag{2-2}$$

The torque τ_w, exerts a reaction torque that causes the spacecraft to rotate in the opposite direction to conserve the total angular momentum. As the flywheel spin axis is fixed relative to the spacecraft body, the exerted torque is constrained along the flywheel spin axis. In order to have a full 3-axis attitude control of spacecraft, at least three non-coplanar reaction wheels must be used [1].

Reaction wheel systems are usually limited by the driving motor power input. The flywheels usually have large moments of inertia, but most CubeSats do not have enough space for large driving servo, so the power input is limited, and the flywheel angular acceleration is low [1]. Although the output torque is low, it is advantageous for applications that require precision pointing.

2.2) Control Moment Gyroscope:

A control moment gyroscope (CMG) consists of a flywheel and a gimbal motor. Unlike reaction wheels, the flywheel speed is constant. It generates gyroscopic torque by changing the gimbal angles to change the angular momentum vector. Because the gyroscopic torque is nearly orthogonal to the flywheel angular velocity, high torque can be produced for a much less input power when compared to reaction wheels [1]. Therefore, CMGs are suited for large angle attitude control such as rapid retargeting. CMG can be classified based on the number of gimbal axes as single gimbal control moment gyroscope (SGCMG) and dual gimbal control moment gyroscope (DGCMG) [2]. Classifications can also be made based on the flywheel speed. While conventional CMGs flywheel speed is kept constant, variable speed control moment gyroscope (VSCMG) allows for adjustment of both gimbal angle and flywheel speed.

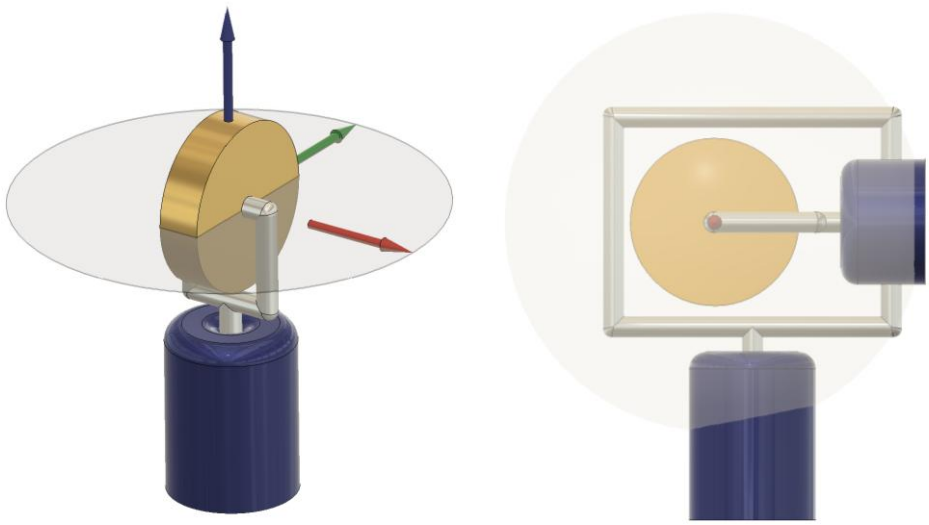

Figure 2-1: SGCMG (Left) DGCMG (Right)

2.2.1) Single Gimbal Control Moment Gyroscope:

The flywheel of an SGCMG is gimbaled on only one axis. The angular momentum of the flywheel, h_w is the same as the expression from (*2-1*), where I_w is the flywheel moment of inertia and Ω is flywheel angular velocity. The gimbal angle is defined as δ, with $\dot{\delta}$ referring to the gimbal angular velocity. With the gimbal motor moment of inertia being I_g, the angular momentum of the gimbal motor, h_g can be expressed as

$$h_g = I_g \dot{\delta} \tag{2-3}$$

Assuming that the center of mass is fixed to the center of the gimbal motor. As shown in Figure 2-1, a Gimbal-fixed reference frame $\boldsymbol{F_G}$ is defined with the origin at the center of mass of an SGCMG with the basis vector $\hat{h}$, $\hat{\tau}$, and $\hat{\delta}$. Where $\hat{h}$ points along the flywheel axis, $\hat{\delta}$ points along the gimbal axis, and $\hat{\tau}$ is formed from the cross product of $\hat{\delta}$ and $\hat{h}$. Usually, many SGCMG work in conjunction in a cluster to produce the desired torque. The angular momentum of the i^{th} SGCMG in a cluster $\boldsymbol{h_i}$, expressed in its own gimbal fixed frame $\boldsymbol{F_{Gi}}$ can be written as

$$\boldsymbol{h_i^{Gi}} = \begin{bmatrix} I_w \Omega_i \\ 0 \\ I_G \dot{\delta}_i \end{bmatrix} \tag{2-4}$$

The torque produced by the i^{th} SGCMG in a CMG cluster is the time derivate of $\boldsymbol{h_i}$ with respect to the inertial reference frame.

$$\boldsymbol{\tau_i} = \frac{d}{dt}^{N} \boldsymbol{h_i^{Gi}} \tag{2-5}$$

Since the gimbal fixed frame rotates with respect to the inertial reference frame, the transport theorem can be applied to solve for $\boldsymbol{\tau_i}$.

$$\boldsymbol{\tau_i} = \frac{d}{dt}^{Gi} \boldsymbol{h_i^{Gi}} + (\vec{\omega} \times \boldsymbol{h_i^{Gi}}) \tag{2-6}$$

$$\boldsymbol{\tau_i} = \dot{\boldsymbol{h}}_{\boldsymbol{i}}^{\boldsymbol{Gi}} + (\dot{\boldsymbol{\delta}}_{\boldsymbol{i}}^{\boldsymbol{Gi}} \times \boldsymbol{h_i^{Gi}}) \tag{2-7}$$

Defining $\boldsymbol{\delta_i^{Gi}}$ as the gimbal angle of an SGCMG and $\dot{\boldsymbol{\delta}}_{\boldsymbol{i}}^{\boldsymbol{Gi}}$ as the angular velocity expressed in the gimbal-fixed frame.

$$\boldsymbol{\delta_i^{Gi}} = \begin{bmatrix} 0 \\ 0 \\ \delta_i \end{bmatrix} \tag{2-8}$$

$$\dot{\boldsymbol{\delta}}_{\boldsymbol{i}}^{\boldsymbol{Gi}} = \begin{bmatrix} 0 \\ 0 \\ \dot{\delta}_i \end{bmatrix} \tag{2-9}$$

Equation (2-7) can be rewritten as

$$\boldsymbol{\tau_i} = \begin{bmatrix} I_w \dot{\Omega}_\iota \\ 0 \\ I_G \ddot{\delta}_i \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \dot{\delta}_i \end{bmatrix} \times \begin{bmatrix} I_w \Omega_i \\ 0 \\ I_G \dot{\delta}_i \end{bmatrix} \tag{2-10}$$

For a constant speed CMG, $\dot{\Omega}_\iota$ is equal to zero. The term $I_G \ddot{\delta}_i$ is on many orders of magnitude less than the torque contribute on from the gyroscopic term $\vec{\omega} \times \boldsymbol{h_i^{Gi}}$, which can be considered negligible [1]. Therefore, the torque produced by an individual SGCMG will have a magnitude of $\dot{\delta}_i h_w$ pointing in the $\hat{\tau}$ direction.

$$\boldsymbol{\tau_i} \approx \begin{bmatrix} 0 \\ \dot{\delta}_i h_w \\ 0 \end{bmatrix} \tag{2-11}$$

As shown in equation (*2-11*) and Figure 2-1, the gyroscopic torque produced by an SGCMG will always point along $\hat{\tau}$, which lies on the same plane as $\hat{h}$. This plane is known as the torque plane, and it prevents a single SGCMG from having full control authority over all 3 axes of the spacecraft. Thus, more than three SGCMG, and in practice four, are used in a cluster.

2.2.2) Dual Gimbal Control Moment Gyroscope:

Dual gimbal or double gimbal control moment gyroscope (DGCMG) has one flywheel and two gimbal axes. The added degree of freedom from an extra gimbal axis makes the torque envelope spherical. This allows DGCMGs to have full 3 axis attitude control of the spacecraft with less than 3 DGCMGs in a cluster [1]. However, when both gimbal axes are aligned, control authority is lost as the torque envelope becomes planar rather than spherical. This alignment is known as the gimbal lock. Additionally, the gyroscopic torque given by both gimbal motors must balance each other out, leading to higher power consumption. While steering algorithms for DGCMG singularity avoidance exist, DGCMG is not the focus of this thesis as SGCMG is more suitable for hybrid control moment gyroscopes due to its simplicity.

2.2.3) Variable Speed Control Moment Gyroscope:

Variable speed control gyroscope (VSCMG) is an SGCMG with a variable speed flywheel. This gives the CMG an additional controllable degree of freedom, without requiring an extra gimbal axis. VSCMGs are mainly used for 2 applications. The first and main application is singularity escape and avoidance. VSCMGs can provide flywheel angular acceleration to produce additional toque, whenever the gimbals enter a singular configuration. Thus, it eliminates the need for the complex singularity-escape logic required for SGCMGs. VSCMGs are also used in Integrated Power and Attitude Control Systems (IPACS) or Flywheel Attitude Control and Energy Transmission Systems (FACETS) [2].

2.2.4) Hybrid Control Moment Gyroscope:

A hybrid control moment gyroscope (HCMG) is similar to VSCMG in that they both are SGCMG with a variable speed flywheel. However, they differ in the concept of operation. Unlike VSCMG where its main application is for singularity avoidance and energy management, Kunal Shrikant Patankar introduced HCMG in 2015 to utilize CMG rapid retargeting capability and reaction wheels precision pointing capability. HCMG utilizes a mode switch parameter to switch between rapid retargeting (R2) phase and precision pointing (P2) phase. During the R2 phase, HCMG operates as an SGCMG [1]. During P2 phase the gimbal angles are locked in place while the flywheels spin with varying speed. Since the variable speed flywheel and SGCMG do not operate concurrently, more than 3 HCMGs are needed to have a full 3-axis attitude control of spacecraft during R2 and P2 phases.

3) Dynamics Model:

To formulate the dynamics model of a satellite with N HCMGs the satellite is modeled as a rigid body. The model also assumes no disturbance torques such as aerodynamic and gravity gradient torques. However, since the focus of this thesis is regarding the excess momentum that accumulates due to disturbances, disturbance torque will be modeled separately and added into the system. Two reference frames needed to model the dynamics are spacecraft body-fixed frame $\boldsymbol{F_B}$ and gimbal-fixed frame of each HCMG in a cluster, $\boldsymbol{F_{G_i}}$. Figure 3-1 shows and example of a pyramid cluster with 4 HCMGs showing gimbal fixed frames of HCMG#1, HCMG#2, HCMG#3, HCMG#4, and a common spacecraft body frame.

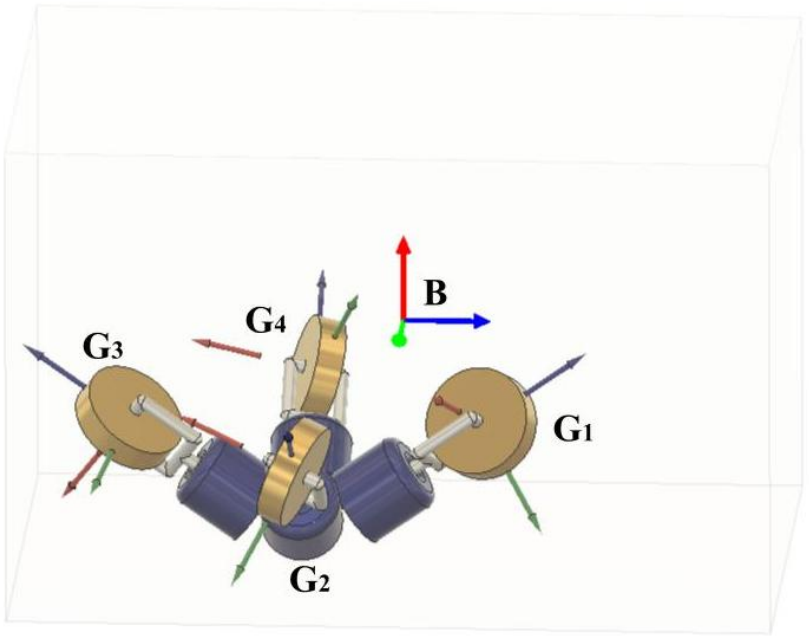

Figure 3-1: Body frame and Gimbal Frames

The total centroidal angular momentum of the spacecraft modeled in its body-fixed frame, $\boldsymbol{H_C^B}$ can be expressed as the sum of spacecraft's angular momentum and total angular momentum of the HCMG cluster, $\boldsymbol{h^B}$ expressed in body frame.

$$\boldsymbol{H_C^B = J_C^B \omega_B^B + h^B} \tag{3-1}$$

Where $\boldsymbol{J_C^B}$ represents the centroidal moment of inertia of the spacecraft and $\boldsymbol{\omega_B^B}$ is the angular velocity of the spacecraft. Since it is assumed that there are no disturbance torques, the spacecraft angular momentum is constant, and its derivative is equal to zero. Therefore, differentiating (*3-1*) with respect to time results in

$$\frac{d}{dt}\boldsymbol{H}_{\boldsymbol{C}}^{\boldsymbol{B}} = \frac{d}{dt}(\boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}}\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}} + \boldsymbol{h}^{\boldsymbol{B}}) + \vec{\boldsymbol{\omega}} \times (\boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}}\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}} + \boldsymbol{h}^{\boldsymbol{B}})$$

(3-2)

$$\dot{\boldsymbol{H}}_{\boldsymbol{C}}^{\boldsymbol{B}} = \boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}}\dot{\boldsymbol{\omega}}_{\boldsymbol{B}}^{\boldsymbol{B}} + \dot{\boldsymbol{J}}_{\boldsymbol{C}}^{\boldsymbol{B}}\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}} + \dot{\boldsymbol{h}}^{\boldsymbol{B}} + \boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{N}} \times (\boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}}\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}} + \boldsymbol{h}^{\boldsymbol{B}})$$

(3-3)

Defining $\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}\times}$ as the skew symmetric matrix of spacecraft angular velocity as

$$\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}\times} = \begin{bmatrix} 0 & -\omega_z^B & \omega_y^B \\ \omega_z^B & 0 & -\omega_x^B \\ -\omega_y^B & \omega_x^B & 0 \end{bmatrix}$$

(3-4)

Using $\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}\times}$ and defining $\dot{\boldsymbol{h}}^{\boldsymbol{B}}$ as the total output torque from a HCMG cluster expressed in body frame $\boldsymbol{\tau}^{\boldsymbol{B}}$, (*3-3*) can be rewritten as follows

$$\boldsymbol{0} = \boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}}\dot{\boldsymbol{\omega}}_{\boldsymbol{B}}^{\boldsymbol{B}} + \dot{\boldsymbol{J}}_{\boldsymbol{C}}^{\boldsymbol{B}}\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}} + \boldsymbol{\tau}^{\boldsymbol{B}} + \boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}\times}(\boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}}\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}} + \boldsymbol{h}^{\boldsymbol{B}})$$

(3-5)

The total angular momentum of an HCMG cluster expressed in spacecraft body frame is defined as the sum of individual HCMG angular momentum in body fixed frame. The individual HCMG angular momentum expressed in body fixed frame is

$$\boldsymbol{h}_{\boldsymbol{i}}^{\boldsymbol{B}} = \boldsymbol{C}_{\boldsymbol{BGi}}\boldsymbol{h}_{\boldsymbol{i}}^{\boldsymbol{Gi}}$$

(3-6)

Therefore, the total angular momentum of the cluster is

$$\boldsymbol{h}^{\boldsymbol{B}} = \sum \boldsymbol{C}_{\boldsymbol{BGi}}\,\boldsymbol{h}_{\boldsymbol{i}}^{\boldsymbol{Gi}}$$

(3-7)

Where $\boldsymbol{C}_{\boldsymbol{BGi}}$ is the direction cosine matrix that transforms each HCMG to spacecraft body fixed frame defined as

$$\boldsymbol{C}_{\boldsymbol{BGi}} = \begin{bmatrix} C11_i & C12_i & C13_i \\ C21_i & C22_i & C23_i \\ C31_i & C32_i & C33_i \end{bmatrix}$$

(3-8)

The elements inside the direction cosine matrix are functions of the chosen HCMG orientation of the cluster, which is fixed in time and the instantaneous gimbal angle, which is time a varying quantity. Thus, the rate of change of the transformation matrix is

$$\dot{\boldsymbol{C}}_{\boldsymbol{BGi}} = \boldsymbol{C}_{\boldsymbol{BGi}} \dot{\boldsymbol{\delta}}_{\boldsymbol{i}}^{\boldsymbol{Gi}\times} \tag{3-9}$$

Where $\dot{\boldsymbol{\delta}}_{\boldsymbol{i}}^{\boldsymbol{Gi}\times}$ is the skew symmetric matrix of the gimbal's relative angular velocity.

$$\boldsymbol{\delta}_{\boldsymbol{i}}^{\boldsymbol{Gi}\times} = \begin{bmatrix} 0 & -\dot{\delta}_i & 0 \\ \dot{\delta}_i & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \tag{3-10}$$

The torque produced by each HCMG in body-fixed frame is the time derivative of $\boldsymbol{h}_{\boldsymbol{i}}^{\boldsymbol{B}}$.

$$\boldsymbol{\tau}_{\boldsymbol{i}}^{\boldsymbol{B}} = \dot{\boldsymbol{h}}_{\boldsymbol{i}}^{\boldsymbol{B}} = \dot{\boldsymbol{C}}_{\boldsymbol{BGi}} \boldsymbol{h}_{\boldsymbol{i}}^{\boldsymbol{Gi}} + \boldsymbol{C}_{\boldsymbol{BGi}} \dot{\boldsymbol{h}}_{\boldsymbol{i}}^{\boldsymbol{Gi}} \tag{3-11}$$

Therefore $\boldsymbol{h}_{\boldsymbol{i}}^{\boldsymbol{B}}$ and $\boldsymbol{\tau}_{\boldsymbol{i}}^{\boldsymbol{B}}$ can be rewritten as

$$\boldsymbol{h}_{\boldsymbol{i}}^{\boldsymbol{B}} = \boldsymbol{B}_{\boldsymbol{i}} I_w \Omega_i + \boldsymbol{C}_{\boldsymbol{i}} I_{gw} \dot{\delta}_i \tag{3-12}$$

$$\boldsymbol{\tau}_{\boldsymbol{i}}^{\boldsymbol{B}} = \boldsymbol{A}_{\boldsymbol{i}} I_w \Omega_i \dot{\delta}_i + \boldsymbol{B}_{\boldsymbol{i}} I_w \dot{\Omega}_i + \boldsymbol{C}_{\boldsymbol{i}} I_{gw} \ddot{\delta}_i \tag{3-13}$$

Where

$$\boldsymbol{A}_{\boldsymbol{i}} = \begin{bmatrix} C12_i \\ C22_i \\ C32_i \end{bmatrix} \tag{3-14}$$

$$\boldsymbol{B}_{\boldsymbol{i}} = \begin{bmatrix} C11_i \\ C21_i \\ C31_i \end{bmatrix} \tag{3-15}$$

$$\boldsymbol{C}_{\boldsymbol{i}} = \begin{bmatrix} C13_i \\ C23_i \\ C33_i \end{bmatrix} \tag{3-16}$$

Therefore, the total angular momentum of a cluster of N_{cmg} HCMGs are as shown

$$\boldsymbol{h}_{\boldsymbol{CMG}}^{\boldsymbol{B}} = I_w \boldsymbol{B} \boldsymbol{\Omega} + I_{gw} \boldsymbol{C} \dot{\boldsymbol{\delta}} \tag{3-17}$$

$$\boldsymbol{\tau^B} = I_w \boldsymbol{A}\widehat{\boldsymbol{\Omega}}\dot{\boldsymbol{\delta}} + I_w \boldsymbol{B}\dot{\boldsymbol{\Omega}} + I_{gw}\boldsymbol{C}\ddot{\boldsymbol{\delta}} \tag{3-18}$$

Where

$$\boldsymbol{A} = [\boldsymbol{A_1}, \boldsymbol{A_2}, \dots, \boldsymbol{A_{N_{cmg}}}] \tag{3-19}$$

$$\boldsymbol{B} = [\boldsymbol{B_1}, \boldsymbol{B_2}, \dots, \boldsymbol{B_{N_{cmg}}}] \tag{3-20}$$

$$\boldsymbol{C} = [\boldsymbol{C_1}, \boldsymbol{C_2}, \dots, \boldsymbol{C_{N_{cmg}}}] \tag{3-21}$$

$$\boldsymbol{\Omega} = [\Omega_1, \Omega_2, \dots, \Omega_{N_{cmg}}]^{\boldsymbol{T}} \tag{3-22}$$

$$\widehat{\boldsymbol{\Omega}} = \begin{bmatrix} \Omega_1 & 0 & 0 & 0 \\ 0 & \Omega_2 & 0 & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \Omega_{N_{cmg}} \end{bmatrix} \tag{3-23}$$

$$\dot{\boldsymbol{\delta}} = [\dot{\delta}_1, \dot{\delta}_2, \dots, \dot{\delta}_{N_{cmg}}]^{\boldsymbol{T}} \tag{3-24}$$

$$\ddot{\boldsymbol{\delta}} = [\ddot{\delta}_1, \ddot{\delta}_2, \dots, \ddot{\delta}_{N_{cmg}}]^{\boldsymbol{T}} \tag{3-25}$$

The spacecraft moment of inertia term from equation (3-3), $\boldsymbol{J_C^B}$ is the sum of nonmoving parts total moment of inertial and time varying moment of inertia of the HCMG cluster. Given that the nonmoving part total moment of inertia is $\boldsymbol{J_C^B}$. The time varying moment of inertia of the HCMG cluster is a function of $\boldsymbol{\delta}$ and cand be expressed as

$$\sum_{\boldsymbol{i=1}}^{N_{cmg}} \left[\boldsymbol{C_{BGi}} \boldsymbol{I_{CMGi}^{Gi}} \boldsymbol{C_{BGi}^T}\right] \tag{3-26}$$

Where

$$\boldsymbol{I}_{\boldsymbol{CMG}_i}^{\boldsymbol{G}_i} = \begin{bmatrix} I_{wi} & 0 & 0 \\ 0 & I_{gi} & 0 \\ 0 & 0 & I_{gi} \end{bmatrix} \tag{3-27}$$

The gimbal and flywheel assembly is assumed axisymmetric about the flywheel spin axis, which makes $\boldsymbol{I}_{\boldsymbol{CMG}_i}^{\boldsymbol{G}_i}$ constant when expressed in the gimbal-fixed frame. Additionally, the HCMG cluster does not sit exactly at the center of mass of the spacecraft as shown in Figure 3-1. Parallel axis theorem is used to account for the offset of HCMG. Applying the parallel axis theorem, the parallel axis component is therefore

$$\sum_{\boldsymbol{i=1}}^{N_{cmg}} m_{CMG_i} \left[\boldsymbol{R}_{\boldsymbol{CMGi}}^{\boldsymbol{B}}{}^{\boldsymbol{T}} \boldsymbol{R}_{\boldsymbol{CMGi}}^{\boldsymbol{B}} \mathbf{1} - \boldsymbol{R}_{\boldsymbol{CMGi}}^{\boldsymbol{B}} \boldsymbol{R}_{\boldsymbol{CMGi}}^{\boldsymbol{B}}{}^{\boldsymbol{T}} \right] \tag{3-28}$$

Where m_{cmg_i} is the mass of ith HCMG in the cluster, $\boldsymbol{R}_{\boldsymbol{CMG_i}}^{\boldsymbol{B}}$ is the position of the instantaneous center of mass of i^{th} HCMG expressed in body-fixed frame, and $\mathbf{1}$ is a 3 by 3 identity matrix. Thus, the total moment of inertia of a satellite with N_{cmg} HCMGs is

$$\boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}} = \bar{\boldsymbol{J}}_{\boldsymbol{C}}^{\boldsymbol{B}} + \sum_{\boldsymbol{i=1}}^{N_{cmg}} \left[\boldsymbol{C}_{\boldsymbol{BGi}} \boldsymbol{I}_{\boldsymbol{CMGi}}^{\boldsymbol{Gi}} \boldsymbol{C}_{\boldsymbol{BGi}}^{\boldsymbol{T}} \right] + \sum_{\boldsymbol{i=1}}^{N_{cmg}} m_{cmg_i} \left[\boldsymbol{R}_{\boldsymbol{CMGi}}^{\boldsymbol{B}}{}^{\boldsymbol{T}} \boldsymbol{R}_{\boldsymbol{CMGi}}^{\boldsymbol{B}} \mathbf{1} - \boldsymbol{R}_{\boldsymbol{CMGi}}^{\boldsymbol{B}} \boldsymbol{R}_{\boldsymbol{CMGi}}^{\boldsymbol{B}}{}^{\boldsymbol{T}} \right] \tag{3-29}$$

When spacecraft's center of mass does not sit at the geometric center, $\bar{\boldsymbol{J}}_{\boldsymbol{C}}^{\boldsymbol{B}}$ must include an extra parallel axis term to account for the center of mass offset. With $\boldsymbol{J}_{\boldsymbol{gc}}^{\boldsymbol{B}}$ representing the inertia calculated based on the uniform CubeSat dimension about its geometric center and $\boldsymbol{R}_{\boldsymbol{COM}}^{\boldsymbol{B}}$ representing the center of mass offset expressed in spacecraft body-fixed frame.

$$\bar{\boldsymbol{J}}_{\boldsymbol{C}}^{\boldsymbol{B}} = \boldsymbol{J}_{\boldsymbol{gc}}^{\boldsymbol{B}} - m_{cmg} \left[\left(\boldsymbol{R}_{\boldsymbol{COM}}^{\boldsymbol{B}}{}^{\boldsymbol{T}} \boldsymbol{R}_{\boldsymbol{COM}}^{\boldsymbol{B}} \right) \mathbf{1} - \boldsymbol{R}_{\boldsymbol{COM}}^{\boldsymbol{B}} \boldsymbol{R}_{\boldsymbol{COM}}^{\boldsymbol{B}}{}^{\boldsymbol{T}} \right] \tag{3-30}$$

The expression shown in equation (3-29) can be simplified further. The HCMG cluster inertia about its own center of mass $\left[\boldsymbol{C}_{\boldsymbol{BGi}} \boldsymbol{I}_{\boldsymbol{CMGi}}^{\boldsymbol{Gi}} \boldsymbol{C}_{\boldsymbol{BGi}}^{\boldsymbol{T}} \right]$ is several orders of magnitude smaller than the spacecraft inertia and is therefore neglected [1]. The parallel-axis contribution is retained, because it scales with $m_i \boldsymbol{R}_{\boldsymbol{CMGi}}^{\boldsymbol{B}}{}^{\boldsymbol{2}}$ and is not negligible for HCMGs mounted away from spacecraft center of mass.

$$\boldsymbol{J_C^B} = \bar{\boldsymbol{J}}_{\boldsymbol{C}}^{\boldsymbol{B}} + \sum_{i=1}^{N_{cmg}} m_{cmg_i} \left[\boldsymbol{R_{CMGi}^B}^{\boldsymbol{T}} \boldsymbol{R_{CMGi}^B} \mathbf{1} - \boldsymbol{R_{CMGi}^B} \boldsymbol{R_{CMGi}^B}^{\boldsymbol{T}} \right] \tag{3-31}$$

Typically, CMGs are designed so that their center of mass lie along their gimbal axes [1]. This makes $\boldsymbol{R_{CMGi}^B}$ constant. Since the center of mass of the satellite is also fixed, the derivative of the satellite's moment of inertia term, $\dot{\boldsymbol{J}}_{\boldsymbol{C}}^{\boldsymbol{B}} \boldsymbol{\omega_B^B}$ can be neglected from the equation of motion, equation (*3-5*).

$$\mathbf{0} = \boldsymbol{J_C^B} \dot{\boldsymbol{\omega}}_{\boldsymbol{B}}^{\boldsymbol{B}} + \boldsymbol{\tau}^{\boldsymbol{B}} + \boldsymbol{\omega_B^B}^{\times} (\boldsymbol{J_C^B} \boldsymbol{\omega_B^B} + \boldsymbol{h}^{\boldsymbol{B}}) \tag{3-32}$$

$$\mathbf{0} = \boldsymbol{J_C^B} \dot{\boldsymbol{\omega}}_{\boldsymbol{B}}^{\boldsymbol{B}} + I_w \boldsymbol{A} \hat{\boldsymbol{\Omega}} \dot{\boldsymbol{\delta}} + I_w \boldsymbol{B} \dot{\boldsymbol{\Omega}} + I_{gw} \boldsymbol{C} \ddot{\boldsymbol{\delta}} + \boldsymbol{\omega_B^B}^{\times} \left(\boldsymbol{J_C^B} \boldsymbol{\omega_B^B} \right) + \boldsymbol{\omega_B^B}^{\times} (\boldsymbol{h_w^B}) + \boldsymbol{\omega_B^B}^{\times} (\boldsymbol{h_g^B}) \tag{3-33}$$

In order to simplify equation (*3-33*) the following terms are defined: 1) Gyroscopic torque $\boldsymbol{\tau_{gyro}^B}$ 2) flywheel motors direct torque $\boldsymbol{\tau_{D_w}^B}$ 3) gimbal motors direct torque $\boldsymbol{\tau_{D_g}^B}$ 4) gyroscopic coupling due to satellite asymmetry $\boldsymbol{\tau_{C_{sat}}^B}$ 5) gyroscopic coupling between flywheel angular momentum and satellite $\boldsymbol{\tau_{C_w}^B}$ 6) gyroscopic coupling between gimbal angular momentum and satellite $\boldsymbol{\tau_{C_g}^B}$. The corresponding term to each definition is as shown

$$\boldsymbol{\tau_{gyro}^B} = I_w \boldsymbol{A} \hat{\boldsymbol{\Omega}} \dot{\boldsymbol{\delta}} \tag{3-34}$$

$$\boldsymbol{\tau_{D_w}^B} = I_w \boldsymbol{B} \dot{\boldsymbol{\Omega}} \tag{3-35}$$

$$\boldsymbol{\tau_{D_g}^B} = I_{gw} \boldsymbol{C} \ddot{\boldsymbol{\delta}} \tag{3-36}$$

$$\boldsymbol{\tau_{C_{sat}}^B} = \boldsymbol{\omega_B^B}^{\times} \left(\boldsymbol{J_C^B} \boldsymbol{\omega_B^B} \right) \tag{3-37}$$

$$\boldsymbol{\tau_{C_w}^B} = \boldsymbol{\omega_B^B}^{\times} (\boldsymbol{h_w^B}) \tag{3-38}$$

$$\boldsymbol{\tau_{C_g}^B} = \boldsymbol{\omega_B^B}^{\times} (\boldsymbol{h_g^B}) \tag{3-39}$$

The angular acceleration of the gimbal motor $\ddot{\boldsymbol{\delta}}$ is much less than the angular acceleration of the flywheel $\dot{\boldsymbol{\Omega}}$ and $\boldsymbol{h}_{\boldsymbol{g}}^{\boldsymbol{B}}$ is much smaller than h_w^B. Thus, the terms $\boldsymbol{\tau}_{\boldsymbol{D}_{\boldsymbol{g}}}^{\boldsymbol{B}}$ and $\boldsymbol{\tau}_{\boldsymbol{C}_{\boldsymbol{g}}}^{\boldsymbol{B}}$ can be neglected from the governing equation of motion. The final equation of motion that describes satellite's with HCMG operating under CMG mode is shown in equation (*3-40*) and operating under RW mode is shown in equation (*3-41*).

The gimbal motor direct torque $\boldsymbol{\tau}_{\boldsymbol{D}_{\boldsymbol{g}}}^{\boldsymbol{B}}$ is negligible compared with the gyroscopic torque $\boldsymbol{\tau}_{\boldsymbol{gyro}}^{\boldsymbol{B}}$**.** Because the gimbal assembly inertial I_{gw} is several orders of magnitude smaller than the flywheel angular momentum $I_w\Omega$. Likewise, $\boldsymbol{h}_{\boldsymbol{g}}^{\boldsymbol{B}}$ is much smaller than $\boldsymbol{h}_{\boldsymbol{w}}^{\boldsymbol{B}}$**.** The terms $\boldsymbol{\tau}_{\boldsymbol{D}_{\boldsymbol{g}}}^{\boldsymbol{B}}$ and $\boldsymbol{\tau}_{\boldsymbol{C}_{\boldsymbol{g}}}^{\boldsymbol{B}}$ are therefore omitted.

$$\mathbf{0} = \boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}}\dot{\boldsymbol{\omega}}_{\boldsymbol{B}}^{\boldsymbol{B}} + \boldsymbol{\tau}_{\boldsymbol{gyro}}^{\boldsymbol{B}} + \boldsymbol{\tau}_{\boldsymbol{C}_{\boldsymbol{sat}}}^{\boldsymbol{B}} + \boldsymbol{\tau}_{\boldsymbol{C}_{\boldsymbol{w}}}^{\boldsymbol{B}} \tag{3-40}$$

$$\mathbf{0} = \boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}}\dot{\boldsymbol{\omega}}_{\boldsymbol{B}}^{\boldsymbol{B}} + \boldsymbol{\tau}_{\boldsymbol{D}_{\boldsymbol{w}}}^{\boldsymbol{B}} + \boldsymbol{\tau}_{\boldsymbol{C}_{\boldsymbol{sat}}}^{\boldsymbol{B}} + \boldsymbol{\tau}_{\boldsymbol{C}_{\boldsymbol{w}}}^{\boldsymbol{B}} \tag{3-41}$$

4)HCMG Singularity:

HCMG can operate differently during the two phases, CMG and RW. HCMG operates exactly as SGCMG system during CMG mode, and it operates as a reaction wheel system where the gimbal axes are fixed, during RW mode. Thus, HCMG singularity includes both SGCMG and RW singularities.

4.1) SGCMG Singularity:

Singularity for a CMG cluster occurs when the gimbals are oriented relative to each other in a way that gyroscopic torque lines up and control authority is lost [2]. When HCMG is operating as CMG mode, the torque output is given as

$$\boldsymbol{\tau}_{gyro}^{B} = I_w \boldsymbol{A}\widehat{\boldsymbol{\Omega}}\dot{\boldsymbol{\delta}} \tag{4-1}$$

The gyroscopic torque can be mapped onto the gimbal rate by taking the weighted right inverse of (4-1). This inverse requires that every flywheel speed must be non-zero.

$$\dot{\boldsymbol{\delta}} = \frac{1}{I_w}\widehat{\boldsymbol{\Omega}}^{-1}\boldsymbol{A}^T(\boldsymbol{A}\boldsymbol{A}^T)^{-1}\boldsymbol{\tau}_{gyro}^{B} \tag{4-2}$$

Since the Jacobian matrix $\boldsymbol{A}$ is a function of gimbal angles, the gimbal rate cannot be mapped onto gyroscopic torque and vice versa when matrix $\boldsymbol{A}$ becomes rank deficient. Null vector or null solution $\boldsymbol{d}$ is a set of gimbal rates or angles that reside in the null space of the Jacobian matrix $\boldsymbol{A}$, mathematically satisfying the condition

$$\boldsymbol{Ad} = \boldsymbol{0} \tag{4-3}$$

Because $\boldsymbol{Ad} = \boldsymbol{0}$, this solution produces no net gyroscopic torque on the satellite and does not change its total angular momentum. A continuous set of null solutions is called null motion.

Singularity can be classified into elliptic or hyperbolic based on whether null motion is possible. Elliptic singularity is when there is no null motion for that specific CMG configuration, while hyperbolic singularity is when null motion is possible [2]. In order to mathematically differentiate between elliptic and hyperbolic singularity, consider the first order Taylor Series expansion of CMG angular momentum about a singular configuration

$$\boldsymbol{h}(\boldsymbol{\delta}) - \boldsymbol{h}(\boldsymbol{\delta^S}) = \sum_{i=1}^{N_{cmg}} \left[\frac{\partial h_i}{\partial \delta_i}|_{\delta_i^S} \Delta\delta_i + \frac{1}{2}\frac{\partial^2 h_i}{\partial \delta_i^2}|_{\delta_i^S} \Delta\delta_i^2 + \cdots \right] \tag{4-4}$$

Where $\boldsymbol{\delta^S}$ is the gimbal angle at a specific singular configuration, i is the column of the Jacobian matrix, N is the number of CMGs, and $\Delta\delta_i$ is defined as

$$\Delta\delta_i = \delta_i - \delta_i^S \tag{4-5}$$

The terms in equation (*4-4*) can be simplified as follows

$$\frac{\partial h_i}{\partial \delta_i}|_{\delta_i^S} = h_i \hat{\boldsymbol{\tau}}_{\boldsymbol{i}} \tag{4-6}$$

$$\frac{\partial^2 h_i}{\partial \delta_i^2}|_{\delta_i^S} = \frac{\partial \hat{\tau}_i}{\partial \delta_i}|_{\delta_i^S} = -h_i \widehat{\boldsymbol{h}}_{\boldsymbol{i}} = -\boldsymbol{h}_{\boldsymbol{i}} \tag{4-7}$$

Substituting these expressions back into (*4-4*) yields

$$\boldsymbol{h}(\boldsymbol{\delta}) - \boldsymbol{h}(\boldsymbol{\delta^S}) \approx \sum_{i=1}^{N_{cmg}} \left[h_i \hat{\boldsymbol{\tau}}_{\boldsymbol{i}} \Delta\delta_i - \frac{1}{2}\boldsymbol{h}_{\boldsymbol{i}} \Delta\delta_i^2 \right]$$

(4-8)

Defining a singular direction vector $\boldsymbol{s}$ to be the direction orthogonal to torque direction, therefore no torque can be produced. $\boldsymbol{s}$ can be found from the left null space of the Jacobian matrix

$$\boldsymbol{s} = \text{null}(\boldsymbol{A^T})$$

(4-9)

$$\boldsymbol{A^T s} = \boldsymbol{s^T A} = \boldsymbol{0}$$

(4-10)

Taking the inner product of the expansion with $\boldsymbol{s}$

$$\boldsymbol{s^T}[\boldsymbol{h}(\boldsymbol{\delta}) - \boldsymbol{h}(\boldsymbol{\delta^S})] = \boldsymbol{s^T} \sum_{i=1}^{N_{cmg}} \hat{\boldsymbol{\tau}}_{\boldsymbol{i}}\, \Delta\delta_i - \frac{1}{2} \sum_{i=1}^{N_{cmg}} (\boldsymbol{s^T h_i}) \Delta\delta_i^2$$

(4-11)

Since $\boldsymbol{s}$ is orthogonal to $\hat{\boldsymbol{\tau}}_{\boldsymbol{i}}$, the first term can be neglected. Additionally, a projection matrix $\boldsymbol{P}$ can be defined as a diagonal matrix containing the projection of each CMG's spin axis onto the singular direction. It can be written as

$$\boldsymbol{P} = \text{diag}(\boldsymbol{s^T h_i})$$

(4-12)

The expansion (4-9) can then be rewritten as

$$\boldsymbol{s^T}[\boldsymbol{h}(\boldsymbol{\delta}) - \boldsymbol{h}(\boldsymbol{\delta^S})] = -\frac{1}{2} \Delta\boldsymbol{\delta^T P} \Delta\boldsymbol{\delta}$$

(4-13)

Null motion is a gimbal movement that does not change the system's total angular momentum. Thus, the term $\boldsymbol{h}(\boldsymbol{\delta}) - \boldsymbol{h}(\boldsymbol{\delta^S})$ can be neglected, resulting in the equation

$$\boldsymbol{0} = \Delta\boldsymbol{\delta^T P} \Delta\boldsymbol{\delta}$$

(4-14)

Any valid null motion can be written as

$$\Delta\boldsymbol{\delta} = \boldsymbol{N\kappa}$$

(4-15)

Where $\boldsymbol{N}$ is the null motion expressed in terms of a basis $\boldsymbol{N} = \text{null}(\boldsymbol{A})$ and $\boldsymbol{\kappa}$ is a column matrix of the scaling components of the null space basis vectors. Substituting (*4-15*) into (*4-14*) yields

$$\mathbf{0} = (\boldsymbol{N\kappa})^{\boldsymbol{T}}\boldsymbol{P}(\boldsymbol{N\kappa}) \tag{4-16}$$

$$\mathbf{0} = \boldsymbol{\kappa}^{\boldsymbol{T}}(\boldsymbol{N}^{\boldsymbol{T}}\boldsymbol{PN})\boldsymbol{\kappa} \tag{4-17}$$

$$\mathbf{0} = \boldsymbol{\kappa}^{\boldsymbol{T}}(\boldsymbol{Q})\boldsymbol{\kappa} \tag{4-18}$$

Where **Q** is the classification matrix defined as

$$\boldsymbol{Q} = \boldsymbol{\kappa}^{\boldsymbol{T}}\boldsymbol{P}\boldsymbol{\kappa} \tag{4-19}$$

The eigen values of the classification matrix are used to determine elliptic and hyperbolic singularity. When $\boldsymbol{Q}$ has either all positive or all negative eigenvalues, it is definite and a nonzero null vector that satisfies (*4-19*) does not exist. Thus, the matrix does not contain null space and the singularity is elliptic. On the other hand, when $\boldsymbol{Q}$ is semi-definite and has at least one zero eigenvalue, the null space exists. Thus, null motion is possible near singularity. When $\boldsymbol{Q}$ is indefinite it always admits non-zero $\boldsymbol{\kappa}$ for which (*4-19*) vanishes, by continuity the quadratic form between its positive and negative directions. Therefore, both semi-definite and indefinite cases lead to hyperbolic singularity.

Singularity can also be classified into External and Internal singularities based on the location of the singularity in the angular momentum envelope. Angular momentum envelope is a collection of singular surfaces, which are all angular momentum states that correspond to singular gimbal configurations. The total angular momentum $\boldsymbol{h}^{\boldsymbol{B}}$ at a singularity in any given singular direction $\boldsymbol{s}$ is given as

$$\boldsymbol{h}^{\boldsymbol{B}} = \sum_{i=1}^{N_{cmg}} \mathrm{h}_i\hat{\mathbf{h}}_{\boldsymbol{i}} = \sum_{i=1}^{N_{cmg}} I_w\Omega_i(\hat{\boldsymbol{\tau}}_{\boldsymbol{i}} \times \widehat{\boldsymbol{\delta}}_{\boldsymbol{i}}) \tag{4-20}$$

Where $\widehat{\boldsymbol{\delta}}_{\boldsymbol{i}}$ is a unit direction pointing along the i^{th} gimbal axis. For any singular direction $\boldsymbol{s} \neq \pm\widehat{\boldsymbol{\delta}}_{\boldsymbol{i}}$, equation (4-8) can be rewritten as

$$\boldsymbol{s}^{\boldsymbol{T}}\hat{\boldsymbol{\tau}}_{\boldsymbol{i}} = 0 \tag{4-21}$$

$$\boldsymbol{s}^{\boldsymbol{T}}\hat{\mathbf{h}}_{\boldsymbol{i}} \neq 0 \tag{4-22}$$

Where the sign parameter is defined as $\epsilon_i = \text{sign}\left(\boldsymbol{s}^{\boldsymbol{T}}\hat{\mathbf{h}}_{\boldsymbol{i}}\right)$, taking the value of +1 or -1 according to whether the spin axis of i^{th} CMG is aligned with or against the singular direction. Therefore, the torque unit direction can be defined as

$$\hat{\boldsymbol{\tau}}_{\boldsymbol{i}} = \epsilon_i \frac{(\hat{\boldsymbol{\delta}}_{\boldsymbol{i}} \times \boldsymbol{s})}{\left\|\hat{\boldsymbol{\delta}}_{\boldsymbol{i}} \times \boldsymbol{s}\right\|} \tag{4-23}$$

Substituting (*4-23*) into (*4-20*) yields

$$\boldsymbol{h^B} = \sum_{i=1}^{N} \epsilon_i \frac{(\hat{\boldsymbol{\delta}}_{\boldsymbol{i}} \times \boldsymbol{s}) \times \hat{\boldsymbol{\delta}}_{\boldsymbol{i}}}{\left\|\hat{\boldsymbol{\delta}}_{\boldsymbol{i}} \times \boldsymbol{s}\right\|} I_w \Omega_i \tag{4-24}$$

Using equation (*4-24*) the internal and external surfaces of the angular momentum envelope can be mapped on a 3-dimensional space. However, for the purpose of mapping, $I_w\Omega_i$ is assumed to have the magnitude of 1. The external singularity can be mapped by setting the sign parameter to strictly positive or strictly negative. The internal surfaces are found by using mixed sign combinations of ϵ_i. There are 2^{N-1} distinct singular surfaces that can be mapped using these mixed combinations. An example of external and internal surfaces of a pyramid configuration of HCMG with a pyramid angle of 54.74 degrees are shown in Figure 4-1 and Figure 4-2, respectively.

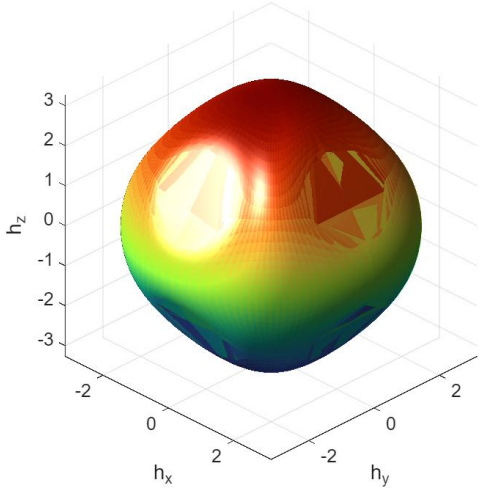

Figure 4-1: External Singular Surfaces of pyramid Configuration

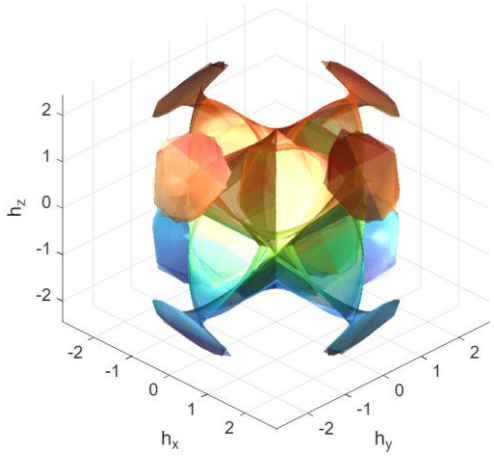

Figure 4-2: Internal Singular Surfaces of Pyramid Configuration

4.1.1) External Singularity:

External singularity is also known as saturation singularity [1]. It is associated with the maximum projection of angular momentum of the cluster. Therefore, it lies on the outer surface of the envelope, representing the physical limit of the CMG cluster ability to exchange momentum with the satellite. This usually occurs when the actuator reaches its physical limit. External singularity is always elliptic, because null motion does not exist on the singular surface of the envelope. Therefore, steering algorithms that use null motion, cannot get rid of external singularities.

4.1.2) Internal Singularity:

Internal singularity, on the other hand, lies inside the angular momentum envelope. Since it lies on the internal singularity surfaces, it can be either hyperbolic or elliptic. Hyperbolic internal singularity is characterized by the existence of null motion, which provides a continuous set of gimbal solutions that do not change the cluster's total angular momentum or torque on the spacecraft. In non-degenerate hyperbolic cases, the available null solutions allow the steering algorithm to navigate around the singularity without inducing torque errors. However, degenerate hyperbolic singularity exists where all available null paths lead only to other singular configurations, making the null motion ineffective.

On the other hand, null motion does not exist for elliptic internal singularity. In other words, there is no motion to reconfigure the gimbals while maintaining constant momentum. Unlike external singularity where the singularity is predictable as the actuator limit, elliptic internal singularity tends to appear unpredictably trapping the actuator in a gimbal lock. To escape, steering algorithms that perturb the angular momentum with torque error to escape the singular state must be used.

4.2) RW Singularity:

Most reaction wheel systems are designed to be able to induce control torque in any direction. However, HCMGs operate as a reaction wheel system after the gimbal motors are brought to a complete stop. Therefore, there may exist orientation of the gimbal axes, where singularities can be encountered. From equation (*3-18*), the generated torque when operating as RW mode is given by

$$\boldsymbol{\tau^B} = I_w \boldsymbol{B\dot{\Omega}} \tag{4-25}$$

Were $\boldsymbol{B}$ is the flywheel spin-axis distribution matrix, whose columns are the unit spin axes of each HCMG expressed in the body frame. It is a function of gimbal angles and is time varying. The torque can be mapped onto flywheel acceleration $\boldsymbol{\dot{\Omega}}$ as

$$\boldsymbol{\dot{\Omega}} = \frac{1}{I_w} \boldsymbol{B^T (BB^T)^{-1} \tau^B_{D\_w}} \tag{4-26}$$

Like CMG mode, the torque cannot be mapped onto flywheel acceleration if $\boldsymbol{BB^T}$ is singular, proving that singularity can be encountered when operating as RW mode.

5)HCMG Arrangements:

Since angular momentum envelope and singularity surfaces are functions of the Jacobian matrices which are functions of HCMG orientations, different arrangements of HCMGs will lead to different envelopes and singular surfaces. Therefore, different arrangements provide different advantages and disadvantages. The common HCMG arrangements include 1) Rooftop 2) Box 3) Scissor Pair and 4) Pyramid.

5.1) Rooftop Arrangement:

The rooftop arrangement consists of two sets of parallel HCMGs. Each set is mounted with a rooftop angle θ, which is measured from the base to the rooftop face that the torque plane lies on. The arrangement is shown in Figure 5-1. Additionally, using equation (4-22), the angular momentum envelope and its internal singular surfaces for rooftop angle of 45 degrees are shown in Figure 5-2.

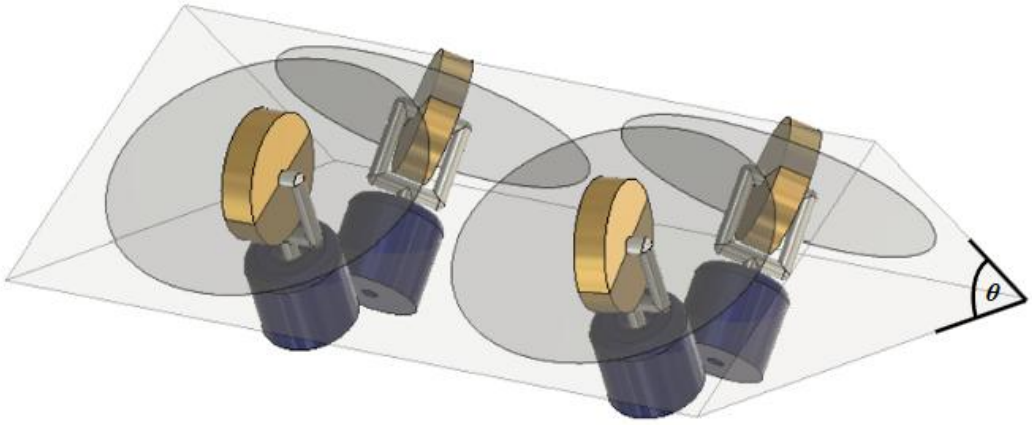

Figure 5-1: HCMG Rooftop Arrangement

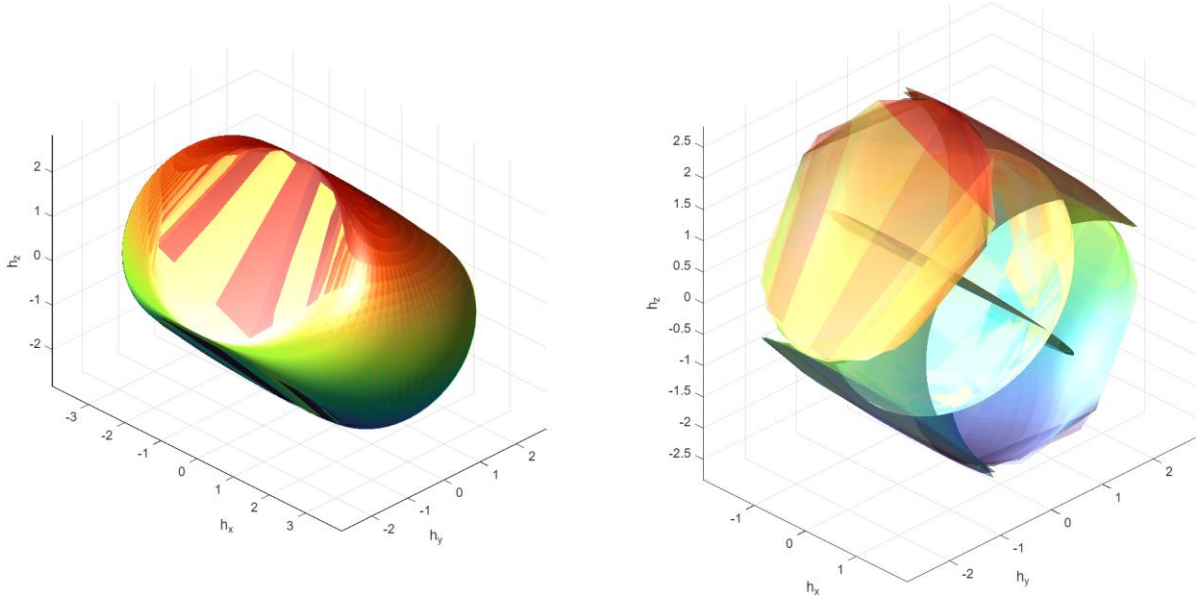

Figure 5-2: Rooftop Arrangement External and Internal Singularities

Rooftop arrangement is one of the most widely used arrangements on satellites, due to its singularity-free workspace in the center. They are also free from elliptic internal singularities [2]. However, as can be seen on Figure 5-2, the angular momentum envelop is ellipsoidal. This suggests that rooftop arrangement does not have equal control authority in all directions. They may still encounter degenerate hyperbolic singularities.

5.2) Box Arrangement:

Box arrangement consists of 4 HCMGs mounted on each side of the cube with 90 degrees between each torque plane. The arrangement is shown in Figure 5-3. The singularity surfaces are shown in Figure 5-4.

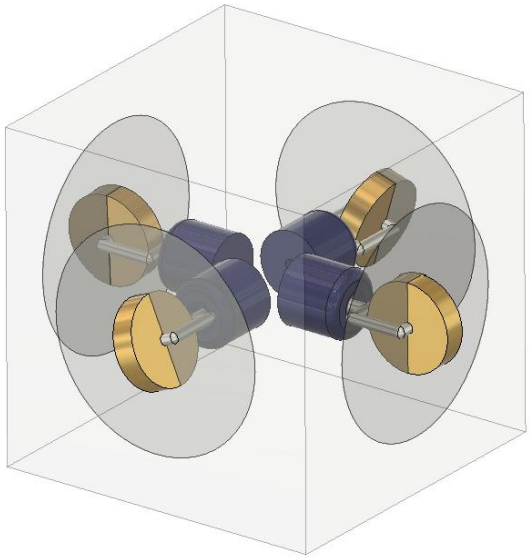

Figure 5-3: HCMG Box Arrangement

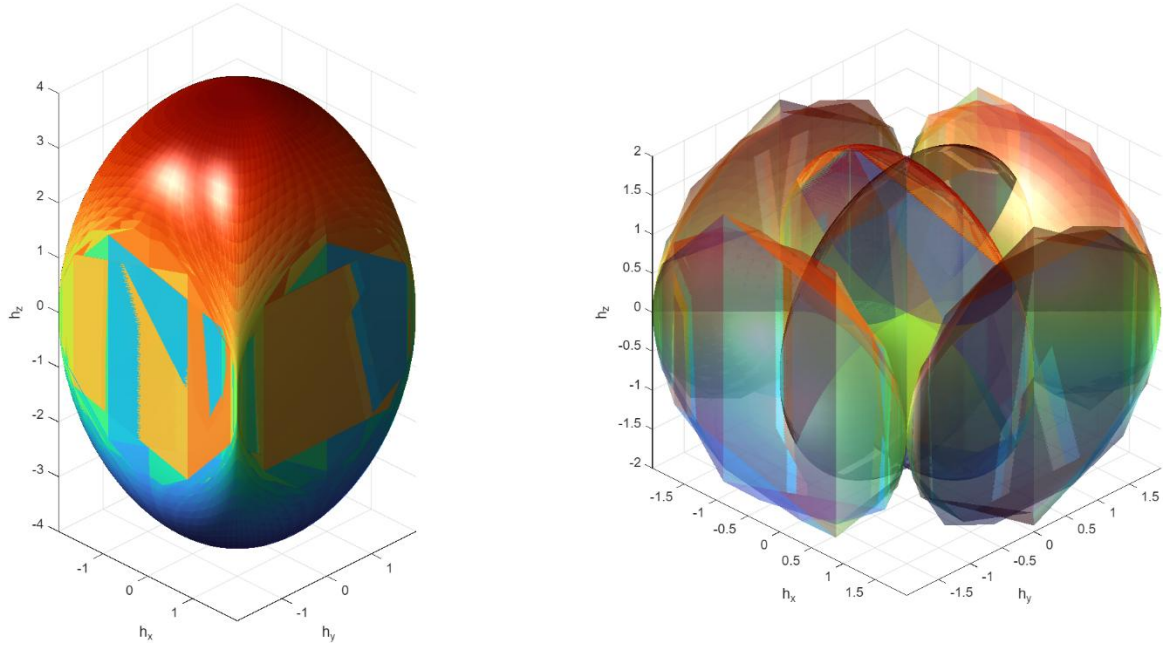

Figure 5-4: Box Arrangement External and Internal Singularities

As can be seen, the envelope also consists of a singularity-free workspace in the middle. Similar to rooftop arrangement, it is also free from elliptic internal singularities. However the angular momentum envelop is highly ellipsoidal, so there is an unequal control authority across the three axes. Since certain axes have less control authority, HCMG in these arrangements must be made larger. Thus, making them unsuitable for applications in nano satellites. Additionally, they may still encounter degenerate hyperbolic singularities.

5.3) Scissor Pair Arrangement:

Scissor pair arrangement consists of 3 pairs of parallel HCMGs. Each pair is mounted to one axis of rotation, and they operate together with kinematic constraints on gimbal angles. They are constrained in a way that their gimbals always move in the opposite direction. This ensures that the torque output always lies along a fixed axis. Due to the constraints, there are no internal singularities. This means that the arrangement has full control authority over three axes of rotation as long as it does not exceed the maximum angular momentum of the system. Since torque produced is unidirectional, it is common among space robotics applications. The arrangement is as shown in Figure 5-5.

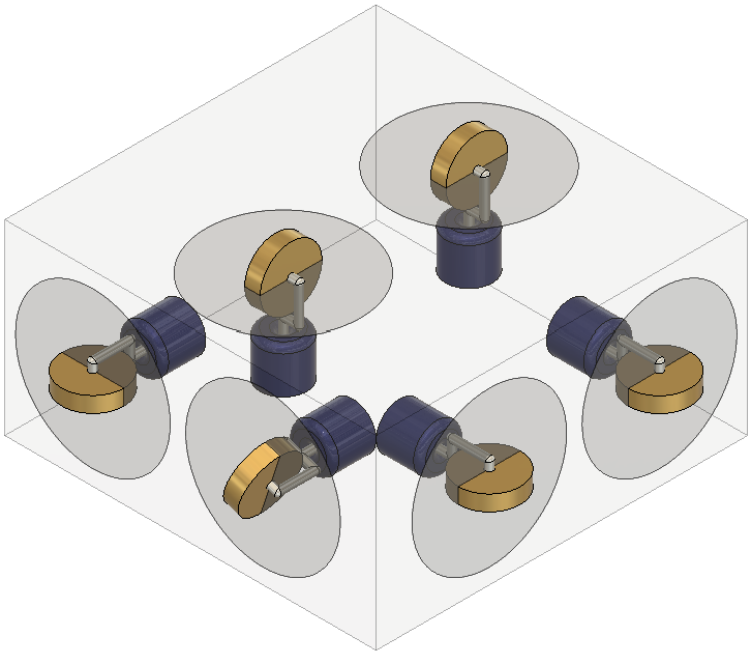

Figure 5-5: HCMG Scissor Pair Arrangement

5.4) Pyramid Arrangement:

Pyramid arrangement consists of four CMGs arranged in a way that their torque planes form the sides of a pyramid. The pyramid angle θ dictates the angle between the bottom face and each torque plane. The arrangement is shown in Figure 5.6 and the singularity surfaces of a pyramid with an angle of 54.74 degrees is shown in Figure 5-7.

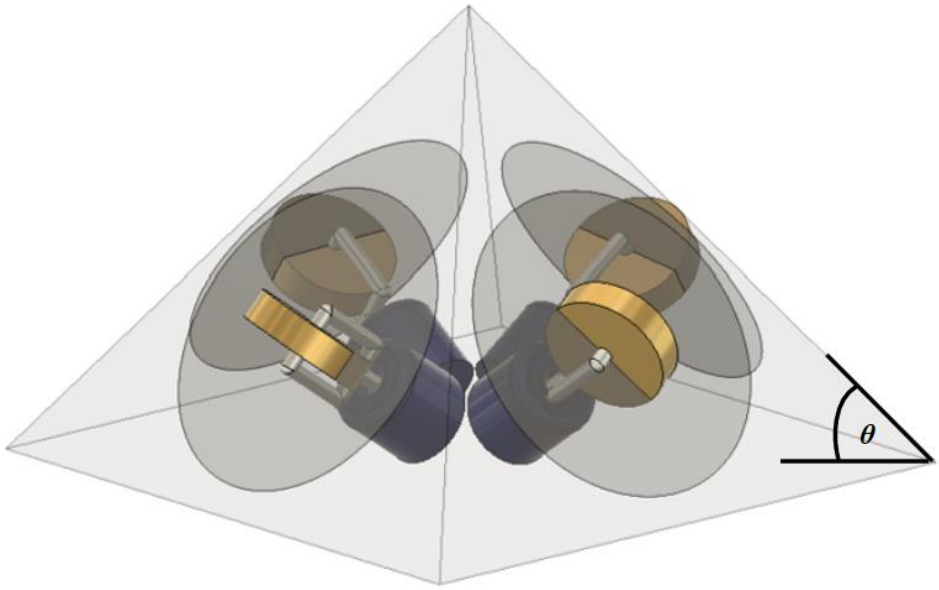

Figure 5-6: HCMG Pyramid Arrangement

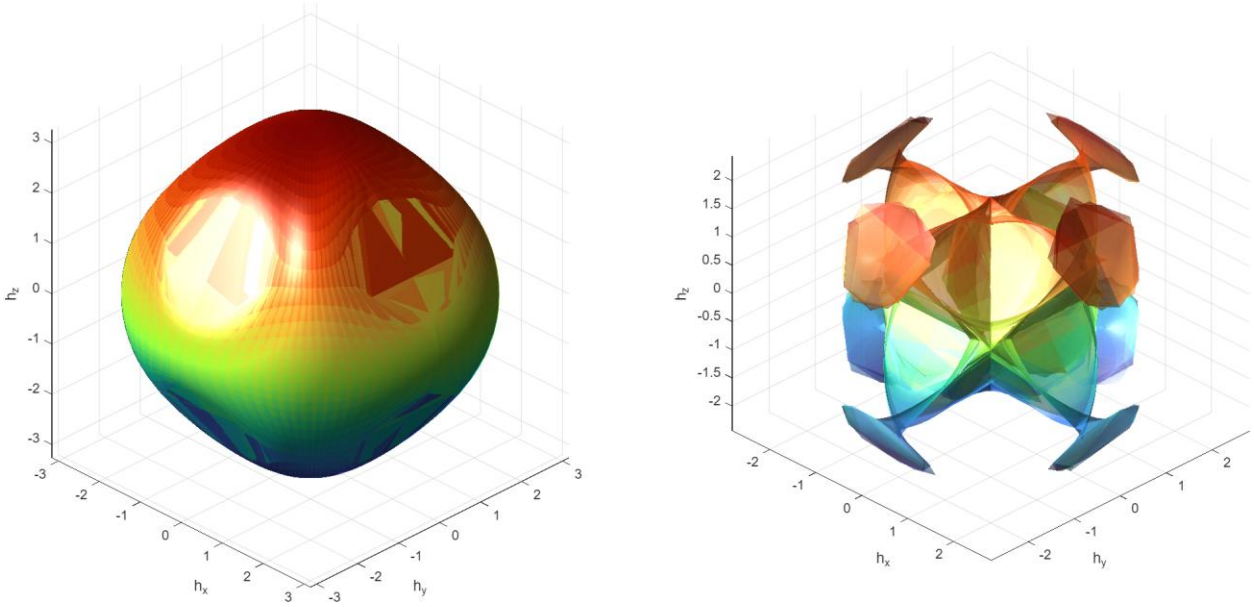

Figure 5-7: Pyramid Arrangement External and Internal Singularities

As can be seen, a pyramid with the pyramid angle of 54.74 has a near spherical angular momentum envelope. The relative size of the envelope is also large compared to those of rooftop and box arrangements. This makes pyramid arrangement common among nanosatellite applications, because it can provide more torque across three axes without needing the actuator to be oversized. However, its singularity free workspace in the center is smaller than that of rooftop and box arrangement. It is therefore more prone to encountering elliptic internal singularities, requiring more complex steering algorithms to escape from these singularities.

6) HCMG Steering Algorithm:

Steering algorithms are used to either avoid or escape from internal singularities. Singularity avoidance algorithms attempt to keep the Jacobian matrix from becoming rank deficient by reconfiguring the gimbals before a singularity is reached [2]. These algorithms can be classified into constrained steering algorithms and null motion algorithms. On the other hand, singularity escape algorithms are used when a system is already trapped in a singularity. Torque error algorithms are classified as singularity escape algorithms.

6.1) Singularity Avoidance Algorithms

6.1.1) Constrained Steering Algorithms

Constrained steering algorithms work by constraining gimbal angles or usable angular momentum to prevent gimbals from entering singular configuration. These algorithms utilize prior knowledge of locations of singular surfaces. Examples of constrained steering algorithms include the scissor pair arrangement where a pair of CMGs produce torque only along a fixed direction and to achieve three directions of torque, three pairs are used. Although, these algorithms operate freely without singularities, they are not widely used due to the reduced availability in workspace from the constraints. Thus, making the method only common among larger satellites, where CMGs can be designed to be oversized.

6.1.2) Null Motion Algorithms

Null motion steering algorithms reconfigure the gimbals to a more favorable position, when the gimbals are near singular configurations by adding null motion. This method is useful because it allows the gimbals to reconfigure without producing net torque or changing total angular momentum of the satellite. Null motion algorithms can be classified into algorithms such as 1) Local Gradient 2) Global Avoidance or Preferred Trajectory Tracking 3) Generalized Inverse Steering Law and many more.

6.1.2.1) Local Gradient (LG)

Local gradient prevents the Jacobian matrix from becoming singular by choosing the null vector $\boldsymbol{d}$ that maximizes an objective function that relates the distance from singularity. These objective functions include Jacobian matrix condition number, smallest singular value, or the singularity measure m_S

$$m_S = \sqrt{\det(\boldsymbol{A}\boldsymbol{A}^{\boldsymbol{T}})} \tag{6-1}$$

Where the null vector can be calculated from

$$\boldsymbol{d} = \nabla_{\delta} f = \frac{\partial f}{\partial}\left(\frac{\partial m_S}{\partial \boldsymbol{\delta}}\right)^T = \frac{-1}{{m_S}^2}\left(\frac{\partial m_S}{\partial \boldsymbol{\delta}}\right)^T \tag{6-2}$$

Where f is the objective function defined as $f = \frac{1}{m_S}$. Minimizing the objective function will maximize the distance from singularity by maximizing m_S. The steering algorithm using LG has the following form

$$\dot{\boldsymbol{\delta}} = \frac{1}{I_w}\widehat{\boldsymbol{\Omega}}^{-1}\boldsymbol{A}^T(\boldsymbol{A}\boldsymbol{A}^T)^{-1}\boldsymbol{\tau}_{gyro}^B + \lambda \boldsymbol{d} \tag{6-3}$$

Where λ is the singularity parameter defined as

$$\lambda = \lambda_0 e^{-\mu {m_S}^2} \tag{6-4}$$

The parameter measures how close the system is to the CMG singularity. The constants λ_0 and μ must be tuned such that the parameter is large near singularity and is negligible away from singularity.

6.1.2.2) Preferred Trajectory Tracking

Preferred trajectory tracking is a global method that calculates the alternate nonsingular gimbal trajectories separately, before choosing the null motion vector that steer the gimbals to those configurations. This is done by minimizing the gimbal angle error $(\boldsymbol{\delta} - \boldsymbol{\delta}^*)$. Using this method, the gimbal rate can be written as

$$\dot{\boldsymbol{\delta}} = \frac{1}{I_w}\widehat{\boldsymbol{\Omega}}^{-1}\boldsymbol{A}^T(\boldsymbol{A}\boldsymbol{A}^T)^{-1}\boldsymbol{\tau}_{gyro}^B + \lambda[\boldsymbol{1} - \boldsymbol{A}^+\boldsymbol{A}](\boldsymbol{\delta} - \boldsymbol{\delta}^*) \tag{6-5}$$

$$\dot{\boldsymbol{\delta}} = \frac{1}{I_w}\widehat{\boldsymbol{\Omega}}^{-1}\boldsymbol{A}^T(\boldsymbol{A}\boldsymbol{A}^T)^{-1}\boldsymbol{\tau}_{gyro}^B + \lambda[\boldsymbol{1} - \boldsymbol{A}^T(\boldsymbol{A}\boldsymbol{A}^T)^{-1}\boldsymbol{A}](\boldsymbol{\delta} - \boldsymbol{\delta}^*) \tag{6-6}$$

Where $\boldsymbol{\delta}^*$ are the preferred trajectories and $\boldsymbol{A}^+$ is the Moore-Penrose pseudoinverse of the Jacobian matrix. Since the method pre-calculates the preferred trajectory, it is not suitable for real-time applications.

6.1.2.3) Generalized Inverse Steering Law (GISL)

Generalized inverse steering law indirectly add null motion to the gimbal rate commands through a modified pseudo-inverse. Unlike standard null motion algorithms that explicitly add a null vector to the minimum-norm solution, GISL integrates the reconfiguration motion into the mapping itself. This is achieved by defining auxiliary Jacobian matrix, $\boldsymbol{B}$, whose columns are chosen to be orthogonal to the corresponding columns of the gyroscopic Jacobian matrix $\boldsymbol{A}$. For CMGs, the matrix representing the flywheel spin axes naturally satisfies this orthogonality condition because the torque axis is always perpendicular to the spin axis. Thus, the matrix $\boldsymbol{B}$ in equation (*3-13*) can be used as the auxiliary Jacobian matrix. The modified pseudo-inverse can be written as

$$\boldsymbol{A}^{GISL} = (\boldsymbol{A} + \boldsymbol{B})^T(\boldsymbol{A}(\boldsymbol{A} + \boldsymbol{B})^T)^{-1} \tag{6-7}$$

Using the modified pseudoinverse, the steering algorithm becomes

$$\dot{\boldsymbol{\delta}} = \frac{1}{I_w}\widehat{\boldsymbol{\Omega}}^{-1}\boldsymbol{A}^{GISL}\boldsymbol{\tau}_{gyro}^{B} \tag{6-8}$$

The modified inverse reproduces exact commanded torque, since $\boldsymbol{A}\boldsymbol{A}^{GISL} = \boldsymbol{A}(\boldsymbol{A} + \boldsymbol{B})^{\boldsymbol{T}}[\boldsymbol{A}(\boldsymbol{A} + \boldsymbol{B})^{\boldsymbol{T}}]^{-1} = \boldsymbol{1}$. The reconfiguration motion introduced through $\boldsymbol{B}$ therefore generates no torque error by construction. This allows the gimbals to reconfigure toward more favorable orientations while still meeting the primary torque command with zero torque error. However, GISL along with LG and preferred trajectory tracking are all unable to avoid and escape elliptic internal singularities, once trapped in it. Therefore, null motion algorithms such as GISL are typically used in environments where the momentum envelope is constrained to avoid elliptic traps or is combined with other robust singularity escape algorithms.

6.2) Singularity Escape Algorithms

Singularity escape algorithms are also known as pseudoinverse solutions. They function by adding torque error to escape internal singularity without taking into consideration the type of internal singularity. There exist many singularity escape algorithms, however the most well-known ones include Singularity Robust Inverse and Generalize Singularity Robust Inverse.

6.2.1) Singularity Robust Inverse (SR)

Singularity robust inverse adds torque error by adding a positive definite matrix, $\lambda\boldsymbol{1}$ to the product of the Jacobian matrix $\boldsymbol{A}\boldsymbol{A}^T$ before it is inverted. Where λ is the singularity parameter defined in (*6-4*). The formulation is expressed as

$$\boldsymbol{A^{SR} = A^T(AA^T + \lambda 1)^{-1}}$$

(6-9)

The gimbal rates become

$$\dot{\boldsymbol{\delta}} = \frac{1}{I_w}\widehat{\boldsymbol{\Omega}}^{-1}\boldsymbol{A^{SR}\tau_{gyro}^{B}}$$

(6-10)

By adding an identity matrix scaled by the singularity parameter, the product $\boldsymbol{AA^{SR}}$ does not equal identity. As a result, the actual torque produced by the gimbals $\boldsymbol{A^{SR}(\dot{\delta})}$ is mathematically different from the torque requested by the attitude controller. Thus, adding torque error to escape from both hyperbolic and elliptic internal singularities. However, near singularities the singularity parameter becomes exponentially large. As the torque error is directly proportional to the parameter, the torque generated may affect satellite's pointing precision. Additionally, SR inverse is ineffective when trapped in gimbal lock configuration. Gimbal lock is when the output torque is along the singular direction. In this configuration, SR calculates a gimbal rate of zero, leading to no torque and the system remains in the gimbal lock configuration.

6.2.2) Generalized Singularity Robust Inverse (GSR)

Generalized singularity robust inverse was introduced to escape gimbal lock. It can escape gimbal lock by replacing the identity matrix with a matrix with time varying off diagonal elements to ensure that the torque error is time varying. The GSR can be written in the following form

$$\boldsymbol{A^{GSR} = A^T(AA^T + \lambda E)^{-1}}$$

(6-11)

Where $\boldsymbol{E}$ is defined as

$$E = \begin{bmatrix} 1 & e_1 & e_2 \\ e_1 & 1 & e_3 \\ e_2 & e_3 & 1 \end{bmatrix}$$

(6-12)

Where the off diagonal elements e_i where $i = 1,2,3$ can be defined as

$$e_i = e_0 \sin(\omega_i t + \phi_i)$$

(6-13)

Where ω is the frequency and ϕ is the phase shift. Thus, the gimbal rate becomes

$$\dot{\boldsymbol{\delta}} = \frac{1}{I_w}\widehat{\boldsymbol{\Omega}}^{-1}\boldsymbol{A}^{GSR}\boldsymbol{\tau}_{gyro}^{B} \tag{6-14}$$

6.3) Hybrid Steering Algorithm

The hybrid steering algorithm is proposed by Patankar to allow for HCMGs to switch between CMG and RW mode. The algorithm is a combination of GSR torque error algorithm and null motion algorithm. When operating as CMG mode, GSR is used to induce torque error to escape from singularities. Additionally, during CMG mode, null motion is used to drive the gimbals in an orientation that maximizes S_{RW} before switching RW mode. S_{RW} is the singularity parameter of flywheels, which indicates how close or far away the system is from RW singularity. When the parameter is small, the system is close to singularity, vice versa. It is defined as

$$S_{RW} = \det(\boldsymbol{B}\boldsymbol{B}^{\boldsymbol{T}}) \tag{6-13}$$

Similarly, the parameter can also be defined for CMG singularity.

$$S_{CMG} = {m_s}^2 = \det(\boldsymbol{A}\boldsymbol{A}^{\boldsymbol{T}}) \tag{6-14}$$

The commanded torque can be mapped onto gimbal rates and flywheel acceleration as shown

$$\dot{\boldsymbol{\delta}} = \alpha(I_w\widehat{\boldsymbol{\Omega}})^{-1}\left[\boldsymbol{A}^T(\boldsymbol{A}\boldsymbol{A}^T + \lambda\boldsymbol{E})^{-1}\boldsymbol{\tau}_{gyro}^{B} + \beta\boldsymbol{d}_{\boldsymbol{RW}}\right] \tag{6-15}$$

$$\dot{\boldsymbol{\Omega}} = (1-\alpha)\frac{1}{I_w}\boldsymbol{B}^{\boldsymbol{T}}(\boldsymbol{B}\boldsymbol{B}^{\boldsymbol{T}})^{-1}\boldsymbol{\tau}_{\boldsymbol{D\_w}}^{\boldsymbol{B}} \tag{6-16}$$

Where α is the mode switch parameter. It is a parameter ranging between 0 and 1 and is chosen externally. When α is 1, HCMG operates as SGCMG. When α is 0, the gimbal axes are locked in place, and the system operates purely as a reaction wheel system. λ and β are functions of singularity parameters defined as

$$\lambda = \lambda_0 e^{-\mu_1 S_{CMG}} \tag{6-17}$$

$$\beta = \beta_0 e^{-\mu_2 S_{RW}} \tag{6-18}$$

Where λ_0 and β_0 dictate the maximum values of λ and β. Additionally, μ_1 and μ_2 dictate how fast the error falls off as the system moves away from CMG or RW singularity. The vector $\boldsymbol{d_{RW}}$ is, however, not the same vector as described in local gradient algorithm (*6-2*). The vector $\boldsymbol{d}$ from (*6-2*) steer the gimbals in a direction that maximizes S_{CMG} to keep the cluster from locking up during a slew. However, $\boldsymbol{d_{RW}}$ is the normalized null vector that maximizes S_{RW}. This ensures the flywheels have triaxial control authority once the gimbals are locked for RW phase.

For a four CMG system, the null vector $\boldsymbol{N}$ is

$$\boldsymbol{N} = \pm \begin{bmatrix} +|\boldsymbol{A}_1| \\ -|\boldsymbol{A}_2| \\ +|\boldsymbol{A}_3| \\ -|\boldsymbol{A}_4| \end{bmatrix} \tag{6-19}$$

Where $\boldsymbol{A}_i$ is the Jacobian matrix $\boldsymbol{A}$ with its i^{th} column removed. $\boldsymbol{d_{RW}}$ is obtained by normalizing $\boldsymbol{N}$ and choosing an appropriate sign that improve S_{RW}. Hybrid steering algorithm can escape internal singularities while also driving the gimbals toward a more favorable orientation for RW phase. However, there are several limitations that the algorithm cannot handle.

One of the most critical limitations is flywheel speed drift. Because the precision pointing phase uses direct flywheel motor torque to control the spacecraft, the flywheel speeds $\boldsymbol{\Omega}$ are constantly modulated. Multiple maneuvers cause these speeds to drift significantly from their nominal values, as the algorithm does not guarantee that wheel speeds will return to their starting points after reorientation. This could potentially lead to a loss of control. If a flywheel reaches its maximum speed, control authority in the RW mode is lost because the motor can no longer accelerate the wheel to provide reaction torque. Conversely, if flywheel speeds become too low, the gyroscopic torque produced during the CMG mode is also small. Lastly, the angular momentum envelope and the location of internal singular surfaces are functions of the individual flywheel speeds. As the speeds drift, the angular momentum envelope may change shape and size, which can limit the maximum slew rates in certain directions.

Because the flywheel speed drift is cumulative over sequential missions, the system will eventually reach a state of saturation or insufficient gyroscopic authority. This makes angular momentum management, the process of resetting flywheel speeds to nominal values necessary to maintain the long-term operational capability of HCMG.

7) Common Disturbances in Space:

The major causes that lead to excess momentum building up over time are disturbances. Disturbances can be both forces that lead to the change in orbit of the satellite and torques that

cause the satellite to rotate. They are usually caused by the environment and interactions between different bodies in space. Common disturbances include gravity gradient, Solar radiation pressure, aerodynamic drag and torque, and J2 Perturbation.

7.1) Gravity Gradient Torque

Gravity gradient torque is the torque caused by uneven gravitational pull on the body of a non-spherical object [3]. Let the vector $\boldsymbol{R}$ denote the displacement from center of the Earth to the center of mass (CM) of the satellite. Considering infinitesimal mass element dm located at a position $\boldsymbol{\varsigma}$ relative to the CM. The gravitational force on this element is

$$d\boldsymbol{F} = -\frac{\mu dm}{|\boldsymbol{R}+\boldsymbol{\varsigma}|^3}(\boldsymbol{R}+\boldsymbol{\varsigma}) \tag{7-1}$$

Where μ is Earth's standard gravitational parameter. The torque produced by this force about CM is therefore

$$d\boldsymbol{\tau_{gg}} = \boldsymbol{\varsigma} \times d\boldsymbol{F} \tag{7-2}$$

Integrating (7-2) yields

$$\boldsymbol{\tau_{gg}} = -\mu \int \frac{\rho \times \boldsymbol{R}}{|\boldsymbol{R}+\boldsymbol{\varsigma}|^3} dm \tag{7-3}$$

Assuming the dimensions of the spacecraft are negligible compared to its orbital radius. The denominator from (*7-3*) can be approximated as

$$|\boldsymbol{R}+\boldsymbol{\rho}|^{-3} \approx R^{-3}\left(1-3\frac{\mathbf{R}\cdot\boldsymbol{\varsigma}}{R^2}\right) \tag{7-4}$$

Substituting back into (7-2) and using the definition of the moment of inertia tensor $\boldsymbol{I}$, the gravity gradient torque can be written as

$$\boldsymbol{\tau_{gg}} = \frac{3\mu}{\|\boldsymbol{R}\|^5}\boldsymbol{R} \times (\boldsymbol{I}\cdot\boldsymbol{R}) \tag{7-5}$$

7.2) Aerodynamic Drag and Torque

Satellites that operate in Low Earth Orbit (LEO) experience the force of air resistance that opposes their motion, known as aerodynamic drag [3]. For a satellite with N flat plates, representing each of its faces, with A_{face_i} representing the area of the ith face and $\hat{\boldsymbol{n}}_{\boldsymbol{i}}$ representing its outward-pointing unit normal vector in the Body frame. Assuming a drag-only interaction model where the force only acts along the wind vector, the aerodynamic force acting on each face of the satellite in satellite body fixed frame is

$$\boldsymbol{F}^{\boldsymbol{B}}_{\boldsymbol{aero\_i}} = \frac{1}{2}\rho C_D A_{face_i} \left\|\boldsymbol{v}^{\boldsymbol{2}}_{\boldsymbol{body}}\right\| \max(0, \hat{\boldsymbol{n}}_{\boldsymbol{i}} \cdot \hat{\boldsymbol{v}})\hat{\boldsymbol{v}} \tag{7-6}$$

Where ρ is the atmospheric density at that specific altitude, C_D is flat plate drag coefficient, $\boldsymbol{v}_{\boldsymbol{body}}$ is the orbital velocity of the satellite expressed in body frame, and $\hat{\boldsymbol{v}}$ is the unit vector that opposes the wind defined as

$$\hat{\boldsymbol{v}} = -\frac{\boldsymbol{v}_{\boldsymbol{body}}}{\left\|\boldsymbol{v}_{\boldsymbol{body}}\right\|} \tag{7-7}$$

The max(_) operator in equation (*7-6*) enforces the physical constraint of geometric shadowing, ensuring that faces turn away from the oncoming flow experience zero force. The total aerodynamic drag force on the satellite is the sum of all the force each face experiences.

$$\boldsymbol{F}^{\boldsymbol{B}}_{\boldsymbol{aero}} = \sum_{i=1}^{N_{face}} \boldsymbol{F}^{\boldsymbol{B}}_{\boldsymbol{aero\_i}} \tag{7-8}$$

Assuming the mass of the satellite is m_{sat}, the satellite acceleration due to the drag force is therefore

$$\boldsymbol{a}^{\boldsymbol{ECI}}_{\boldsymbol{aero}} = \boldsymbol{R}^{\boldsymbol{ECI}}_{\boldsymbol{B}} \left(\frac{\boldsymbol{F}^{\boldsymbol{B}}_{\boldsymbol{aero}}}{m_{sat}}\right) \tag{7-9}$$

Often times the satellite's COM does not align with the center of pressure (COP) of each face. This allows the drag force to generate torque about COM known as aerodynamic torque. Let $\boldsymbol{r}_{\boldsymbol{cp_i}} = \boldsymbol{r}_{\boldsymbol{COP_i}} - \boldsymbol{r}_{\boldsymbol{COM}}$ be the position vector from the COM to the COP of face i. The cumulative aerodynamic torque in the Body frame can be expressed as

$$\boldsymbol{\tau}_{\boldsymbol{aero}}^{\boldsymbol{B}} = \sum_{i=1}^{N_{face}} \left(\boldsymbol{r}_{\boldsymbol{cp}_i} \times \boldsymbol{F}_{\boldsymbol{aero}_i}^{\boldsymbol{B}}\right)$$

(7-10)

7.3) J2 Perturbation

J2 Perturbation arises due to Earth's oblateness . The oblateness causes a secular drift in the Right Ascension of Ascending Node and Argument of Perigee. The acceleration of the satellite due to J2 is given by

$$\boldsymbol{a}_{J2} = -\frac{3\mu J_2 R_E^2}{2{r^{ECI}}^5}\begin{bmatrix} x\left(1 - 5\dfrac{z^2}{{r^{ECI}}^2}\right) \\ y\left(1 - 5\dfrac{z^2}{{r^{ECI}}^2}\right) \\ z\left(3 - 5\dfrac{z^2}{{r^{ECI}}^2}\right) \end{bmatrix}$$

(7-11)

Where J_2 is the perturbing coefficient, which is equal 1.0826×10^{-3} according to the World Geodetic System model in 1984 [Sidi]. μ is Earth's gravitational parameter. R_E is Earth's equatorial radius. Lastly, $\boldsymbol{r}^{\boldsymbol{ECI}}$ is the satellite's position expressed in ECI frame given as

$$\boldsymbol{r}^{\boldsymbol{ECI}} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

(7-12)

7.4) Solar Radiation Pressure

The Sun emits sunlight as a form of electromagnetic radiation through photons. These photons carry momentum that may exert pressure on bodies in space [4]. The magnitude of solar pressure is based on the mean solar energy flux, S and the speed of light c [5].

$$P_{srp} = \frac{S}{c}$$

(7-13)

At the distance of 1 AU, S is 1358 $\mathrm{W/m^2}$ [5]. Therefore, the mean momentum flux due to photons is 4.4×10^{-6} $\mathrm{kg/m \cdot s^2}$. Since the satellite does not always directly face the direction of the momentum flux, the angle between the satellite's body and flux direction must be considered.

Using the same flat plate model of satellite as before, for a panel i with area A_{face_i} and the panel unit normal vector $\hat{n}_i$ the force due to solar radiation pressure is

$$\boldsymbol{F^B_{srp\_i}} = -P_{srp}A_{face_i}(\hat{\boldsymbol{s}} \cdot \hat{\boldsymbol{n}}_{\boldsymbol{i}})[1-\varepsilon]\hat{\boldsymbol{s}} + \mathbf{2}\varepsilon(\hat{\boldsymbol{s}} \cdot \hat{\boldsymbol{n}}_{\boldsymbol{i}})\hat{\boldsymbol{n}}_{\boldsymbol{i}}, \qquad \hat{\boldsymbol{s}} \cdot \hat{\boldsymbol{n}}_{\boldsymbol{i}} > 0 \tag{7-14}$$

Where $\hat{\boldsymbol{s}}$ is the flux direction vector expressed in body frame and ε is the reflectivity coefficient. The value of ε lies between 0 and 1. For a perfectly absorbing material $\varepsilon = 0$, while for a perfectly reflecting material $\varepsilon = 1$. The sum of the forces acting on all panels is the total force due to solar radiation pressure. Additionally, the acceleration of satellites due to SRP can be found by dividing by satellite's mass.

$$\boldsymbol{F^B_{srp}} = \sum_{i=1}^{N_{face}} \boldsymbol{F^B_{srp\_i}} \tag{7-15}$$

$$\boldsymbol{a^{ECI}_{srp}} = \boldsymbol{R^{ECI}_B}\left(\frac{\boldsymbol{F^B_{srp}}}{m_{sat}}\right) \tag{7-16}$$

Similar to aerodynamic disturbances, when the satellite's center of mass is offset from the geometric center, the torque is generated. Thus, the total torque due to solar radiation pressure is given as

$$\boldsymbol{\tau^B_{srp}} = \sum_{i=1}^{N_{face}} \left(\boldsymbol{r_{cp_i}} \times \boldsymbol{F^B_{srp_i}}\right) \tag{7-17}$$

Although SRP is a common disturbance in space, its contribution to the overall disturbance is small in LEO orbit. Thus, it will not be modeled as part of this thesis.

8) Momentum Management Methods:

Over the years, many methods were introduced to manage angular momentum of both CMG and RW systems, which also apply to HCMGs. These methods were introduced to help prevent actuator saturation and prolong the lifetime of those actuators. The methods can be divided into passive and active management. Passive methods utilize the environment to slow down the rate that excess momentum builds up, while active methods utilize actuators to generate torques that unload the excess momentum.

8.1) Passive Momentum Management Methods

8.1.1) Gravity Gradient Stabilization

Gravity gradient stabilization utilizes gravity gradient torque to stabilize the satellite [3]. Based on (7-5), the satellite will always try to orient its long axis along the nadir direction. By commanding the spacecraft into a specific, slightly off-nominal attitude, a gradient torque can be strategically induced to oppose the momentum vector accumulated in the wheels. This method was used in Skylab space station, which was crewed from 1973 to 1974 and remained in orbit until 1979 [6]. During the day Skylab operates as normal, but switches to momentum dumping phase to dump excess momentum from CMGs at night utilizing the gravity gradient torque. However, this method depends heavily on the spatial distribution of mass. For a compact 1U, 3U, or even 6U CubeSat, the natural gravity gradient torque is small and may not be as effect as other methods for momentum management.

8.1.2) Solar Radiation Pressure (SRP)

Solar radiation pressure is the force exerted on an exposed surface of the spacecrafts by photons. The force depends on spacecraft's size, shape, surface reflectivity, and its angle between its surfaces and the Sun. SRP can be applied for angular momentum management by strategically utilizing the torques it generates about a satellite's center of mass. SRP has been used for angular momentum management and attitude control for a long time. In 1964, Mariner 4 was the first satellite to utilize solar vanes, which were four adjustable, reflective surfaces attached to the tips of the solar panels to offset SRP unbalance for attitude control [7]. Hayabusa 1 and 2 launched in 2007 and 2017 also proved that full attitude control solely based on SRP were possible. The benefit of SRP was extended to angular momentum management in 2018 when PROCYON successfully operate for two years without the need for RCS desaturation [8]. PROCYON introduced a new method by only selecting a single inertially fixed attitude with a bias-momentum state to reduce the control demand. In 2025, a new method utilizing Solar Array Paddle (SAP) was proven to be able to perform both attitude control and angular momentum management on EQUULEUS [7]. These missions proved SRP as an effective method with little to no fuel consumption for momentum management. However, this strategy is not efficient for nanosatellites in LEO orbits due to atmospheric drag being a bigger disturbance.

8.1.3) Aerodynamic Disturbances

Aerodynamic torque can be used to deteriorate the excess momentum in the flywheel. The Satellite for Orbital Aerodynamic Research (SOAR) flown in 2021 in VLEO orbit under the European Discoverer project, used steerable aerodynamic control surfaces such as fins, flaps, or panels to generate the external torques that oppose the accumulated momentum in the actuator cluster [9]. Although the method has been proven to effectively desaturate CMG and RW clusters, the primary drawback is that generating control torques often requires high-drag

configurations. Moving panels into the flow increases the drag area, which accelerates orbital decay and shortens the satellite's lifespan. Additionally, aerodynamic torques are highly sensitive to atmospheric density fluctuations, which are difficult to predict.

8.2) Active Momentum Management Methods

8.2.1) Flywheel Angular Momentum Redistribution

Flywheel angular momentum redistribution is a method proposed by Patankar specifically for HCMG. The method resets flywheel back to nominal speed without perturbing satellite's attitude using gyroscopic compensation torque [1]. The process starts by performing attitude stabilization in CMG mode, meaning gimbals are used to maintain the current altitude instead of the flywheels. Then the flywheels are commanded with a specific acceleration profile that drives them back to the nominal speed. The reaction torque from flywheel acceleration is treated as a disturbance torque. The controller will simultaneously drive the gimbals to produce an equal and opposite gyroscopic torque to compensate for the reaction torque, keeping satellite's attitude stable.

While this method was proven to successfully reset the flywheel back to nominal speed, it might not work well for real satellites in orbit. Patankar's method assumed a closed system redistribution of momentum with no external disturbances. These disturbances are difficult to predict for the controller to command a gyroscopic compensation torque that does not affect satellite's overall attitude stability. Additionally, these disturbances are the main cause of excess momentum building up in the system over many orbits. A redistribution method is only a temporary solution, to constantly dump these excess momentum Reaction control systems (RCS) or Magnetorquers (MTQ) must be used.

8.2.2) Reaction Control Systems

Reaction control system utilizes mass expulsion from thrusters to generate both translational forces and rotational torques [10]. They can be categorized into 4 categories based on their firing mechanisms. 1) Cold gas thruster generates thrust through the expansion of stored, pressurized gas such as Nitrogen. Since it does not involve combustion, the specific impulse is low. 2) Monopropellant chemical thruster generates torque through exothermic chemical decomposition of a single liquid propellant usually toxic hydrazine (N_2H_4), or LMP-103S. 3) Bipropellant chemical thruster combines liquid fuel with liquid oxidizer such as Nitrogen Tetroxide, N_2O_4 [11]. They are typically hypergolic, meaning they ignite spontaneously upon contact, providing intense, high-thrust capabilities. 4) Electric propulsion uses electric current to ionize and accelerate a propellant. Although it has flown in large spacecraft since 1960, recent miniaturization has made it more viable for CubeSat. The thrust generated is low, but it is fuel efficient making it suitable for long operation in space.

RCS has a long flight heritage used for attitude control, orbit maneuvers, and momentum management. However, they are not widely used among nanosatellites due to mass, volume, and power limitations. Although electric propulsion systems have been introduced in recent years to mainly tackle these limitations, they still rely on propellant that could run out over time. Additionally, Small-scale propulsion systems are noted to be prone to anomalies, requiring complex robust selection algorithms to ensure the satellite remains controllable if a thruster fails during a maneuver.

8.2.3) Magnetorquers

Magnetorquer (MTQ) is an actuator that generates torquer by interacting with Earth's magnetic field [12]. MTQ may come in three different forms Torque rods, Air-Core Coils, and Embedded PCB Coils, which are used in different satellite scales. Torque rods consist of insulated copper wire wound along a long ferromagnetic core. It provides high magnetic output for minimal volume, making it the most common type of MTQ. Air-Core coils are open loops of wound wires with no core. They are immune to magnetic saturation and hysteresis effects but require larger power to achieve the same output as a torque rod. Embedded PCB coils are coils of electromagnetic loops etched directly into the PCB. They are common among highly integrated small satellites.

For a given MTQ, the magnitude of magnetic dipole m moment is given as

$$m = N_{turn} I A_{coil} \tag{8-1}$$

Where N_{turn} is the number of turns in the coil, I is the input current, and A_{coil} is the cross-sectional area of each coil loop. Three MTQs are usually used to have a full attitude control over 3 axes. They are usually aligned with the principal axes of satellite's body frame (x, y, z). Therefore, the total dipole moment becomes

$$\boldsymbol{m} = \begin{bmatrix} m_x \\ m_y \\ m_z \end{bmatrix} \tag{8-2}$$

Net torque acting on the satellite is the cross product of the commanded dipole moment and Earth's local magnetic field strength expressed in satellite's body frame $\boldsymbol{b^B}$.

$$\boldsymbol{\tau_{MTQ}^B} = \boldsymbol{m} \times \boldsymbol{b^B} \tag{8-3}$$

Since the torque is based on the cross-product relationship, MTQ cannot generate torque parallel to the local geomagnetic field line. Thus, the control authority is strictly constrained to a 2D

plane perpendicular to $\boldsymbol{b^B}$. To continuously, dump excess momentum in the flywheels the Cross-Product Desaturation Law is used.

Let $\boldsymbol{h_{ex}^B}$ be the total excess momentum of flywheel in a HCMG cluster expressed in body frame defined as

$$\boldsymbol{h_{ex}^B = h_w^B - h_{nom}^B} \tag{8-4}$$

Where $\boldsymbol{h_w^B}$ is the flywheel total angular momentum in body frame and $\boldsymbol{h_{nom}^B}$ is the nominal flywheel angular momentum in body frame. They are defined as

$$\boldsymbol{h_w^B} = I_w \boldsymbol{B\Omega} \tag{8-5}$$

$$\boldsymbol{h_{nom}^B} = I_w \boldsymbol{B\Omega_{nom}} \tag{8-6}$$

Where $\boldsymbol{\Omega_{nom}}$ is the nominal flywheel speed vector. The commanded magnetic dipole moment required to desaturate the flywheel is calculated as

$$\boldsymbol{m_{cmd}} = \frac{k}{\|\boldsymbol{B}^{body}\|^2}(\boldsymbol{h_{ex}^B} \times \boldsymbol{b^B}) \tag{8-7}$$

Where k is the control gain and $\boldsymbol{b^B}$ is the local magnetic field vector expressed in body frame. Thus, the commanded MTQ torque used for desaturation is

$$\boldsymbol{\tau_{MTQ\_cmd}^B} = \boldsymbol{m_{cmd}} \times \boldsymbol{b^B} \tag{8-8}$$

9) Simulation:

The simulation was set up to evaluate the effectiveness of MTQ in excess angular momentum management after HCMG performed both rapid retargeting and precision pointing over many orbits. The simulation is a 6 Degrees of Freedom simulation written and the results were plotted on MATLAB.

9.1) Satellite Geometry

The satellite used in the simulation is a 12U Earth observational CubeSat with the dimension of 230 by 240 by 360 mm. The satellite's body fixed frame has its origin at satellite's

geometric center as shown in Figure 9-1. The z-axis points along satellite's long axis, the x-axis is parallel to the side that is 240 mm wide, and y-axis is formed from the vector product of z and x axes.

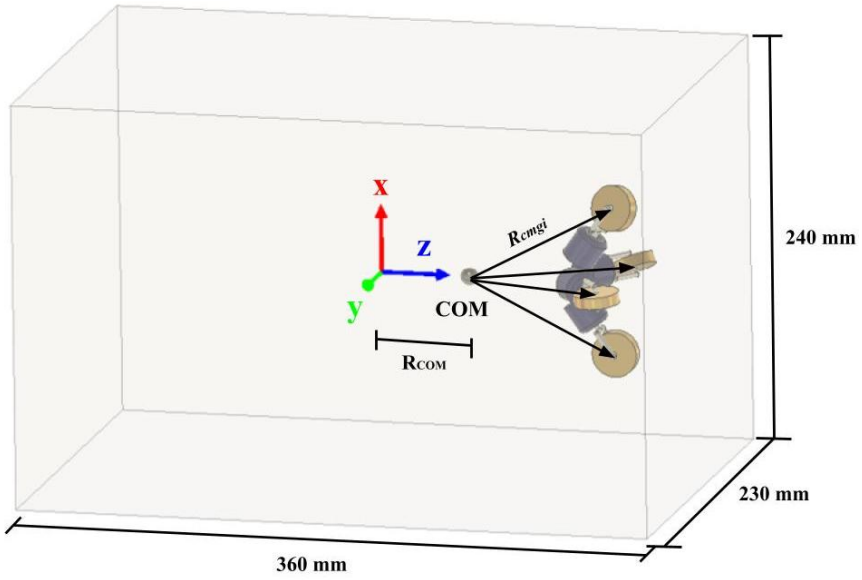

Figure 9-1: 12U CubeSat Geometry

The center of mass (COM) is assumed to be shifted a distance R_{COM} along the positive z direction. This assumption was made to allow for the effects of disturbance torques on the satellite. Additionally, the position of each gimbal flywheel relative to COM is denoted as $\boldsymbol{R}_{cmgi}$.

Based on the given dimension, $\bar{\boldsymbol{J}}_{\boldsymbol{C}}^{\boldsymbol{B}}$ is given as

$$\bar{\boldsymbol{J}}_{\boldsymbol{C}}^{\boldsymbol{B}} = \begin{bmatrix} 0.38 & 0 & 0 \\ 0 & 0.37 & 0 \\ 0 & 0 & 0.33 \end{bmatrix} \text{kgm}^2$$

Other parameters of the satellite are listed in Table 9-1.

Parameter	Value	Unit
m	24	kg
C_D	2.2	N/A
$\boldsymbol{R_{COM}}$	$[0 \quad 0 \quad 0.5]^T$	m
$\boldsymbol{R_{cmg_i}}$	$[\pm 0.01 \quad 0 \quad 0.1]^T$ $[0 \quad \pm 0.01 \quad 0.1]^T$	m

Table 9-1: Satellite Model Parameters

9.2) Simulation Setup

The simulation follows the following block diagram

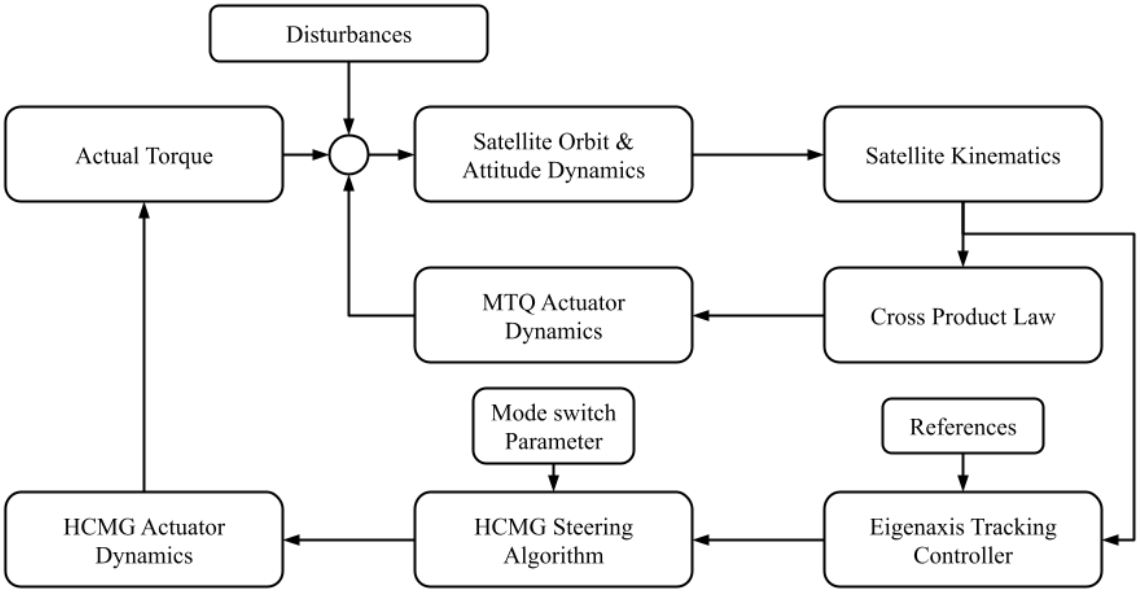

Figure 9-2: Simulation Block Diagram

9.2.1) Actual Torque

The actual torque is the torque generated by the HCMG system. It is based on equation (3-18) with the term $I_{gw}\boldsymbol{C}\ddot{\boldsymbol{\delta}}$ neglected, which is given as

$$\boldsymbol{\tau}_{act} = I_w \boldsymbol{A}\hat{\boldsymbol{\Omega}}\dot{\boldsymbol{\delta}}_{act} + I_w \boldsymbol{B}\dot{\boldsymbol{\Omega}}_{act} \tag{9-1}$$

Where $\dot{\boldsymbol{\delta}}_{act}$ and $\dot{\boldsymbol{\Omega}}_{act}$ are the actual gimbal rates and flywheel acceleration produced by the actuator, respectively.

9.2.2) Satellite Orbital and Attitude Dynamics

9.2.2.1) Satellite Orbital Dynamics

Satellite Orbital Dynamics is taken into consideration, because gravity gradient, aerodynamic drag, J2 Perturbation, and Earth's magnetic model are functions of satellite's position relative to ECI frame. In this simulation, the orbit is modeled using standard two body equations of motion.

$$\ddot{\boldsymbol{r}}^{\boldsymbol{ECI}} = -\mu \frac{\boldsymbol{r}^{\boldsymbol{ECI}}}{{r^{ECI}}^3} + \boldsymbol{a}_{\boldsymbol{J2}} + \boldsymbol{a}_{\boldsymbol{aero}}^{\boldsymbol{ECI}}$$

(9-2)

Where μ is Earth's standard gravitational parameter and $\boldsymbol{r^{ECI}}$ is the satellite's position vector in ECI coordinate frame. The initial position and velocity of the orbit were arbitrarily chosen based on the orbital parameters shown in Table 9-2Figure 9-1.

Parameter	Value	Unit
μ	$3.986004418 \times 10^{14}$	m^3/s^2
R_{Earth}	6,378,137	m
a	6878137	m
e	0.01	N/A
i	97.6	deg
Ω_{RAAN}	45	deg
ω_p	45	deg
f_o	0	deg

Table 9-2: Initial orbital parameters

Using the chosen parameters, initial position and velocity of the satellite expressed in ECI frame can be found through perifocal coordinate frame (PQW). The position and velocity in PQW frame can be found from

$$\boldsymbol{r_o^{PQW}} = \frac{a(1-e^2)}{1+e\cos(f)}\begin{bmatrix}\cos(f_o)\\ \sin(f_o)\\ 0\end{bmatrix} \tag{9-3}$$

$$\boldsymbol{v_o^{PQW}} = \sqrt{\frac{\mu}{a(1-e^2)}}\begin{bmatrix}-\sin(f_o)\\ e+\cos(f_o)\\ 0\end{bmatrix} \tag{9-4}$$

The position and velocity can then be transformed into ECI coordinate frame using a direction cosine matrix $\boldsymbol{R_{PQW}^{ECI}}$ based on the 3-1-3 Euler sequence through $-\omega$, $-i$, and $-\Omega$.

$$\boldsymbol{R_{PQW}^{ECI}} = \boldsymbol{R_3}(-\Omega)\boldsymbol{R_1}(-\mathrm{i})\,\boldsymbol{R_3}(-\omega) \tag{9-5}$$

Thus, the initial position and velocity vectors in ECI frame can be found from

$$\boldsymbol{r_o^{ECI}} = \boldsymbol{R_{PQW}^{ECI}}\boldsymbol{r_o^{PQW}}$$

(9-6)

$$\boldsymbol{v_o^{ECI} = R_{PQW}^{ECI} v_o^{PQW}}$$

(9-7)

9.2.2.2) Satellite Attitude Dynamics

Satellite attitude dynamics equation is obtained by rearranging equation (*3-40*) and (*3-41*). The equation of motion is given as

$$\dot{\boldsymbol{\omega}}_{\boldsymbol{B}}^{\boldsymbol{B}} = \boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}^{-1}}\left(-\boldsymbol{\tau}_{tot} - \boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}^{\times}}\left(\boldsymbol{J}_{\boldsymbol{C}}^{\boldsymbol{B}}\boldsymbol{\omega}_{\boldsymbol{B}}^{\boldsymbol{B}} + \boldsymbol{h}^{\boldsymbol{B}}\right)\right)$$

(9-8)

Where $\boldsymbol{\tau}_{tot}$ is the total torque which is the sum of actual torque $\boldsymbol{\tau}_{act}$, disturbance torque $\boldsymbol{\tau}_{dist}$, and desaturation torque $\boldsymbol{\tau}_{mtq}$.

$$\boldsymbol{\tau}_{tot} = \boldsymbol{\tau}_{act} - \boldsymbol{\tau}_{dist} - \boldsymbol{\tau}_{mtq}$$

(9-9)

9.2.3) Disturbance Forces and Torques

Disturbances that are modeled in the simulation include aerodynamic drag, aerodynamic torque, gravity gradient torque, and J2 Perturbation. Solar radiation pressure is not considered because its contribution to overall disturbance torque is small compared to gravity gradient and aerodynamic disturbances in Low Earth Orbit. The governing equation for each disturbance is described in Chapter 7 and is summarized as shown in Table 9-3.

J2 Perturbation	$\boldsymbol{a}_{J2} = -\frac{3\mu J_2 R_E^2}{2\boldsymbol{r}^5}\begin{bmatrix} x\left(1 - 5\frac{z^2}{r^2}\right) \\ y\left(1 - 5\frac{z^2}{r^2}\right) \\ z\left(3 - 5\frac{z^2}{r^2}\right) \end{bmatrix}$
Aerodynamic Drag	$\boldsymbol{F_{aero\_i}^B} = \frac{1}{2}\rho C_D A_{face_i}\left\|\boldsymbol{v_{body}^2}\right\| \max(0, \hat{\boldsymbol{u}} \cdot \hat{\boldsymbol{n}}_{\boldsymbol{i}})\, \hat{\boldsymbol{u}}$ $\boldsymbol{F_{aero}^B} = \sum_{i=1}^{N_{face}} \boldsymbol{F_{aero\_i}^B}$

Aerodynamic Torque	$\boldsymbol{\tau}_{\boldsymbol{aero}}^{\boldsymbol{B}} = \sum_{i=1}^{N_{face}} \left(\boldsymbol{r}_{\boldsymbol{cp}_i} \times \boldsymbol{F}_{\boldsymbol{aero}_i}^{\boldsymbol{B}}\right)$
Gravity Gradient Torque	$\boldsymbol{\tau}_{\boldsymbol{gg}} = \frac{3\mu}{\|\boldsymbol{r}^{\boldsymbol{ECI}}\|^5} \boldsymbol{r}^{\boldsymbol{ECI}} \times (\boldsymbol{I} \cdot \boldsymbol{r}^{\boldsymbol{ECI}})$

Table 9-3: Summary of Disturbances Governing Equations

The density in aerodynamic drag formula is modeled using exponential atmospheric density model shown in Appendix A.

9.2.4) Satellite Kinematics

In this simulation, satellite attitude is described with quaternions rather than Euler angles. A unit quaternion representing a rotation by an angle θ about a unit axis $\hat{\boldsymbol{a}}$ is defined as

$$\boldsymbol{q} = \begin{bmatrix} \boldsymbol{\epsilon} \\ \eta \end{bmatrix} \tag{9-10}$$

Where $\boldsymbol{\epsilon}$ is the vector or imaginary part that encodes the rotation axis, scaled by $\sin\left(\frac{\theta}{2}\right)$ and η is the scalar or real part that encodes rotation angle via $\cos\left(\frac{\theta}{2}\right)$.

$$\boldsymbol{\epsilon} = \hat{\boldsymbol{a}} \sin\left(\frac{\theta}{2}\right) = \begin{bmatrix} \epsilon_x \\ \epsilon_y \\ \epsilon_z \end{bmatrix} \tag{9-11}$$

$$\eta = \cos\left(\frac{\theta}{2}\right) \tag{9-12}$$

This quaternion follows a scalar last and uses the Hamiltonian product convention. It represents the rotation from satellite body frame to ECI frame. The angular velocity is always resolved in the body frame. Under these conventions the kinematics is

$$\dot{\boldsymbol{q}} = \frac{1}{2} \boldsymbol{q} \otimes \boldsymbol{\omega}_q \tag{9-13}$$

Where $\boldsymbol{q}$ represents the body to inertial rotation and $\boldsymbol{\omega}_q$ is defined as

$$\boldsymbol{\omega}_q = \begin{bmatrix} \boldsymbol{\omega} \\ 0 \end{bmatrix}$$

(9-14)

Therefore, the governing differential equations of the quaternion kinematics are given by

$$\dot{\boldsymbol{\epsilon}} = \frac{1}{2}\boldsymbol{\omega}^{\times} + \frac{1}{2}\eta\boldsymbol{\omega}$$

(9-15)

$$\dot{\eta} = -\frac{1}{2}\boldsymbol{\omega}^{T}\boldsymbol{\epsilon}$$

(9-16)

Where $\boldsymbol{\omega}$ and $\dot{\boldsymbol{\epsilon}}$ have an inverse relationship given by

$$\boldsymbol{\omega} = \frac{1}{2}\left[\frac{\eta^2\mathbf{1} - \eta\boldsymbol{\epsilon}^{x} + \boldsymbol{\epsilon}\boldsymbol{\epsilon}^{T}}{\eta}\right]\dot{\boldsymbol{\epsilon}}$$

(9-17)

Additionally, the direction cosine matrix recovered from $\boldsymbol{q}$ is found by applying the Euler-Rodrigues formula. The formula is

$$\boldsymbol{R_B^{ECI}}(\boldsymbol{q}) = (\eta^2 - \boldsymbol{\varepsilon}^{\mathbf{T}}\boldsymbol{\varepsilon})\mathbf{1}_{\mathbf{3\times 3}} + 2\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^{\mathbf{T}} - 2\eta\boldsymbol{\varepsilon}^{\times}$$

(9-18)

9.2.5) Eigenaxis Tracking Controller

Eigenaxis Tracking Controller is a Proportional Derivative controller that is based on eigen axis regulator [cite same source as Patankar 69,70]. The controller minimizes the attitude error $[\boldsymbol{\epsilon}_e, \eta_e]$ which is the eigen axis rotation between satellite's current attitude $[\boldsymbol{\epsilon}, \eta]$ and the reference or desired attitude $[\boldsymbol{\epsilon}_d, \eta_d]$. The eigen axis rotation is given by

$$\begin{bmatrix}\boldsymbol{\epsilon}_e \\ \eta_e\end{bmatrix} = \begin{bmatrix}\eta_d\mathbf{1}_{3\times 3} + \boldsymbol{\epsilon}_d{}^{\times} & \boldsymbol{\epsilon}_d \\ -\boldsymbol{\epsilon}_d{}^{T} & \eta_d\end{bmatrix}\begin{bmatrix}-\boldsymbol{\epsilon} \\ \eta\end{bmatrix}$$

(9-19)

According to (9-10) and (9-11) attitude and angular velocity are coupled, therefore the angular velocity error $\boldsymbol{\omega}_e$ must also be tracked. It is given as

$$\boldsymbol{\omega}_e = \boldsymbol{\omega} - \boldsymbol{\omega}_d$$

(9-20)

Where $\boldsymbol{\omega}_d$ is the reference or desired satellite's angular velocity expressed in body fixed frame. Thus, the control torque can be given as

$$\boldsymbol{\tau} = -\boldsymbol{\omega}^{\times}(\boldsymbol{J\omega} + \boldsymbol{h_w}) - \boldsymbol{J\dot{\omega}_d} + K_p\boldsymbol{J\epsilon}_e + K_d\boldsymbol{J\omega_e} \tag{9-21}$$

The term $-\boldsymbol{\omega}^{\times}(\boldsymbol{J\omega} + \boldsymbol{h_w})$ is the feedback linearization term, which is fed back to cancel out the nonlinear gyroscopic in satellite. $\boldsymbol{J\dot{\omega}_d}$ is the feedforward term that provides torque necessary to keep up with the desired acceleration. $K_p\boldsymbol{J\epsilon}_e + K_d\boldsymbol{J\omega}_e$ are proportional and derivative term scaled by the moment of inertia matrix to ensure that the resulting angular acceleration is always parallel to the instantaneous attitude error vector. Where K_p and K_d are proportional and derivative gain which were chosen to be $K_p = 0.3$ and $K_d = 1$, respectively. With the given control torque (*9-21*), Patankar proved that the system becomes asymptotically stable according to Lasalle's invariant set theorem [1].

9.2.6) Tracking References

Since the satellite is an Earth observational CubeSat, its main objective is to perform nadir pointing or earth pointing. Nadir pointing can be achieved by tracking the Local Horizontal, Local Vertical (LVLH) frame as the reference. LVLH frame is a rotating orbital frame centered around satellite's COM. The coordinate basis can be defined as

$$\hat{z}^{LVLH} = -\frac{\boldsymbol{r^{ECI}}}{\|\boldsymbol{r^{ECI}}\|} \tag{9-22}$$

$$\hat{y}^{LVLH} = -\frac{\boldsymbol{h}}{\|\boldsymbol{h}\|} = -\frac{\boldsymbol{r^{ECI}} \times \boldsymbol{v^{ECI}}}{\|\boldsymbol{r^{ECI}} \times \boldsymbol{v^{ECI}}\|} \tag{9-23}$$

$$\hat{x}^{LVLH} = \hat{y}^{LVLH} \times \hat{z}^{LVLH} \tag{9-24}$$

Where $\boldsymbol{r^{ECI}}$, $\boldsymbol{v^{ECI}}$, and $\boldsymbol{h}$ are column vectors. $\hat{z}^{LVLH}$ is known as the local vertical axis, $\hat{y}^{LVLH}$ is the orbit normal, and $\hat{x}_{LVLH}$ is the local horizontal axis. Using $\hat{x}^{LVLH}$, $\hat{y}^{LVLH}$, and $\hat{z}^{LVLH}$ a rotation matrix that transforms LVLH to ECI frame can be formed

$$\boldsymbol{R}_{LVLH}^{ECI} = [\hat{x}^{LVLH} \quad \hat{y}^{LVLH} \quad \hat{z}^{LVLH}] \tag{9-25}$$

Since the simulation expresses attitude in quaternions, the rotation matrix can be transformed into the quaternion form using Shepperd's method.

$$\eta_d = \frac{1}{2}\sqrt{1 + \mathrm{tr}(\boldsymbol{R}_{LVLH}^{ECI})} \tag{9-26}$$

$$\boldsymbol{\epsilon_d} = \frac{1}{4\eta}\left(\boldsymbol{R}_{LVLH}^{ECI} - \boldsymbol{R}_{LVLH}^{ECI\ T}\right)^{\vee} \tag{9-27}$$

Where $\mathrm{tr}(\boldsymbol{R}_{LVLH}^{ECI})$ is Trace of rotation matrix $\boldsymbol{R}_{LVLH}^{ECI}$, which is the sum of the diagonal elements of the matrix. Additionally, for any given skew symmetric matrix $\boldsymbol{S}$

$$S = \begin{bmatrix} 0 & -s_3 & s_2 \\ s_3 & 0 & -s_1 \\ -s_2 & s_1 & 0 \end{bmatrix} \tag{9-28}$$

The vee map $\boldsymbol{S}^{\vee}$ is

$$\boldsymbol{S}^{\vee} = \begin{bmatrix} s_1 \\ s_2 \\ s_3 \end{bmatrix} \tag{9-29}$$

The quaternion form of the rotation matrix is the reference attitude $\boldsymbol{q_d}$ that maps the rotation from the desired body to ECI frame.

$$\boldsymbol{q_d} = \begin{bmatrix} \boldsymbol{\epsilon_d} \\ \eta_d \end{bmatrix} \tag{9-30}$$

Since the satellite is orbiting around the Earth, LVLH frame is always rotating and accelerating in the inertial space. Therefore, the desired angular velocity $\boldsymbol{\omega_d}$ and acceleration $\boldsymbol{\dot{\omega}_d}$ must also be tracked. The desired angular velocity must match satellite's orbital rate, which is given by the rate of change of the true anomaly $\dot{f}$. The true anomaly can be written in terms of angular momentum and position as

$$\dot{f} = \frac{\boldsymbol{h}}{\boldsymbol{r}^2} \tag{9-31}$$

Since the angular momentum lies along $\hat{y}_{LVLH}$, the desired angular velocity can be written as

$$\boldsymbol{\omega_d^{LVLH}} = \begin{bmatrix} 0 \\ -\dfrac{\boldsymbol{h}}{\boldsymbol{r}^2} \\ 0 \end{bmatrix}_{LVLH} \tag{9-32}$$

Furthermore, the desired angular velocity can be computed from the time derivative of (9-24).

$$\dot{\boldsymbol{\omega}}_{\boldsymbol{d}}^{\boldsymbol{LVLH}} = \begin{bmatrix} 0 \\ -\dfrac{2\boldsymbol{h}\dot{\boldsymbol{r}}}{\boldsymbol{r}^3} \\ 0 \end{bmatrix}_{LVLH} \tag{9-33}$$

Since the references are still expressed in LVLH frame but the controller minimizes the errors that are expressed in body frame, $\boldsymbol{\omega}_{\boldsymbol{d}}^{\boldsymbol{LVLH}}$ and $\dot{\boldsymbol{\omega}}_{\boldsymbol{d}}^{\boldsymbol{LVLH}}$ must be transformed into satellite's body frame. The transformation is done by

$$\boldsymbol{\omega}_{\boldsymbol{d}}^{\boldsymbol{B}} = \boldsymbol{R}_{\boldsymbol{LVLH}}^{\boldsymbol{B}} \boldsymbol{\omega}_{\boldsymbol{d}}^{\boldsymbol{LVLH}} \tag{9-34}$$

$$\dot{\boldsymbol{\omega}}_{\boldsymbol{d}}^{\boldsymbol{B}} = \boldsymbol{R}_{\boldsymbol{LVLH}}^{\boldsymbol{B}} \dot{\boldsymbol{\omega}}_{\boldsymbol{d}}^{\boldsymbol{LVLH}} + \dot{\boldsymbol{R}}_{\boldsymbol{LVLH}}^{\boldsymbol{B}} \boldsymbol{\omega}_{\boldsymbol{d}}^{\boldsymbol{LVLH}} \tag{9-35}$$

When the tracking error is small, the term $\dot{\boldsymbol{R}}_{\boldsymbol{LVLH}}^{\boldsymbol{B}} \boldsymbol{\omega}_{\boldsymbol{d}}^{\boldsymbol{LVLH}}$ can be neglected. Additionally, $\boldsymbol{R}_{\boldsymbol{LVLH}}^{\boldsymbol{B}}$ is the rotation matrix from LVLH frame to body frame which is defined as

$$\boldsymbol{R}_{\boldsymbol{LVLH}}^{\boldsymbol{B}} = \boldsymbol{R}_{\boldsymbol{ECI}}^{\boldsymbol{B}} \boldsymbol{R}_{\boldsymbol{LVLH}}^{\boldsymbol{ECI}} \tag{9-36}$$

The rotation matrix $\boldsymbol{R}_{\boldsymbol{LVLH}}^{\boldsymbol{ECI}}$ is given in (9-19). The rotation matrix $\boldsymbol{R}_{\boldsymbol{ECI}}^{\boldsymbol{B}}$ is the transformation from ECI frame to body frame, which is the inverse of the rotation matrix that transform body frame to ECI frame $\boldsymbol{R}_{\boldsymbol{B}}^{\boldsymbol{ECI}}$.

$$\boldsymbol{R}_{\boldsymbol{ECI}}^{\boldsymbol{B}} = \boldsymbol{R}_{\boldsymbol{B}}^{\boldsymbol{ECI}^{-1}} = \boldsymbol{R}_{\boldsymbol{B}}^{\boldsymbol{ECI}^{T}} \tag{9-37}$$

This approach, however, is susceptible to numerical errors as $\boldsymbol{q}$ is directly affected by disturbances. A better approach for computing $\boldsymbol{R}_{\boldsymbol{LVLH}}^{\boldsymbol{B}}$ is from a direction transformation of the quaternion error from (9-18) using Euler-Rodrigues formula. Therefore, $\boldsymbol{R}_{\boldsymbol{LVLH}}^{\boldsymbol{B}}$ can be written as

$$\boldsymbol{R}_{\boldsymbol{LVLH}}^{\boldsymbol{B}} = (\eta_e{}^2 - \boldsymbol{\varepsilon_e}^{\mathbf{T}} \boldsymbol{\varepsilon_e}) \mathbf{1}_{\mathbf{3\times 3}} + 2\boldsymbol{\varepsilon_e} \boldsymbol{\varepsilon_e}^{\mathbf{T}} - 2\eta_e \boldsymbol{\varepsilon_e}^{\times} \tag{9-38}$$

9.2.6) HCMG Steering Algorithm

The steering algorithm maps the control torque output from Eigenaxis controller onto the gimbal rates and flywheel acceleration using the following formulas.

$$\dot{\boldsymbol{\delta}} = \alpha (I_w \widehat{\boldsymbol{\Omega}})^{-1} \left[\boldsymbol{A}^T (\boldsymbol{A}\boldsymbol{A}^T + \lambda \boldsymbol{E})^{-1} \boldsymbol{\tau}_{gyro}^{B} + \beta \boldsymbol{d}_{\boldsymbol{RW}} \right] \tag{9-39}$$

$$\dot{\boldsymbol{\Omega}} = (1-\alpha)\frac{1}{I_w}\boldsymbol{B^T}(\boldsymbol{BB^T})^{-1}\boldsymbol{\tau}_{\boldsymbol{D\_w}}^{\boldsymbol{B}} \tag{9-40}$$

And the formulas for λ and β are

$$\lambda = \lambda_0 e^{-\mu_1 S_{CMG}} \tag{9-41}$$

$$\beta = \beta_0 e^{-\mu_2 S_{RW}} \tag{9-42}$$

As discussed in chapter 6, $\boldsymbol{E}$, λ_0, β_0, μ_1, and μ_2 are tunable parameters. The chosen values for the parameters are shown in Table 9-4.

Parameter	Value	Unit
$\boldsymbol{E}$	$\boldsymbol{E} = \begin{bmatrix} 1 & e_1 & e_2 \\ e_1 & 1 & e_3 \\ e_2 & e_3 & 1 \end{bmatrix}$ $e_1 = 0.001\sin(t+0.25)$ $e_2 = 0.001\sin(t+0.13)$ $e_3 = 0.001\sin(t+0.05)$	N/A
λ_0	0.1	N/A
β_0	0.1	N/A
μ_1	5	N/A
μ_2	5	N/A

Table 9-4: HCMG Steering Algorithm Parameters

9.2.7) HCMG Actuator Dynamics

The HCMG chosen for this simulation is a pyramid cluster made up of 4 HCMG assemblies. Each assembly weighs 400 grams, with a total cluster mass of 1.6 kg. The cluster takes up 1.5U of satellite's total 12U. The cluster has a pyramid angle of $\theta_{pyramid} = 54.74$ degrees. The Parameters for each flywheel and gimbal assembly are shown in Table 9-5 and Table 9-6, respectively.

Parameter	Value	Unit

I_w	4.2×10^{-5}	kgm^2
$\boldsymbol{\Omega_{max}}$	17,189 1800	RPM rad/s
$\dot{\boldsymbol{\Omega}}_{\boldsymbol{max}}$	55	rad/s^2
$\boldsymbol{\Omega_{nom}}$	8000 838	RPM rad/s
$\boldsymbol{h_{nom}}$	0.036	Nms

Table 9-5: Flywheel Parameters

Parameter	**Value**	**Unit**
I_w	3.5×10^{-4}	kgm^2
$\boldsymbol{\delta_o}$	$\begin{bmatrix} 211 \\ -31 \\ 211 \\ -31 \end{bmatrix}$	deg
$\dot{\boldsymbol{\delta}}_{\boldsymbol{max}}$	110	deg/s
$\dot{\boldsymbol{\Omega}}_{\boldsymbol{max}}$	600	deg/s^2

Table 9-6: Gimbal Parameters

With the chosen pyramid configuration with the angle of $\theta_{pyramid} = 54.74$ degrees the Jacobian matrices $\boldsymbol{A}$ and $\boldsymbol{B}$ are of the form

$$\boldsymbol{A} = \begin{bmatrix} -\cos(\theta)\sin(\delta_1) & -\cos(\delta_2) & \cos(\theta)\sin(\delta_3) & \cos(\delta_4) \\ \cos(\delta_1) & -\cos(\theta)\sin(\delta_2) & -\cos(\delta_3) & \cos(\theta)\sin(\delta_4) \\ \sin(\theta)\sin(\delta_1) & \sin(\theta)\sin(\delta_2) & \sin(\theta)\sin(\delta_3) & \sin(\theta)\sin(\delta_4) \end{bmatrix} \tag{9-43}$$

$$\boldsymbol{B} = \begin{bmatrix} \cos(\theta)\cos(\delta_1) & -\sin(\delta_2) & -\cos(\theta)\cos(\delta_3) & \sin(\delta_4) \\ \sin(\delta_1) & \cos(\theta)\cos(\delta_2) & -\sin(\delta_3) & -\cos(\theta)\cos(\delta_4) \\ -\sin(\theta)\cos(\delta_1) & -\sin(\theta)\cos(\delta_2) & -\sin(\theta)\cos(\delta_3) & -\sin(\theta)\cos(\delta_4) \end{bmatrix} \tag{9-44}$$

Usually, the dynamics of flywheel and gimbal dynamic models usually include the dead zones, bounded random walk, and friction consideration to account for the imperfection of physical HCMGs. However, it was proven that this randomness brought about less than 1% error to the overall efficiency of the HCMG [1].Therefore, the HCMG model was assumed to be perfect for this simulation to improve computational efficiency.

9.2.8) Cross Product Law

Cross product law is used to compute the desaturation torque to remove the total excess angular momentum [12]. Excess angular momentum, magnetic dipole moment, and desaturation torque are as follows

$$\boldsymbol{h}_{\boldsymbol{ex}}^{\boldsymbol{B}} = \boldsymbol{h}_{\boldsymbol{w}}^{\boldsymbol{B}} - \boldsymbol{h}_{\boldsymbol{nom}}^{\boldsymbol{B}} \tag{9-45}$$

$$\boldsymbol{m}_{\boldsymbol{cmd}} = \frac{k}{\|\boldsymbol{b}^{\boldsymbol{B}}\|^2}(\boldsymbol{h}_{\boldsymbol{ex}}^{\boldsymbol{B}} \times \boldsymbol{b}^{\boldsymbol{B}}) \tag{9-46}$$

$$\boldsymbol{\tau}_{\boldsymbol{MTQ\_cmd}}^{\boldsymbol{B}} = \boldsymbol{m}_{\boldsymbol{cmd}} \times \boldsymbol{b}^{\boldsymbol{B}} \tag{9-47}$$

The control gain is chosen to be $k = 3 \times 10^{-4}\ \mathrm{s}^{-1}$. $\boldsymbol{b}^{\boldsymbol{B}}$ comes from the geomatic field modeled using tilted dipole approximation. It represents Earth's magnetic field as the field of a single magnetic dipole, offset from Earth's rotation axis by a fixed tilt angle γ. The magnetic field produced by a dipole with moment μ_m at a position $\boldsymbol{r}_{\boldsymbol{ECI}}$ from the dipole center is given by

$$\boldsymbol{b}^{\boldsymbol{ECI}} = \frac{\mu_o \mu_m}{4\pi r^3}[3(\hat{\mathbf{m}} \cdot \hat{r})\hat{r} - \hat{\boldsymbol{m}}] \tag{9-48}$$

Where μ_o is permeability of free space, r is the magnitude of $\boldsymbol{r}^{\boldsymbol{ECI}}$, $\hat{r}$ is the unit position vector $\frac{r}{\boldsymbol{r}^{\boldsymbol{ECI}}}$, and $\hat{\boldsymbol{m}}$ is the unit vector along the dipole axis. Since the dipole is fixed to the Earth, it rotates with the Earth as time passes. Assuming that the Earth Centered, Earth Fixed (ECEF) coordinate frame is initially aligned with ECI frame. The initial dipole axis is defined in ECI frame as

$$\hat{\boldsymbol{m}}_{\boldsymbol{o}} = -\begin{bmatrix} \sin(\gamma) \\ 0 \\ \cos(\gamma) \end{bmatrix} \tag{9-49}$$

As time passes, the dipole axis rotates about Earth's rotation axis with the rate of $\omega_\oplus$ in ECEF frame. It can be transformed into ECI frame with the standard Euler's rotation matrix about the third axis.

$$\hat{\boldsymbol{m}}(t) = \boldsymbol{R}_{\boldsymbol{3}}(\omega_\oplus t)\hat{\boldsymbol{m}}_{\boldsymbol{o}} \tag{9-50}$$

Since the magnetic field is given in ECI frame, it must be transformed into body frame for the desaturation law.

$$\boldsymbol{b}^{body} = \boldsymbol{R}_{\boldsymbol{ECI}}^{\boldsymbol{B}}\boldsymbol{b}^{\boldsymbol{ECI}} \tag{9-51}$$

The values of μ_o, μ_m, γ, and $\omega_\oplus$ are shown in Table 9-7.

Parameter	Value	Unit
μ_o	$4\pi \times 10^{-7}$	H/m
μ_m	7.94×10^{22}	Am^2
γ	11.3	deg
$\omega_\oplus$	7.2921159×10^{-5}	rad/s

Table 9-7: Earth's Geomagnetic Model Parameters

9.2.9) MTQ Dynamics

Each MTQ is modeled using equation (7-10), in which the dipole moment produced by each MTQ depends on N, I, and A. In this simulation, the MTQs are assumed to be ideal without the faults that actual actuators. However, their output dipole moments and generated torques remain bounded by the constraints as listed in Table 9-8.

Parameter	Value	Unit
N	200	turns
$\boldsymbol{I_{max}}$	0.4	A
A_{coil}	0.02	m^2
m_{max}	4	Am^2
k	3×10^{-4}	s^{-1}

Table 9-8: MTQ Parameters

The attitude states are propagated with a forward Euler step of $dt = 0.2$ seconds, with the quaternion renormalized after each step. The orbital states use a semi-implicit Euler step. The velocity is updated first then the position is propagated using the updated velocity. This simplistic form was used for the orbit because it conserves the energy of the two-body problem better than the forward Euler over 15 orbits.

9.3) Simulation Cases

The simulation starts with the satellite performing rapid retargeting using CMG mode from the initial attitude $[0; 0; 0]$ to nadir pointing with satellite's body y-axis. The retargeting occurs at $t = 0$ seconds into the simulation. At $t = 25$ seconds, the mode switch parameter follows a cosine curve from $\alpha = 1$ to $\alpha = 0$ for 10 seconds, providing a smooth transition from CMG mode to RW mode for precision pointing. The simulation is divided into 2 cases. In case 1, MTQ is turned off. In case 2, MTQ is turned on for momentum desaturation. However, momentum desaturation will only take place during RW mode and never during CMG mode. Once HCMG operates as RW mode, MTQ is enabled and will continuously dump the excess momentum once the threshold is reached.

Since momentum takes time to build up, the simulation is run for 15 orbits to show the effect of MTQ over a long orbit period. MATLAB simulation logs and plot the following variables:

1. 3D Satellite Orbit trajectory expressed in ECI frame [km]
2. Orbit Altitude [km] vs time [s]
3. Orbit Speed [m/s] vs time [s]
4. Satellite's attitude in Euler angles [deg] vs time [s]
5. Satellite's attitude error in quaternions vs time [s]
6. Satellite's angular velocity error [deg/s] vs time [s]
7. Satellite's angular velocity [deg/s] vs time [s]
8. Gimbal rate of each gimbal [deg/s] vs time [s]
9. Gimbal angles of each gimbal [deg] vs time [s]
10. Angular acceleration of each flywheel [rad/s^2] vs time [s]
11. Angular velocity of each flywheel [rad/s] vs time [s]
12. Commanded torque and Actual torque [mNm] vs time [s]
13. Mode switch parameter magnitude vs time [s]
14. Singularity Parameters vs time [s]
15. Gravity gradient torque [μNm] vs time [s]
16. Aerodynamic torque [μNm] vs time [s]
17. Total disturbance torque [μNm] vs time [s]
18. Excess angular momentum vector in satellite's body frame [kgm^2/s] vs time [s]
19. Excess angular momentum magnitude in satellite's body frame [kgm^2/s] vs time [s]
20. MTQ commanded dipole moment [Am^2] vs time [s]
21. MTQ desaturation torque [μNm] vs time [s]

9.4) Simulation Results

9.4.1) Simulated Orbit Results

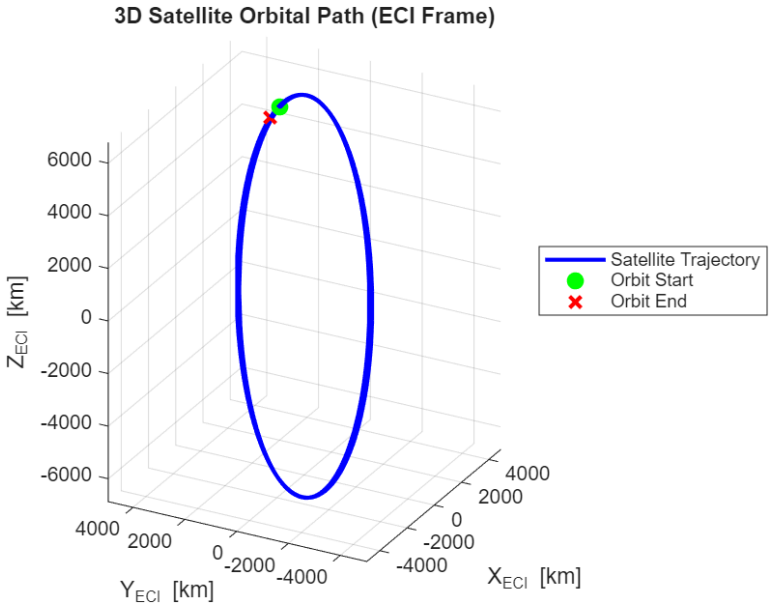

Figure 9-3: Satellite Orbit Trajectory

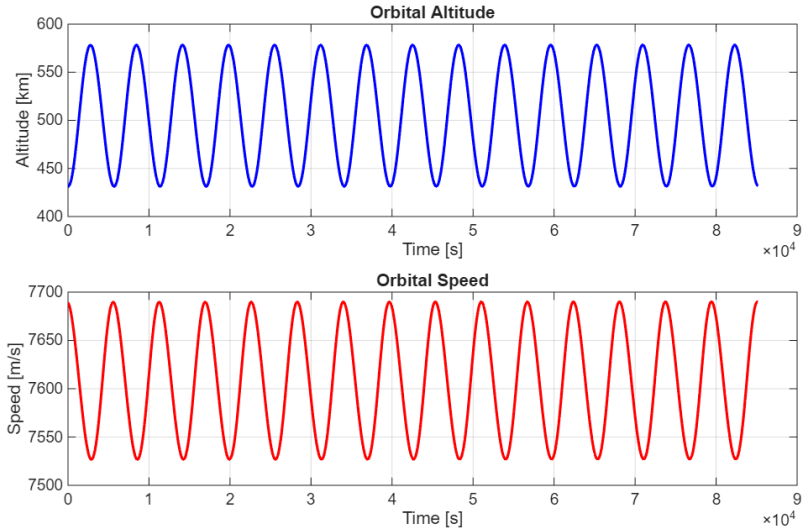

Figure 9-4: Satellite Orbit Altitude (Top) and Speed (Bottom) vs Time

Figure 9-3 and Figure 9-4 show that the satellite's orbit is correctly modeled. With $e = 0.01$, orbit is an ellipse making the altitude oscillate between 431.2 km and 568.8 km. This also directly lead to the orbital speed oscillating between 7689.1 m/s and 7536.9 m/s. Although the starting position and end position of the spacecraft after 15 orbits do not line up, it can be explained by the addition of J2 acceleration in the simulation.

9.4.2) Rapid Retargeting Phase Results

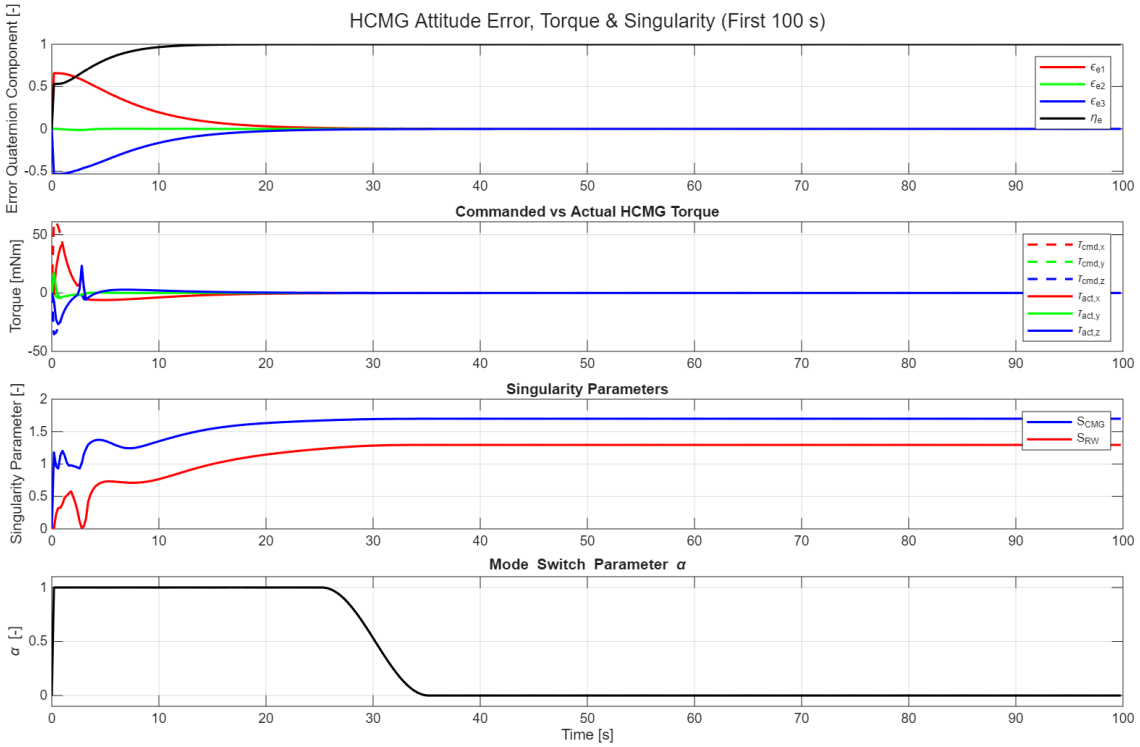

Figure 9-5: Attitude Error (1), Torque Output (2), Singularity Parameters (3), and Mode Switch Parameter (4) vs Time

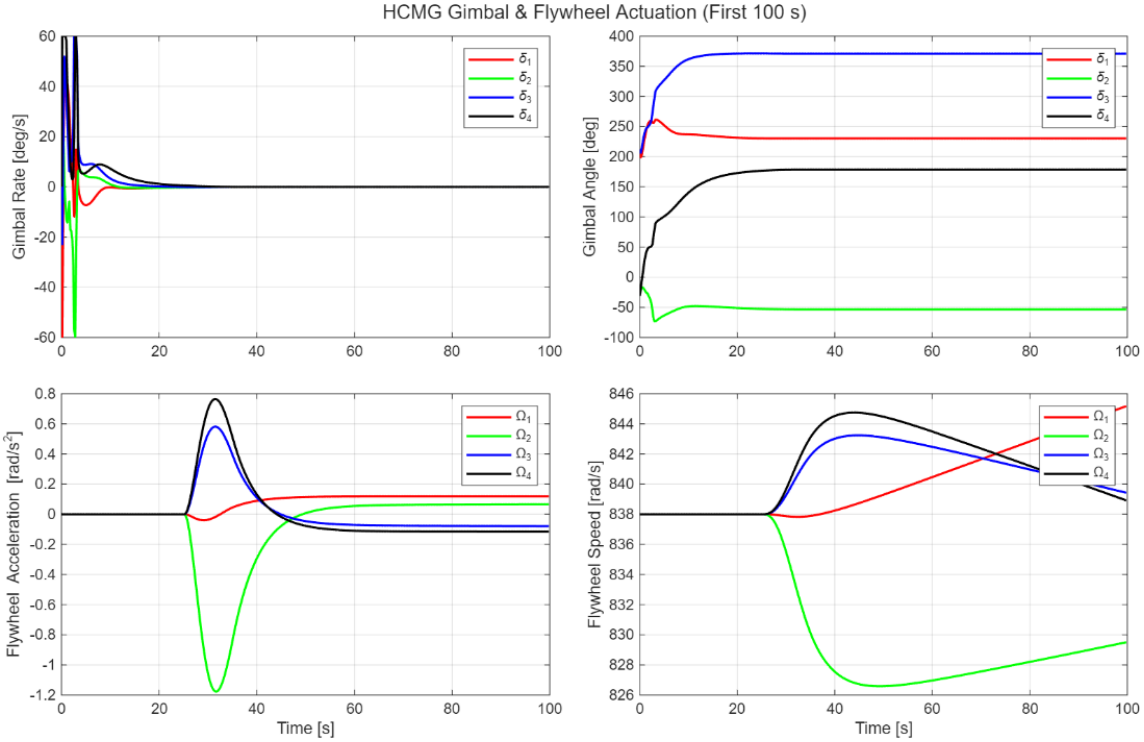

Figure 9-6: HCMG Results During Rapid Retargeting

Figure 9-5 and Figure 9-6 prove that the HCMG system is correctly modeled and was able to perform rapid retargeting during the first 25 seconds, bringing the attitude error to 0, before switching to precision pointing mode. Even though the initial gimbal angles were chosen to be near a singular configuration, the steering algorithm successfully steered the gimbal angles away from singularities as shown by both S_{RW} and S_{CMG} increasing over time.

9.4.3) Disturbances

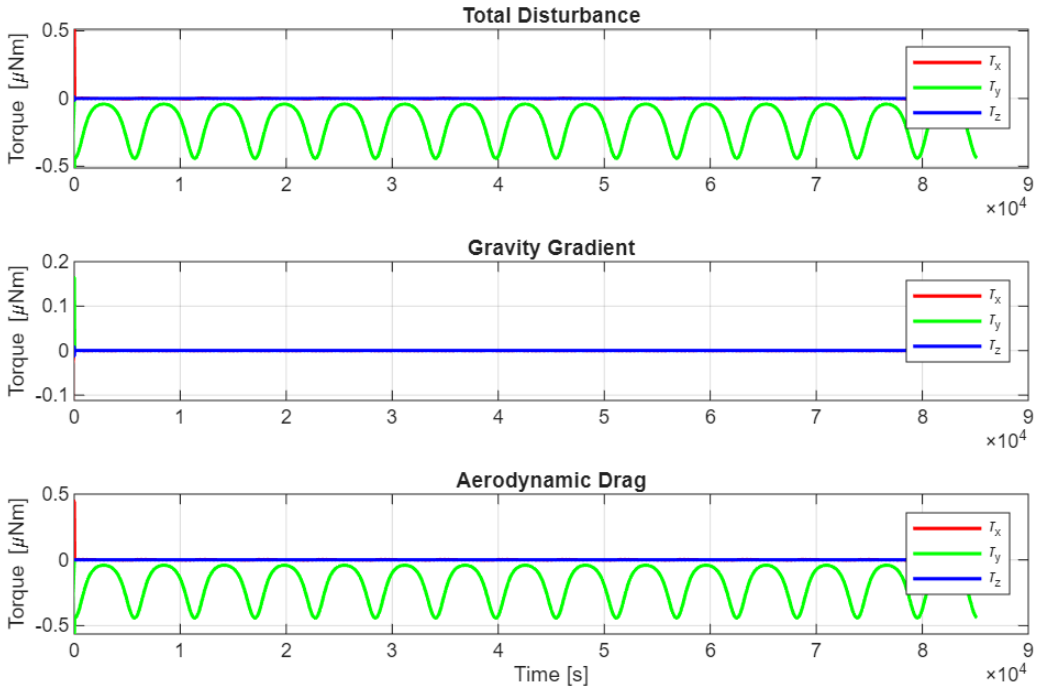

Figure 9-7: Disturbances vs Time

Most of the disturbances that affect the satellite attitude and contribute to excess angular momentum building up come from aerodynamic drag. Gravity gradient torque is shown to be relatively small when compared to torque generated from aerodynamic drag.

9.4.4) Precision Pointing Phase Results

9.4.5.1) Case 1

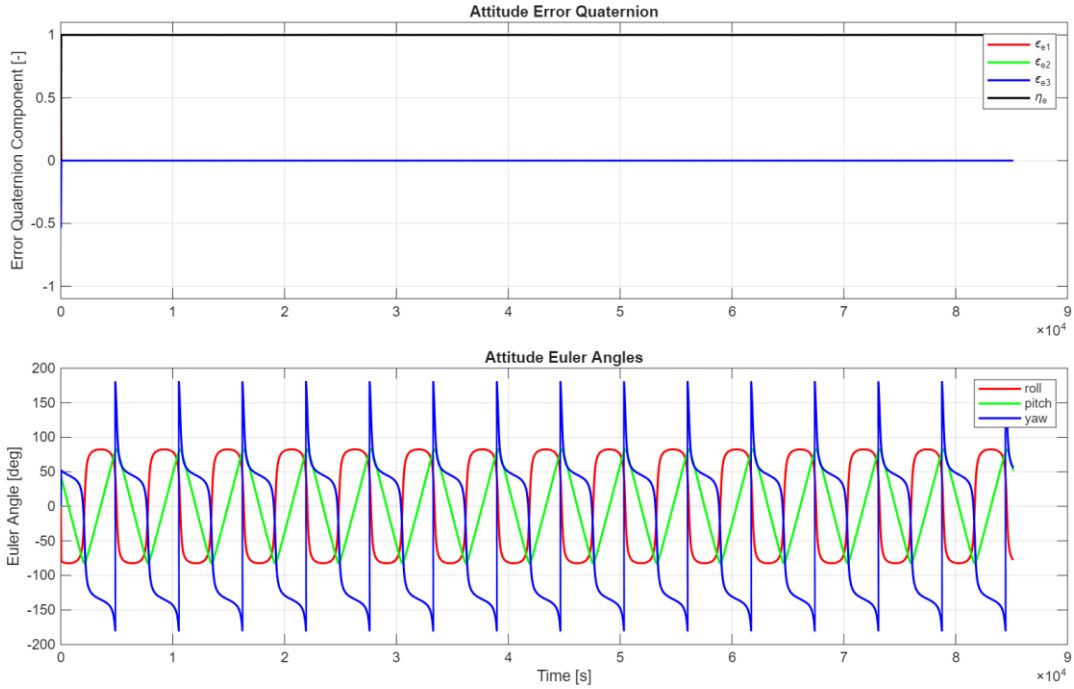

Figure 9-8: Case 1 Attitude Results vs Time

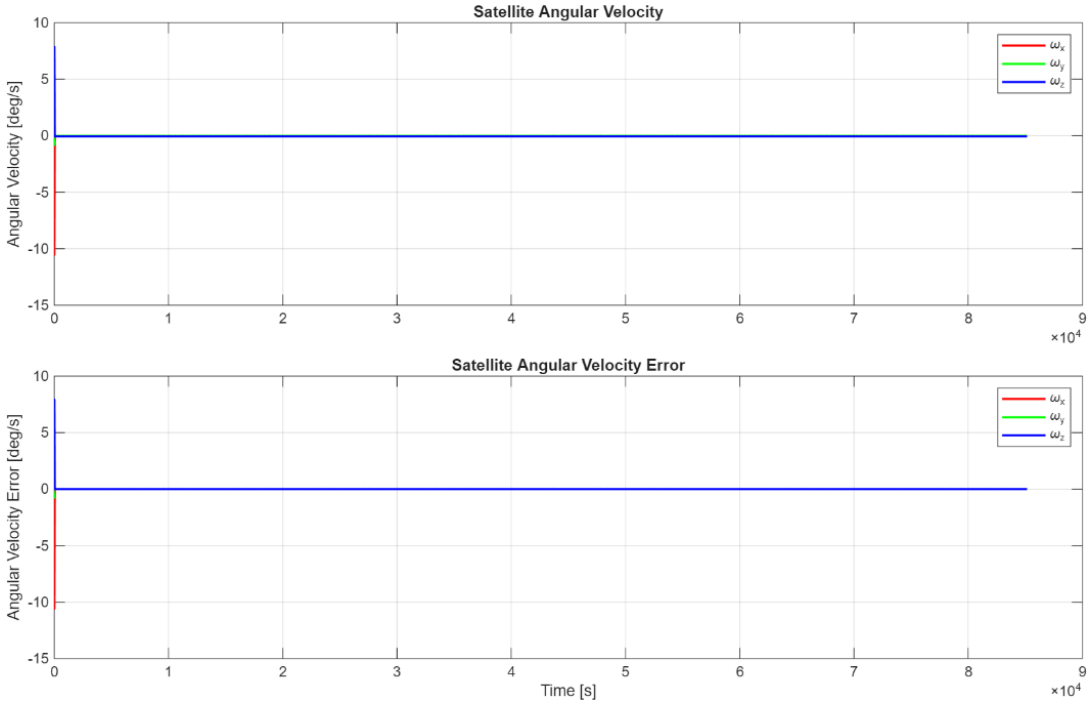

Figure 9-9: Case 1 Angular Velocity vs Time

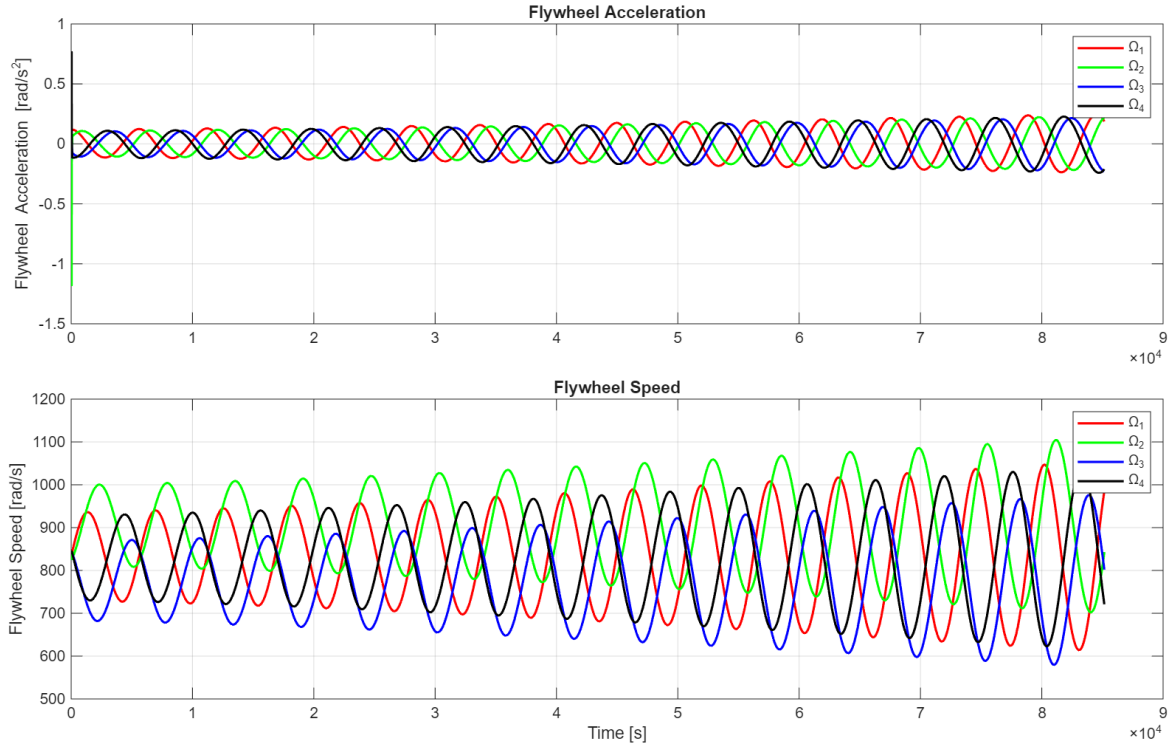

Figure 9-10: Case 1 Flywheel Results vs Time

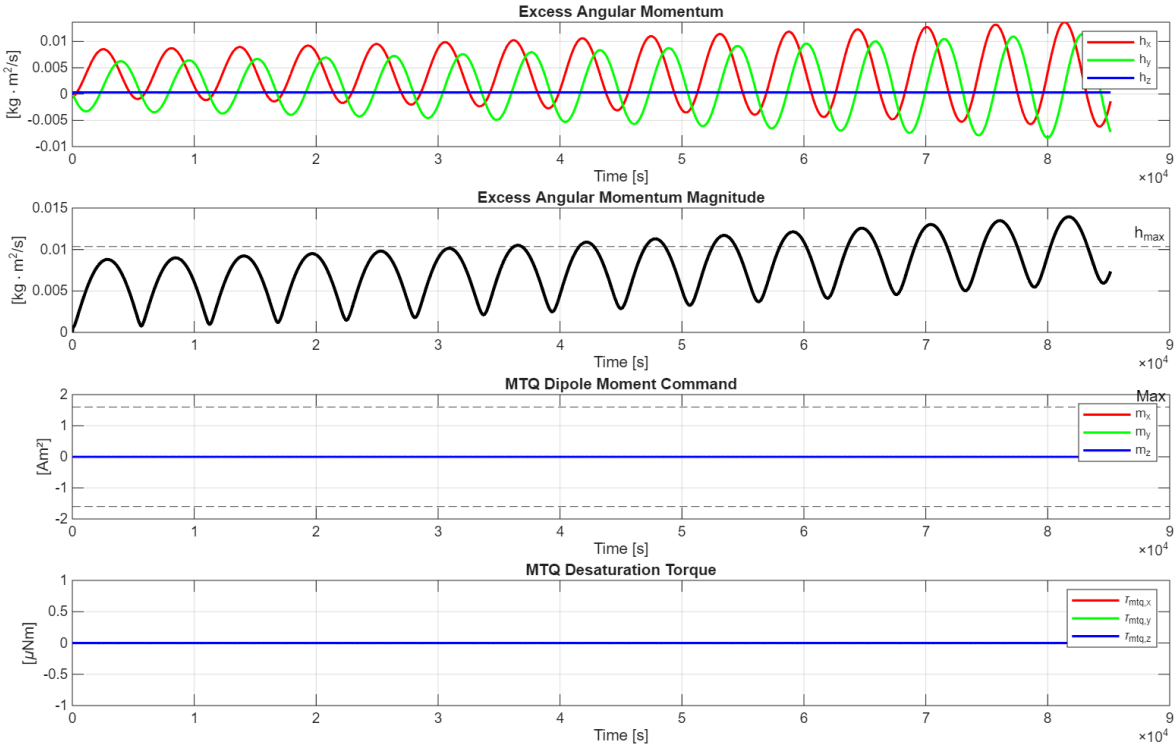

Figure 9-11: Case 1 Excess Angular Momentum (Top) and MTQ Responses (Bottom) vs Time

9.4.5.2) Case 2

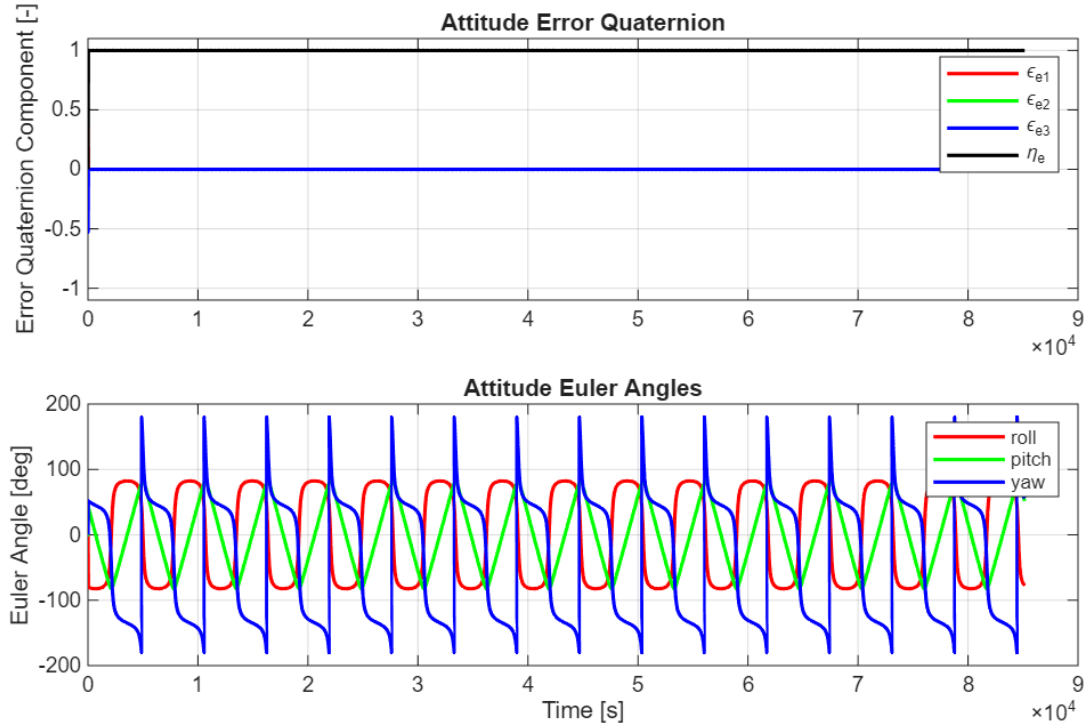

Figure 9-12: Case 2 Attitude Results vs Time

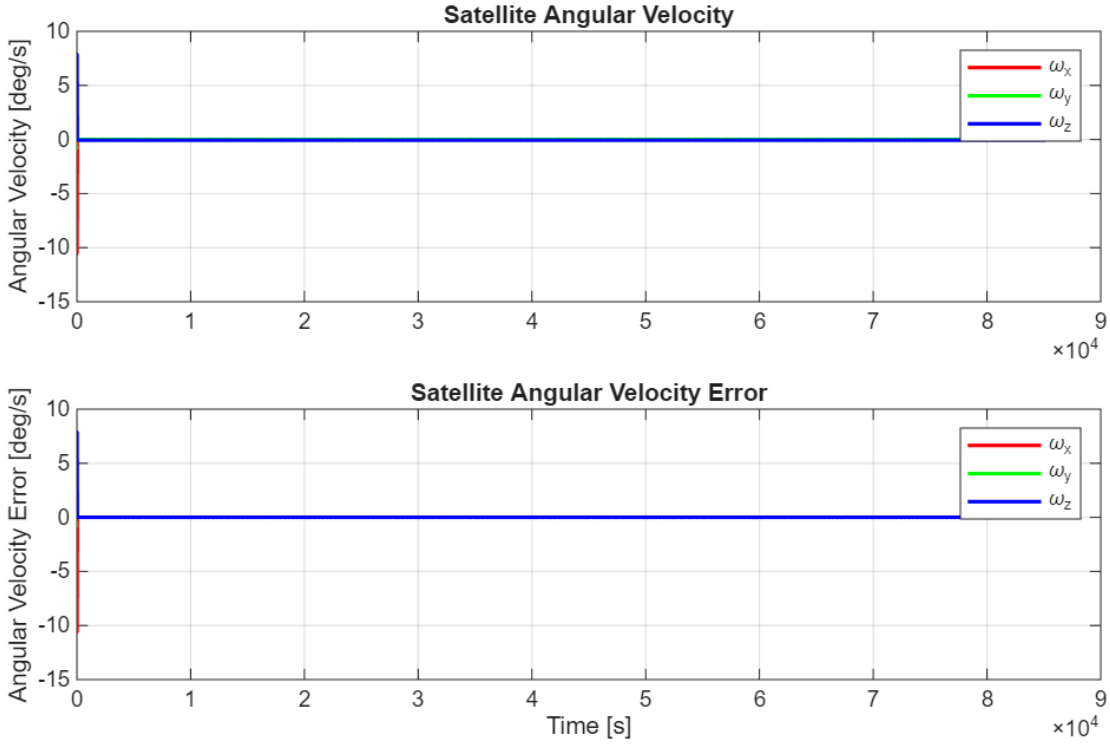

Figure 9-13: Case 2 Angular Velocity vs Time

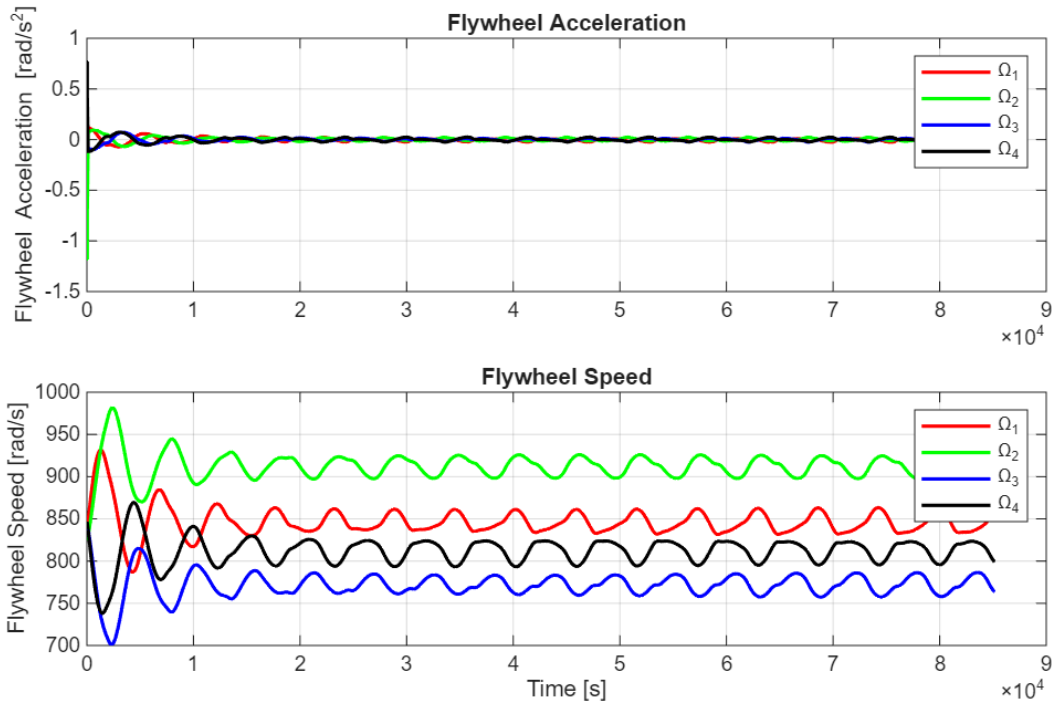

Figure 9-14: Case 2 Flywheel Results vs Time

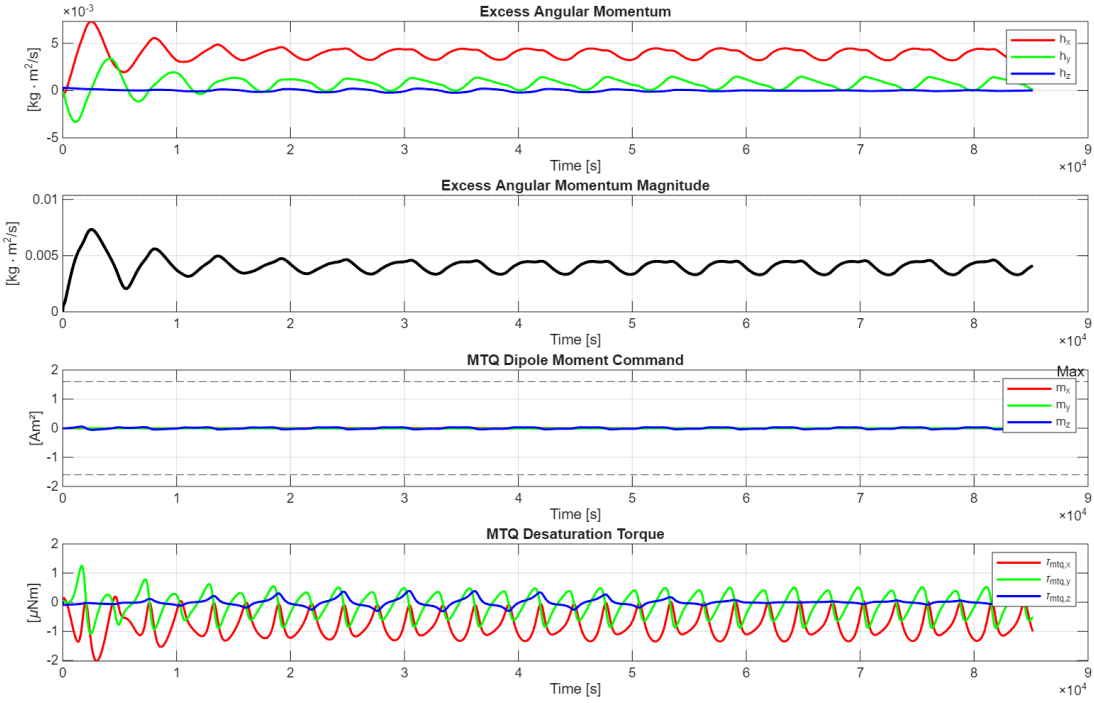

Figure 9-15: Case 2 Excess Angular Momentum (Top) and MTQ Responses (Bottom) vs Time

As shown in Figure 9-11, excess angular momentum in h_x and h_y continues to grow larger with time for a satellite with no MTQ. This also leads to both flywheels speed and acceleration drifting. As shown in Figure 9-10, where acceleration grow in magnitude, oscillate,

and never settle down to 0 rad/s^2. The speed, likewise, drift away from the flywheel speed. On the other hand, a satellite with MTQ equipped constantly provide enough desaturation torque to keep the excess angular momentum from growing, as shown in Figure 9-15. This also leads to the flywheel speed converging to around 838 rad/s, which is the nominal operating flywheel speed.

Generally, with increasing excess angular momentum, satellites should perform more poorly at precision pointing. However as shown in Figure 9-8 and Figure 9-12, both satellite without MTQ and satellite with MTQ were both still; effective at precision pointing throughout the entire simulation as both show zero steady state error in attitude and angular velocity error plot. This suggests that the current HCMG design is able to operate for many orbits before momentum dumping is necessary. This suggests that MTQ does not need to be active throughout the entire RW mode and can be scheduled to function periodically or when the excess angular momentum starts to affect satellite's precision pointing capability. This way, satellites will also save power and limit the current needed to generate the required dipole moment. Additionally, as shown in Figure 9-8 and Figure 9-12, the torque provided by MTQ is on the order of μNm, while the torque generated by the HCMH system is on the order of mNm. This suggests that the disturbances caused by MTQ will not affect satellite's pointing precision. Making MTQ suitable for small satellite applications with limited power supply.

However, MTQ's capability in producing torque heavily depends on satellite's location in orbit. According to (8-3), desaturation torque is the result from cross product of the magnetic dipole moment and Earth's magnetic field. Therefore, dipole moment component that is parallel to the magnetic field line will lead to zero torque, ultimately leading to no torque along the direction of the local geomagnetic field vector. This result is also shown in Figure 9-15, where MTQ dipole moment and MTQ torque oscillate about zero depending on where the satellite is in the orbit. Even with this limitation MTQ is still proven to be an effective desaturation method for CubeSats with HCMG cluster as the provided torque is still proven to be enough to slowly dump the excess momentum over time.

10) Appendix:

Appendix A

Atmospheric drag is computed using the exponential atmospheric density model from Vallado [13]. The atmosphere is divided into altitude bands. Within each band, density is modeled as decaying exponentially from a reference density with a band-specific scale height given as

$$\rho(h) = \rho_o e^{-\frac{(h_{alt} - h_o)}{H}}$$

(10-1)

where, for a given altitude h_{alt}, h_o is the base altitude of the band containing h_{alt}, ρ_o is the nominal density at h_o, and H is that band's scale height. The values of h_o, ρ_o, and H for each band are given below

variable	values
h_o	[0, 25e3, 30e3, 40e3, 50e3, 60e3, 70e3, 80e3, 90e3, 100e3, 110e3, 120e3, 130e3, 140e3, 150e3, 180e3, 200e3, 250e3, 300e3, 350e3, 400e3, 450e3, 500e3, 600e3, 700e3, 800e3, 900e3, 1000e3]
ρ_o	[1.225, 3.899e-2, 1.774e-2, 3.972e-3, 1.057e-3, 3.206e-4, 8.770e-5, 1.905e-5, 3.396e-6, 5.297e-7, 9.661e-8, 2.438e-8, 8.484e-9, 3.845e-9, 2.070e-9, 5.464e-10, 2.789e-10, 7.248e-11, 2.418e-11, 9.518e-12, 3.725e-12, 1.585e-12, 6.967e-13, 1.454e-13, 3.614e-14, 1.170e-14, 5.245e-15, 3.019e-15]
H	[7249, 6349, 6682, 7554, 8382, 7714, 6549, 5799, 5382, 5877, 7263, 9473, 12636, 16149, 22523, 29740, 37105, 45546, 53628, 53298, 58515, 60828, 63822, 71835, 88667, 124640, 181050, 268000]

Table 10-1: Values of h_o, ρ_o, and H for different altitude band

Appendix B

Singularity Surface Script

```
clear; clc; close all;

configs = {
   'Rooftop', [0, sin(pi/4), cos(pi/4); 0, -sin(pi/4), cos(pi/4); 0, sin(pi/4), cos(pi/4); 0, -sin(pi/4), cos(pi/4)];
   'Box', [1, 0, 0; -1, 0, 0; 0, 1, 0; 0, -1, 0];
   'Pyramid', [sin(54.74*pi/180), 0, cos(54.74*pi/180); 0, sin(54.74*pi/180), cos(54.74*pi/180); -sin(54.74*pi/180), 0, cos(54.74*pi/180); 0, -sin(54.74*pi/180), cos(54.74*pi/180)]
};

% Create Spherical Grid
```

```
[az, el] = meshgrid(linspace(0, 2*pi, 80), linspace(-pi/2, pi/2, 40));
ux = cos(el) .* cos(az);
uy = cos(el) .* sin(az);
uz = sin(el);

for c = 1:size(configs, 1)
    configName = configs{c, 1};
    g = configs{c, 2};

    % Setup Figure
    fig = figure('Name', configName, 'Color', 'w', 'Position', [50 + (c-1)*50, 100, 800, 400]);

    % Left Subplot Setup
    subplot(1, 2, 1);
    hold on; grid on; view(3); axis equal;
    xlabel('h_x'); ylabel('h_y'); zlabel('h_z');
    title([configName, ' - External Envelope']);
    set(gca, 'FontSize', 10, 'GridAlpha', 0.15);

    % Right Subplot Setup
    subplot(1, 2, 2);
    hold on; grid on; view(3); axis equal;
    xlabel('h_x'); ylabel('h_y'); zlabel('h_z');
    title([configName, ' - Internal Singularities']);
    set(gca, 'FontSize', 10, 'GridAlpha', 0.15);

    % Initialize arrays
    hx_all = []; hy_all = []; hz_all = [];

    % 4 Independent CMGs
    h_base = zeros(size(ux,1), size(ux,2), 3, 4);
    for i = 1:4
        gx = g(i,1); gy = g(i,2); gz = g(i,3);
        udotg = ux.*gx + uy.*gy + uz.*gz;
        px = ux - udotg .* gx;
        py = uy - udotg .* gy;
        pz = uz - udotg .* gz;
        norm_p = sqrt(px.^2 + py.^2 + pz.^2);
        norm_p(norm_p == 0) = 1e-6;
        h_base(:,:,1,i) = px ./ norm_p;
```

```
        h_base(:,:,2,i) = py ./ norm_p;
        h_base(:,:,3,i) = pz ./ norm_p;
    end

    epsilons = dec2bin(0:15) - '0';
    epsilons = epsilons * 2 - 1;

    for k = 1:16
        eps = epsilons(k, :);
        hx = eps(1)*h_base(:,:,1,1) + eps(2)*h_base(:,:,1,2) + eps(3)*h_base(:,:,1,3) +
eps(4)*h_base(:,:,1,4);
        hy = eps(1)*h_base(:,:,2,1) + eps(2)*h_base(:,:,2,2) + eps(3)*h_base(:,:,2,3) +
eps(4)*h_base(:,:,2,4);
        hz = eps(1)*h_base(:,:,3,1) + eps(2)*h_base(:,:,3,2) + eps(3)*h_base(:,:,3,3) +
eps(4)*h_base(:,:,3,4);

        % Collect points for the external envelope
        hx_all = [hx_all; hx(:)];
        hy_all = [hy_all; hy(:)];
        hz_all = [hz_all; hz(:)];

        % Plot ALL surfaces as internal structures
        subplot(1, 2, 2);
        surf(hx, hy, hz, hz, 'EdgeColor', 'none', 'FaceAlpha', 0.15);
    end

    % Generate External Envelope
    subplot(1, 2, 1);
    K = convhull(hx_all, hy_all, hz_all);

    % Plot the boundary mesh
    trisurf(K, hx_all, hy_all, hz_all, hz_all, 'EdgeColor', 'none', 'FaceAlpha', 0.95);

    for sp = 1:2
        subplot(1, 2, sp);
        colormap(gca, 'turbo');
        camlight('headlight');
        camlight('right');
        lighting gouraud;
        material shiny;
```

```
        view(45, 30);
    end
end
```

Appendix C

Atmospheric_density.m

```
function rho = atmosphere_density(h_m)
    h_table   = [0, 25e3, 30e3, 40e3, 50e3, 60e3, 70e3, 80e3, ...
                 90e3, 100e3, 110e3, 120e3, 130e3, 140e3, 150e3, ...
                 180e3, 200e3, 250e3, 300e3, 350e3, 400e3, 450e3, ...
                 500e3, 600e3, 700e3, 800e3, 900e3, 1000e3];
    rho_table = [1.225, 3.899e-2, 1.774e-2, 3.972e-3, 1.057e-3, ...
                 3.206e-4, 8.770e-5, 1.905e-5, 3.396e-6, 5.297e-7, ...
                 9.661e-8, 2.438e-8, 8.484e-9, 3.845e-9, 2.070e-9, ...
                 5.464e-10, 2.789e-10, 7.248e-11, 2.418e-11, 9.518e-12, ...
                 3.725e-12, 1.585e-12, 6.967e-13, 1.454e-13, 3.614e-14, ...
                 1.170e-14, 5.245e-15, 3.019e-15];
    H_table   = [7249, 6349, 6682, 7554, 8382, 7714, 6549, 5799, ...
                 5382, 5877, 7263, 9473, 12636, 16149, 22523, ...
                 29740, 37105, 45546, 53628, 53298, 58515, 60828, ...
                 63822, 71835, 88667, 124640, 181050, 268000];
    h_m = max(0, min(h_m, 1000e3));
    idx = find(h_table <= h_m, 1, 'last');
    if idx >= length(h_table)
        idx = length(h_table) - 1;
    end
    rho = rho_table(idx) * exp(-(h_m - h_table(idx)) / H_table(idx));
end
```

Attitude_dynamics.m

```
function x_dot = attitude_dynamics(x, I, tau, h)
    E = [x(1); x(2); x(3)];
    n = x(4);
    w = [x(5); x(6); x(7)];
    E_dot = (0.5 * skew(E) * w) + (0.5 * n * w);
    n_dot = (-0.5 * w' * E);
    w_dot = I \ (-tau - skew(w)*(I*w + h));
    x_dot = [E_dot; n_dot; w_dot];
```

```
end
```

Compute_J2.m

```
function a_J2 = compute_J2(x)
   mu      = 3.986004418e14;
   J2      = 1.08263e-3;
   R_earth = 6378137;
   r       = x(1:3);
   z       = x(3);
   r_mag   = norm(r);
   factor  = -1.5 * J2 * mu * R_earth^2 / r_mag^4;
   a_J2 = factor * [(1 - 5*(z/r_mag)^2) * r(1)/r_mag;
                    (1 - 5*(z/r_mag)^2) * r(2)/r_mag;
                    (3 - 5*(z/r_mag)^2) * r(3)/r_mag];
end
```

Compute_aero.m

```
function [F_drag_body, a_drag_ECI, tau_drag_body] = compute_aero(q_curr, v_ECI, r_ECI,
sat)

   %% Altitude and density
   R_earth = 6378137;
   h_alt   = norm(r_ECI) - R_earth;
   rho     = atmosphere_density(h_alt);

   %% Velocity in body frame
   R       = quat2rotm_body(q_curr);   % ECI to body
   v_body  = R' * v_ECI;               % body
   v_mag   = norm(v_body);

   if v_mag < 1e-6
      F_drag_body = zeros(3,1);
      tau_drag_body    = zeros(3,1);
      return;
   end
   v_hat = v_body / v_mag;

   %% Sum over all faces
   F_drag_body = zeros(3,1);
   tau_drag_body    = zeros(3,1);
```

```
    for k = 1:6
        n_hat     = sat.faces(k).normal;
        A_face    = sat.faces(k).area;
        cop       = sat.faces(k).cop - sat.CoM;

        % Only exposed faces
        cos_theta = dot(v_hat, n_hat);
        if cos_theta <= 0
            continue;
        end

        % Drag force
        dF = -0.5 * rho * sat.Cd * A_face * v_mag^2 * cos_theta * v_hat;

        F_drag_body = F_drag_body + dF;
        tau_drag_body    = tau_drag_body + cross(cop, dF);
    end

    F_drag_ECI = R * F_drag_body;    % body to ECI
    a_drag_ECI = F_drag_ECI / sat.m;
end
```

Compute_mtq.m

```
function [tau_mtq, m_dipole] = compute_mtq(h_excess, sat, B_body)
    B_mag2   = dot(B_body, B_body);
    if B_mag2 < 1e-20
        tau_mtq  = zeros(3,1);
        m_dipole = zeros(3,1);
        return;
    end
    m_dipole = sat.mtq.k_desat * cross(h_excess, B_body) / B_mag2;
    % clamp
    for k = 1:3
        m_dipole(k) = max(-sat.mtq.m_max, min(sat.mtq.m_max, m_dipole(k))); % Am²
    end
    % torque
    tau_mtq = cross(m_dipole, B_body); % Nm
end
```

Compute_qd.m

```
function [qd, wd, wd_dot, Rd] = compute_qd(r_ECI, v_ECI)
    r_mag = norm(r_ECI);
    r_hat = r_ECI / r_mag;
    h_vec = cross(r_ECI, v_ECI);
    h_mag = norm(h_vec);
    h_hat = h_vec / h_mag;
    yd = -r_hat;
    zd = -h_hat;
    xd = cross(yd, zd);
    xd = xd / norm(xd);
    Rd = [xd yd zd];
    qd = rotm2quat_scalar_last(Rd);
    omega_mag = h_mag / r_mag^2;
    wd = [0;
          0;
         -omega_mag];
    r_dot = dot(r_ECI, v_ECI) / r_mag;
    omega_dot_mag = -2*(r_dot/r_mag)*omega_mag;
    wd_dot = [0;
              0;
             -omega_dot_mag];
end
```

Earth_magnetic_field.m

```
function B_ECI = earth_magnetic_field2(r_ECI, t)
    mu_mag      = 7.94e22;
    mu0         = 4*pi*1e-7;
    gamma       = deg2rad(11.3);
    omega_earth = 7.2921159e-5;
    m_hat_ECEF = - [sin(gamma); 0; cos(gamma)];
    theta = omega_earth * t;
    R_z = [cos(theta) -sin(theta) 0;
           sin(theta)  cos(theta) 0;
           0           0          1];
    m_hat = R_z * m_hat_ECEF;
    r_mag = norm(r_ECI);
    r_hat = r_ECI / r_mag;
    B_ECI = (mu0*mu_mag/(4*pi*r_mag^3)) * ...
            (3*dot(m_hat, r_hat)*r_hat - m_hat);
```

```
end
```

ECI2BODY.m

```
function r_I_B = ECI2BODY(q, nadir_I)
    qx = q(1);
    qy = q(2);
    qz = q(3);
    qw = q(4);
    R = [1 - 2*(qy^2 + qz^2), 2*(qx*qy + qw*qz),   2*(qx*qz - qw*qy);
         2*(qx*qy - qw*qz),   1 - 2*(qx^2 + qz^2), 2*(qy*qz + qw*qx);
         2*(qx*qz + qw*qy),   2*(qy*qz - qw*qx),   1 - 2*(qx^2 + qy^2)];
    r_I_B = R * nadir_I;
end
```

Eigenaxis_controller.m

```
function [torque, Ee, ne, Re_bd] = eigenaxis_controller(q, qd, w, wd, wd_dot, J, Kp, Kd, h)
    E  = q(1:3);    n  = q(4);
    Ed = qd(1:3);   nd = qd(4);
    Matrix = [nd*eye(3) - skew(Ed), -Ed;
              Ed',                  nd];
    result = Matrix * [E; n];
    Ee = result(1:3);
    ne = result(4);
    if ne < 0
       Ee = -Ee;
       ne = -ne;
    end
    Re_bd  = quat2rotm_body(q)' * quat2rotm_body(qd);
    we     = w - Re_bd * wd;
    wd_dot = Re_bd * wd_dot;

    torque = -skew(w)*(J*w + h) - J*wd_dot + Kp*J*Ee + Kd*J*we;
end
```

Eul2quat.m

```
function q = eul2quat(roll, pitch, yaw)
    cr = cos(roll / 2);
    sr = sin(roll / 2);
    cp = cos(pitch / 2);
```

```
    sp = sin(pitch / 2);
    cy = cos(yaw / 2);
    sy = sin(yaw / 2);
    qx = sr * cp * cy - cr * sp * sy;
    qy = cr * sp * cy + sr * cp * sy;
    qz = cr * cp * sy - sr * sp * cy;
    qw = cr * cp * cy + sr * sp * sy;
    q = [qx; qy; qz; qw];
    q = q / norm(q);
end
```

Main.m

```
    %% Main
    %% By: Krittin Jindamai
clc; clear;

%% Orbital Parameters
mu = 3.986004418e14;
Re = 6378137; % m
a    = Re + 500e3;
e = 0.01;
i = deg2rad(97.6);  % rad
RAAN = deg2rad(45); % rad
aop  = deg2rad(45); % rad
f    = 0;           % rad
n_orbit = sqrt(mu / a^3); % rad/s
orbit_param = [a,e,i,RAAN,aop,f];

%% Initial Orbit State
r0_PQW = (a*(1-e^2) / (1+(e*cos(f)))) * [cos(f);sin(f);0]; % m
r0_ECI = PQW2ECI(r0_PQW, orbit_param); % m
v0_PQW = (sqrt(mu/(a*(1-e^2)))) * [-sin(f);e+cos(f);0]; % m/s
v0_ECI = PQW2ECI(v0_PQW, orbit_param); % m/s
x_orbit = [r0_ECI; v0_ECI];

%% Simulation Time
T_one_orbit = 2*pi*sqrt(a^3/mu); % s
T = 15 * T_one_orbit;            % s
tspan = linspace(0,T,T*5);
dt  = tspan(2) - tspan(1);
```

```
%% Initial Attitude State
q0   = eul2quat(0,0,0);        % [0;0;0;1]
w0_B = [0;0;-0];               % rad/s
x0_att = [q0; w0_B];
x_att = x0_att;

%% Controller Parameters
[qd,wd_LVLH,wd_dot_LVLH,Rd] = compute_qd(r0_ECI,v0_ECI); % LVLH
R = quat2rotm_body(q0);
wd = R' * Rd * wd_LVLH; % body
wd_dot = R' * Rd * wd_dot_LVLH; % body

Kp = 0.3;
Kd = 1;

%% Actuator Parameters
Iw    = 42e-6;                 % kgm^2
theta = deg2rad(54.74);        % rad
delta_dot_max = deg2rad(60);   % deg/s
Omega_dot_max = 55;            % rad/s2

% Initial gimbal angles
delta = [deg2rad(211); deg2rad(-31); deg2rad(211); deg2rad(-31)]; % rad

% flywheel speed parameter
Omega     = [838;838;838;838]; % rad/s (8000 RPM)
Omega_nom = [838;838;838;838]; % rad/s (8000 RPM)
Omega_max_scalar = 2000;       % rad/s
Omega_max =
[Omega_max_scalar;Omega_max_scalar;Omega_max_scalar;Omega_max_scalar];
Omega_actual  = Omega;

%% Steering Logic Parameters
lambda0 = 0.1;    mu1 = 5;
beta0   = 0.1;    mu2 = 5;
alpha = 1;

%% Satellite Geometry  (12U / 230x240x360mm)
sat.Lx = 0.240;   % m
```

```
sat.Ly = 0.230;   % m
sat.Lz = 0.360;   % m
sat.m  = 24;      % kg

% Center of Mass
sat.CoM = [0; 0; 0.05];  % m

% Face definitions
sat.faces(1).normal = [ 1;0;0]; sat.faces(1).area = sat.Ly*sat.Lz; sat.faces(1).cop = [
sat.Lx/2;0;0];
sat.faces(2).normal = [-1;0;0]; sat.faces(2).area = sat.Ly*sat.Lz; sat.faces(2).cop = [-
sat.Lx/2;0;0];
sat.faces(3).normal = [ 0;1;0]; sat.faces(3).area = sat.Lx*sat.Lz; sat.faces(3).cop = [0;
sat.Ly/2;0];
sat.faces(4).normal = [ 0;-1;0]; sat.faces(4).area = sat.Lx*sat.Lz; sat.faces(4).cop = [0;-
sat.Ly/2;0];
sat.faces(5).normal = [ 0;0;1]; sat.faces(5).area = sat.Lx*sat.Ly; sat.faces(5).cop = [0;0;
sat.Lz/2];
sat.faces(6).normal = [ 0;0;-1]; sat.faces(6).area = sat.Lx*sat.Ly; sat.faces(6).cop = [0;0;-
sat.Lz/2];

% Drag coefficient
sat.Cd = 2.2;

% Inertia (J = diag([0.38, 0.37, 0.22]) ) kg/m2
Ixx_gc = sat.m/12 * (sat.Ly^2 + sat.Lz^2);
Iyy_gc = sat.m/12 * (sat.Lx^2 + sat.Lz^2);
Izz_gc = sat.m/12 * (sat.Lx^2 + sat.Ly^2);
J_gc   = diag([Ixx_gc, Iyy_gc, Izz_gc]);

d = sat.CoM;
Jc = J_gc - sat.m*(dot(d,d)*eye(3) - d*d'); % J about COM

% parallel axis for gimbals
m_cmg = 0.4; % kg
R_cmg1 = [0.01; 0; 0.1] - d;   % m
R_cmg2 = [-0.01; 0; 0.1] - d;  % m
R_cmg3 = [0; 0.01; 0.1] - d;   % m
R_cmg4 = [0; -0.01; 0.1] - d;  % m
R_cmg = [R_cmg1';R_cmg2';R_cmg3';R_cmg4'];
```

```
J = Jc;

for ii = 1:4
   r = R_cmg(ii, 1:3)';
   J_sum = m_cmg * ( (r' * r) * eye(3) - (r * r') );
   J = J + J_sum;
end

%% MTQ parameter
sat.mtq.N         = 200;                    % turns
sat.mtq.I_max     = 0.4;                      % A
sat.mtq.A         = 0.02;                   % m2
sat.mtq.m_max     = sat.mtq.N * sat.mtq.I_max * sat.mtq.A;   % Am2
sat.mtq.k_desat  = 3e-4;                              % gain Am2/Nms
sat.mtq.Omega_nom = Omega_nom;  % nominal flywheel speed rad/s

%% Scenario Switches
enable_mtq_desaturation = false;

%% Storage Initialization
N = length(tspan);
x_att_store       = zeros(7, N);
x_att_store(:,1)  = x0_att;
q_store           = zeros(4, N);
q_store(:,1)      = q0;
wd_store          = zeros(3, N);
wd_store(:,1)     = wd;
Omega_store       = zeros(4, N);
Omega_store(:,1)  = Omega;
Omega_dot_store_actual  = zeros(4, N);
Omega_dot_store_cmd  = zeros(4, N);
delta_store       = zeros(4, N);
delta_store(:,1)  = delta;
delta_dot_store_actual  = zeros(4, N);
delta_dot_store_cmd = zeros(4, N);
tau_cmd_store     = zeros(3, N);
tau_store         = zeros(3, N);
S_CMG_store       = zeros(1, N);
S_RW_store        = zeros(1, N);
alpha_store       = zeros(1, N);
```

```
Ee_store          = zeros(3, N);
ne_store          = zeros(1, N);
null_store        = zeros(4, N);
null_motion_store = zeros(4, N);
tau_dist_store    = zeros(3,N);
tau_gg_store      = zeros(3,N);
tau_aero_store    = zeros(3,N);
r_ECI_store       = zeros(3, N);
r_ECI_store(:,1)  = r0_ECI;
x_orbit_store = zeros(6, N);
x_orbit_store(:,1) = x_orbit;
tau_mtq_store    = zeros(3, N);
m_dipole_store   = zeros(3, N);
B_ECI_store      = zeros(3,N);
h_fw_store       = zeros(3,N);
h_excess_store   = zeros(3,N);
qd_store         = zeros(4,N);

%% Main Simulation Loop
for jj = 1:N-1

    %% 1. Extract current states
    t_current = tspan(jj);
    E_curr = x_att(1:3);
    n_curr = x_att(4);
    w_curr = x_att(5:7);
    q_curr = [E_curr; n_curr];
    R      = quat2rotm_body(q_curr);   % body -> ECI (current attitude)

    %% 2. Orbit state
    r_ECI     = x_orbit(1:3);
    v_ECI     = x_orbit(4:6);
    r_mag     = norm(r_ECI);
    r_ECI_mag = r_mag;
    nadir_I   = -r_ECI / r_mag;

    %% 3. Compute A, B, and S
    A     = matA(theta, delta);
    B     = matB(theta, delta);
    S_CMG = det(A*A');
```

```
S_RW  = det(B*B');

%% 4. Angular momentum using Actual flywheel speed
h_fw_B = Iw * B * Omega_actual;
h_nom = Iw * B * Omega_nom;
h_max = Iw * B * Omega_max;

%% 5 Disturbances
% Gravity gradient
radial_I = r_ECI / r_ECI_mag;
o3_body  = ECI2BODY(q_curr, radial_I);
tau_gg   = 3*mu/r_mag^3 * cross(o3_body, J*o3_body);

% Aerodynamic drag
[F_drag_body, a_drag, tau_aero] = compute_aero(q_curr, v_ECI, r_ECI, sat);

% J2
a_J2 = compute_J2(x_orbit);

% Total disturbance
tau_dist = tau_gg + tau_aero;

%% 6. Compute new qd, wd, wd_dot
[qd,wd_LVLH,wd_dot_LVLH,Rd] = compute_qd(r_ECI,v_ECI); % LVLH

% prevent qd from flipping sign midway
if jj > 1
   if dot(qd, qd_prev) < 0
      qd = -qd;
   end
end
qd_prev = qd;

wd     = R' * Rd * wd_LVLH;
wd_dot = R' * Rd * wd_dot_LVLH;

%% 7. Mode switch
if t_current < 25
   alpha = 1;
elseif t_current <= 35
```

```
    alpha = 0.5 * (1 + cos(pi*(t_current-25)/10));
  else
    alpha = 0;
  end

  %% 8. MTQ desaturation
  B_ECI = earth_magnetic_field2(x_orbit(1:3),t_current);
  B_body = R' * B_ECI;              % ECI to body

  h_excess = h_fw_B - h_nom;

  if enable_mtq_desaturation
    [tau_mtq, m_dipole] = compute_mtq(h_excess, sat, B_body);

    % MTQ always on while in RW mode (for now); off during CMG mode.
    if alpha ~= 0
      tau_mtq  = [0;0;0];
      m_dipole = [0;0;0];
    end
  else
    % Desaturation disabled (see enable_mtq_desaturation switch above).
    tau_mtq  = [0;0;0];
    m_dipole = [0;0;0];
  end

  %% 9. Eigen Axis Tracking Controller
  [tau_cmd, Ee, ne] = eigenaxis_controller(q_curr, qd, w_curr, wd_LVLH, wd_dot_LVLH, J,
Kp, Kd, h_fw_B);

  %% 10. Steering law
  if alpha < 0.01
    d = zeros(4,1);  % no null motion in RW mode
  else
    d = null(A);
  end

  [lambda_val, beta_val, delta_dot_cmd, Omega_dot_cmd] = steering_logic(t_current, lambda0,
mu1, S_CMG, ...
    beta0, mu2, S_RW, Iw, Omega_actual, alpha, A, B, tau_cmd, d);
```

```
%% 11. Actuator saturation
% Gimbal
delta_dot_actual = max(-delta_dot_max, min(delta_dot_max, delta_dot_cmd));
% Flywheel
Omega_dot_actual = max(-Omega_dot_max, min(Omega_dot_max, Omega_dot_cmd));

%% 12. Actual torque from actuator outputs
tau = (Iw * A * diag(Omega_actual) * delta_dot_actual) + (Iw * B * Omega_dot_actual);
tau_total = tau - tau_mtq - tau_dist;

%% 13. Attitude dynamics with actual torque
x_att_dot = attitude_dynamics(x_att, J, tau_total, h_fw_B);

%% 14. Integrate attitude (quaternion)
E_next = E_curr + x_att_dot(1:3) * dt;
n_next = n_curr + x_att_dot(4)   * dt;
w_next = w_curr + x_att_dot(5:7) * dt;

% Renormalize quaternion
q_norm = norm([E_next; n_next]);
E_next = E_next / q_norm;
n_next = n_next / q_norm;
x_att  = [E_next; n_next; w_next];

%% 15. Update gimbal angles and flywheel speed
delta  = delta + delta_dot_actual * dt;
Omega_actual = Omega_actual + Omega_dot_actual * dt;
Omega_actual = max(-Omega_max, min(Omega_max, Omega_actual));
Omega = Omega_actual;

%% 16. Update Orbit
x_orbit_dot = orbit_dynamics(x_orbit, a_drag, a_J2);
x_orbit(4:6) = x_orbit(4:6) + x_orbit_dot(4:6) * dt;
x_orbit(1:3) = x_orbit(1:3) + x_orbit(4:6) * dt;

%% 17. Store at jj+1
q_store(:,jj+1)         = [E_next; n_next];
x_att_store(:,jj+1)     = x_att;
wd_store(:,jj+1)        = wd;
Omega_store(:,jj+1)     = Omega;
```

```
    Omega_dot_store_actual(:,jj+1)  = Omega_dot_actual;
    Omega_dot_store_cmd(:,jj+1)  = Omega_dot_cmd;
    delta_store(:,jj+1)      = delta;
    delta_dot_store_actual(:,jj+1)  = delta_dot_actual;
    delta_dot_store_cmd(:,jj+1)  = delta_dot_cmd;
    tau_cmd_store(:,jj+1)    = tau_cmd;
    tau_store(:,jj+1)        = tau;
    S_CMG_store(jj+1)        = S_CMG;
    S_RW_store(jj+1)         = S_RW;
    alpha_store(jj+1)        = alpha;
    Ee_store(:,jj+1)         = Ee;
    ne_store(jj+1)           = ne;
    null_store(:,jj+1)       = d;
    null_motion_store(:,jj+1) = beta_val * d;
    tau_dist_store(:,jj+1) = tau_dist;
    tau_gg_store(:,jj+1) = tau_gg;
    tau_aero_store(:,jj+1) = tau_aero;
    x_orbit_store(:,jj+1) = x_orbit;
    tau_mtq_store(:,jj+1)  = tau_mtq;
    m_dipole_store(:,jj+1) = m_dipole;
    B_ECI_store(:,jj+1) = B_ECI;
    h_excess_store(:,jj+1) = h_excess;
    h_fw_store(:,jj+1)     = h_fw_B;
    qd_store(:,jj+1) = qd;
end

%% Plots
lw = 1.5; % consistent line width across every figure

figure(1); % Attitude
tiledlayout(2,1,'TileSpacing','compact');

nexttile; % Error quaternion
plot(tspan, Ee_store(1,:), 'r-', tspan, Ee_store(2,:), 'g-', ...
     tspan, Ee_store(3,:), 'b-', tspan, ne_store, 'k-', 'LineWidth', lw);
grid on; ylabel('Error Quaternion Component [-]');
title('Attitude Error Quaternion');
legend('\epsilon_{e1}','\epsilon_{e2}','\epsilon_{e3}','\eta_e');
ylim([-1.1 1.1]);
```

```
nexttile; % Euler angles
euler_store = rad2deg(quat2eul(q_store));
plot(tspan, euler_store(1,:), 'r-', tspan, euler_store(2,:), 'g-', ...
    tspan, euler_store(3,:), 'b-', 'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('Euler Angle [deg]');
title('Attitude Euler Angles');
legend('roll','pitch','yaw');

figure(2); % Angular Velocity
tiledlayout(2,1,'TileSpacing','compact');

nexttile;
plot(tspan, rad2deg(x_att_store(5,:)), 'r-', ...
    tspan, rad2deg(x_att_store(6,:)), 'g-', ...
    tspan, rad2deg(x_att_store(7,:)), 'b-', 'LineWidth', lw);
grid on; ylabel('Angular Velocity [deg/s]');
title('Satellite Angular Velocity');
legend('\omega_x','\omega_y','\omega_z');

nexttile;
plot(tspan, rad2deg(x_att_store(5,:)-wd_store(1,:)), 'r-', ...
    tspan, rad2deg(x_att_store(6,:)-wd_store(2,:)), 'g-', ...
    tspan, rad2deg(x_att_store(7,:)-wd_store(3,:)), 'b-', 'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('Angular Velocity Error [deg/s]');
title('Satellite Angular Velocity Error');
legend('\omega_x','\omega_y','\omega_z');

figure(3); % Gimbal
tiledlayout(2,1,'TileSpacing','compact');

nexttile;
plot(tspan, rad2deg(delta_dot_store_actual(1,:)), 'r-', ...
    tspan, rad2deg(delta_dot_store_actual(2,:)), 'g-', ...
    tspan, rad2deg(delta_dot_store_actual(3,:)), 'b-', ...
    tspan, rad2deg(delta_dot_store_actual(4,:)), 'k-', 'LineWidth', lw);
grid on; ylabel('Gimbal Rate [deg/s]');
title('Gimbal Rates (Actual)');
legend('\delta_1','\delta_2','\delta_3','\delta_4');

nexttile;
```

```
plot(tspan, rad2deg(delta_store(1,:)), 'r-', ...
    tspan, rad2deg(delta_store(2,:)), 'g-', ...
    tspan, rad2deg(delta_store(3,:)), 'b-', ...
    tspan, rad2deg(delta_store(4,:)), 'k-', 'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('Gimbal Angle [deg]');
title('Gimbal Angles');
legend('\delta_1','\delta_2','\delta_3','\delta_4');

figure(4); % Reaction Wheel
tiledlayout(2,1,'TileSpacing','compact');

nexttile;
plot(tspan, Omega_dot_store_actual(1,:), 'r-', ...
    tspan, Omega_dot_store_actual(2,:), 'g-', ...
    tspan, Omega_dot_store_actual(3,:), 'b-', ...
    tspan, Omega_dot_store_actual(4,:), 'k-', 'LineWidth', lw);
grid on; ylabel('Flywheel Acceleration [rad/s^2]');
title('Flywheel Acceleration');
legend('\Omega_1','\Omega_2','\Omega_3','\Omega_4');

nexttile;
plot(tspan, Omega_store(1,:), 'r-', ...
    tspan, Omega_store(2,:), 'g-', ...
    tspan, Omega_store(3,:), 'b-', ...
    tspan, Omega_store(4,:), 'k-', 'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('Flywheel Speed [rad/s]');
title('Flywheel Speed');
legend('\Omega_1','\Omega_2','\Omega_3','\Omega_4');

figure(5); % Torque HCMG
plot(tspan, tau_cmd_store(1,:)*1000, 'r--', ...
    tspan, tau_cmd_store(2,:)*1000, 'g--', ...
    tspan, tau_cmd_store(3,:)*1000, 'b--', ...
    tspan, tau_store(1,:)*1000,     'r-',  ...
    tspan, tau_store(2,:)*1000,     'g-',  ...
    tspan, tau_store(3,:)*1000,     'b-',  'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('Torque [mNm]');
title('Commanded vs Actual HCMG Torque');
legend('\tau_{cmd,x}','\tau_{cmd,y}','\tau_{cmd,z}', ...
      '\tau_{act,x}','\tau_{act,y}','\tau_{act,z}');
```

```
figure(6); % Sing Params
tiledlayout(2,1,'TileSpacing','compact');

nexttile;
plot(tspan, S_CMG_store, 'b-', tspan, S_RW_store, 'r-', 'LineWidth', lw);
grid on; ylabel('Singularity Parameter [-]');
title('Singularity Parameters');
legend('S_{CMG}','S_{RW}');

nexttile; % Null motion
plot(tspan, rad2deg(null_motion_store(1,:)), 'r-', ...
    tspan, rad2deg(null_motion_store(2,:)), 'g-', ...
    tspan, rad2deg(null_motion_store(3,:)), 'b-', ...
    tspan, rad2deg(null_motion_store(4,:)), 'k-', 'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('Null Motion Rate [deg/s]');
title('Null Motion');
legend('HCMG 1','HCMG 2','HCMG 3','HCMG 4');

figure(7); % Mode Switch Param
plot(tspan, alpha_store, 'k-', 'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('\alpha [-]');
title('Mode Switch Parameter \alpha');
ylim([-0.1 1.1]);

figure(8); % Disturbances
tiledlayout(3,1,'TileSpacing','compact');

nexttile;
plot(tspan, tau_dist_store(1,:)*1e6,'r-', tspan, tau_dist_store(2,:)*1e6,'g-', ...
    tspan, tau_dist_store(3,:)*1e6,'b-','LineWidth',lw);
grid on; ylabel('Torque [\muNm]'); title('Total Disturbance');
legend('\tau_x','\tau_y','\tau_z');

nexttile;
plot(tspan, tau_gg_store(1,:)*1e6,'r-', tspan, tau_gg_store(2,:)*1e6,'g-', ...
    tspan, tau_gg_store(3,:)*1e6,'b-','LineWidth',lw);
grid on; ylabel('Torque [\muNm]'); title('Gravity Gradient');
legend('\tau_x','\tau_y','\tau_z');
```

```
nexttile;
plot(tspan, tau_aero_store(1,:)*1e6,'r-', tspan, tau_aero_store(2,:)*1e6,'g-', ...
    tspan, tau_aero_store(3,:)*1e6,'b-','LineWidth',lw);
grid on; xlabel('Time [s]'); ylabel('Torque [\muNm]'); title('Aerodynamic Drag');
legend('\tau_x','\tau_y','\tau_z');

figure(9); % flywheel angular moomentum + mtq
h_excess_mag_store = vecnorm(h_excess_store);

subplot(4,1,1); % excess h
plot(tspan, h_excess_store(1,:), 'r-', ...
    tspan, h_excess_store(2,:), 'g-', ...
    tspan, h_excess_store(3,:), 'b-', 'LineWidth', 1.5);
grid on; xlabel('Time [s]'); ylabel('[kg\cdotm^2/s]');
title('Excess Angular Momentum');
legend('h_x','h_y','h_z');

subplot(4,1,2)
plot(tspan,h_excess_mag_store,'k','LineWidth',2);
title('Excess Angular Momentum Magnitude');
grid on; xlabel('Time [s]'); ylabel('[kg\cdotm^2/s]');
hold on
%yline(0.7*norm(h_max),'g--','70%');
yline(norm(h_max),'k--','h_{max}');

subplot(4,1,3); % dipole moment
plot(tspan, m_dipole_store(1,:), 'r-', ...
    tspan, m_dipole_store(2,:), 'g-', ...
    tspan, m_dipole_store(3,:), 'b-', 'LineWidth', 1.5);
yline(sat.mtq.m_max,  'k--', 'Max');
yline(-sat.mtq.m_max, 'k--');
grid on; xlabel('Time [s]'); ylabel('[Am²]');
title('MTQ Dipole Moment Command');
legend('m_x','m_y','m_z');

subplot(4,1,4); % desat torque
plot(tspan, tau_mtq_store(1,:)*1e6, 'r-', ...
    tspan, tau_mtq_store(2,:)*1e6, 'g-', ...
    tspan, tau_mtq_store(3,:)*1e6, 'b-', 'LineWidth', 1.5);
grid on; xlabel('Time [s]'); ylabel('[\muNm]');
```

```
title('MTQ Desaturation Torque');
legend('\tau_{mtq,x}','\tau_{mtq,y}','\tau_{mtq,z}');

%% 3D
figure(10);
plot3(x_orbit_store(1,:)/1000, x_orbit_store(2,:)/1000, x_orbit_store(3,:)/1000, ...
    'b-', 'LineWidth', 2);
hold on;
grid on;
plot3(x_orbit_store(1,1)/1000, x_orbit_store(2,1)/1000, x_orbit_store(3,1)/1000, ...
    'go', 'MarkerSize', 8, 'MarkerFaceColor', 'g');
plot3(x_orbit_store(1,end)/1000, x_orbit_store(2,end)/1000, x_orbit_store(3,end)/1000, ...
    'rx', 'MarkerSize', 8, 'LineWidth', 2);
xlabel('X_{ECI} [km]');
ylabel('Y_{ECI} [km]');
zlabel('Z_{ECI} [km]');
title('3D Satellite Orbital Path (ECI Frame)');
legend('Satellite Trajectory', 'Orbit Start', 'Orbit End', 'Location', 'best');
axis equal;
view(3);
hold off;

figure(11); % Position and Velocity
subplot(2,1,1); % pos
alt_store = vecnorm(x_orbit_store(1:3,:)) - 6378137;
plot(tspan, alt_store/1000, 'b-', 'LineWidth', 1.5);
grid on; xlabel('Time [s]'); ylabel('Altitude [km]');
title('Orbital Altitude');

subplot(2,1,2); % vel
vel_store = vecnorm(x_orbit_store(4:6,:));
plot(tspan, vel_store, 'r-', 'LineWidth', 1.5);
grid on; xlabel('Time [s]'); ylabel('Speed [m/s]');
title('Orbital Speed');

%% HCMG mode
idx100 = tspan <= 100;
t100   = tspan(idx100);

figure(12); % HCMG Gimbal & Flywheel Actuation (zoomed)
```

```
tiledlayout(2,2,'TileSpacing','compact');

nexttile; % Gimbal rate
plot(t100, rad2deg(delta_dot_store_actual(1,idx100)), 'r-', ...
    t100, rad2deg(delta_dot_store_actual(2,idx100)), 'g-', ...
    t100, rad2deg(delta_dot_store_actual(3,idx100)), 'b-', ...
    t100, rad2deg(delta_dot_store_actual(4,idx100)), 'k-', 'LineWidth', lw);
grid on; ylabel('Gimbal Rate [deg/s]');
legend('\delta_1','\delta_2','\delta_3','\delta_4');

nexttile; % Gimbal angle
plot(t100, rad2deg(delta_store(1,idx100)), 'r-', ...
    t100, rad2deg(delta_store(2,idx100)), 'g-', ...
    t100, rad2deg(delta_store(3,idx100)), 'b-', ...
    t100, rad2deg(delta_store(4,idx100)), 'k-', 'LineWidth', lw);
grid on; ylabel('Gimbal Angle [deg]');
legend('\delta_1','\delta_2','\delta_3','\delta_4');

nexttile; % Flywheel accel
plot(t100, Omega_dot_store_actual(1,idx100), 'r-', ...
    t100, Omega_dot_store_actual(2,idx100), 'g-', ...
    t100, Omega_dot_store_actual(3,idx100), 'b-', ...
    t100, Omega_dot_store_actual(4,idx100), 'k-', 'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('Flywheel Acceleration [rad/s^2]');
legend('\Omega_1','\Omega_2','\Omega_3','\Omega_4');

nexttile; % Flywheel speed
plot(t100, Omega_store(1,idx100), 'r-', ...
    t100, Omega_store(2,idx100), 'g-', ...
    t100, Omega_store(3,idx100), 'b-', ...
    t100, Omega_store(4,idx100), 'k-', 'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('Flywheel Speed [rad/s]');
legend('\Omega_1','\Omega_2','\Omega_3','\Omega_4');

sgtitle('HCMG Gimbal & Flywheel Actuation (First 100 s)');

figure(13); % HCMG Attitude Error, Torque, Singularity & Mode (zoomed)
tiledlayout(4,1,'TileSpacing','compact');

nexttile; % Attitude error quaternion
```

```
plot(t100, Ee_store(1,idx100), 'r-', t100, Ee_store(2,idx100), 'g-', ...
    t100, Ee_store(3,idx100), 'b-', t100, ne_store(idx100), 'k-', 'LineWidth', lw);
grid on; ylabel('Error Quaternion Component [-]');
legend('\epsilon_{e1}','\epsilon_{e2}','\epsilon_{e3}','\eta_e');

nexttile; % Torque
plot(t100, tau_cmd_store(1,idx100)*1000, 'r--', ...
    t100, tau_cmd_store(2,idx100)*1000, 'g--', ...
    t100, tau_cmd_store(3,idx100)*1000, 'b--', ...
    t100, tau_store(1,idx100)*1000,     'r-',  ...
    t100, tau_store(2,idx100)*1000,     'g-',  ...
    t100, tau_store(3,idx100)*1000,     'b-',  'LineWidth', lw);
grid on; ylabel('Torque [mNm]');
title('Commanded vs Actual HCMG Torque');
legend('\tau_{cmd,x}','\tau_{cmd,y}','\tau_{cmd,z}', ...
      '\tau_{act,x}','\tau_{act,y}','\tau_{act,z}');

nexttile; % Singularity parameters
plot(t100, S_CMG_store(idx100), 'b-', t100, S_RW_store(idx100), 'r-', 'LineWidth', lw);
grid on; ylabel('Singularity Parameter [-]');
title('Singularity Parameters');
legend('S_{CMG}','S_{RW}');

nexttile; % Mode switch
plot(t100, alpha_store(idx100), 'k-', 'LineWidth', lw);
grid on; xlabel('Time [s]'); ylabel('\alpha [-]');
title('Mode Switch Parameter \alpha');
ylim([-0.1 1.1]);

sgtitle('HCMG Attitude Error, Torque & Singularity (First 100 s)');
```

matA.m

```
function A = matA(theta, delta)
   delta1 = delta(1);
   delta2 = delta(2);
   delta3 = delta(3);
   delta4 = delta(4);
   A = [-cos(theta)*sin(delta1), -cos(delta2),            cos(theta)*sin(delta3),  cos(delta4);
        cos(delta1),          -cos(theta)*sin(delta2),  -cos(delta3),            cos(theta)*sin(delta4);
```

```
        sin(theta)*sin(delta1), sin(theta)*sin(delta2),   sin(theta)*sin(delta3),
sin(theta)*sin(delta4)];
end
```

matB.m

```
function B = matB(theta, delta)
   delta1 = delta(1);
   delta2 = delta(2);
   delta3 = delta(3);
   delta4 = delta(4);
   B = [ cos(theta)*cos(delta1), -sin(delta2),           -cos(theta)*cos(delta3),  sin(delta4);
         sin(delta1),            cos(theta)*cos(delta2), -sin(delta3),           -cos(theta)*cos(delta4);
        -sin(theta)*cos(delta1), -sin(theta)*cos(delta2), -sin(theta)*cos(delta3), -
sin(theta)*cos(delta4)];
end
```

orbit_dynamics.m

```
function xdot = orbit_dynamics(x, a_drag, a_J2)
   mu = 3.986004418e14;
   rx = x(1);
   ry = x(2);
   rz = x(3);
   vx = x(4);
   vy = x(5);
   vz = x(6);
   v = [vx; vy; vz];
   r = sqrt(rx^2 + ry^2 + rz^2);
   ax = -mu * rx / r^3;
   ay = -mu * ry / r^3;
   az = -mu * rz / r^3;
   a = [ax; ay; az];
   xdot = [v; a + a_drag + a_J2];
end
```

PQW2ECI.m

```
function x_ECI = PQW2ECI(x_PQW, parameters)
   i     = parameters(3);
   Omega = parameters(4);
   w     = parameters(5);
```

```
    C = [(cos(Omega)*cos(w) - sin(Omega)*sin(w)*cos(i)), (-cos(Omega)*sin(w) -
sin(Omega)*cos(w)*cos(i)),  (sin(Omega)*sin(i));
        (sin(Omega)*cos(w) + cos(Omega)*sin(w)*cos(i)), (-sin(Omega)*sin(w) +
cos(Omega)*cos(w)*cos(i)), (-cos(Omega)*sin(i));
        (sin(w)*sin(i)),                (cos(w)*sin(i)),                (cos(i))];
    x_ECI = C * x_PQW;
end
```

Quat2eul.m

```
function euler = quat2eul(q)
    Ex = q(1,:);
    Ey = q(2,:);
    Ez = q(3,:);
    n  = q(4,:);
    roll  = atan2(2*(n.*Ex + Ey.*Ez), 1 - 2*(Ex.^2 + Ey.^2));
    pitch = asin(max(-1, min(1, 2*(n.*Ey - Ez.*Ex))));
    yaw   = atan2(2*(n.*Ez + Ex.*Ey), 1 - 2*(Ey.^2 + Ez.^2));
    euler = [roll; pitch; yaw];
end
```

Quat_mult.m

```
function q = quat_mult(q1, q2)
    e1 = q1(1:3);
    n1 = q1(4);
    e2 = q2(1:3);
    n2 = q2(4);
    e = n1*e2 + n2*e1 + cross(e1,e2);
    n = n1*n2 - dot(e1,e2);
    q = [e; n];
    q = q / norm(q);
end
```

Quat2rotm_body.m

```
function R = quat2rotm_body(q)
    Ex = q(1);
    Ey = q(2);
    Ez = q(3);
    n = q(4);
    R = [(1-2*(Ey^2+Ez^2)),  2*(Ex*Ey-n*Ez),   2*(Ex*Ez+n*Ey);
```

```
        2*(Ex*Ey+n*Ez),   (1-2*(Ex^2+Ez^2)),  2*(Ey*Ez-n*Ex);
        2*(Ex*Ez-n*Ey),    2*(Ey*Ez+n*Ex),   (1-2*(Ex^2+Ey^2))];
end
```

skew.m

```
function S = skew(v)
   S = [ 0,    -v(3),  v(2);
         v(3),  0,    -v(1);
        -v(2),  v(1),  0  ];
end
```

steering_logic.m

```
function [lambda, beta, delta_dot, Omega_dot] = steering_logic(t, lambda0, mu1, S_CMG,
beta0, mu2, S_RW, Iw, Omega, alpha, A, B, tau, d)
   e1 = 0.001*sin(t+0.25);
   e2 = 0.001*sin(t+0.13);
   e3 = 0.001*sin(t+0.05);
   E = [1, e1, e2;
        e1, 1, e3;
        e2, e3, 1];
   lambda = lambda0*exp(-mu1 * S_CMG);
   beta   = beta0*exp(-mu2 * S_RW);
   Omega_diag = diag(Omega);
   cmg_torque_term = A' * ((A * A' + lambda * E) \ tau);
   term2           = cmg_torque_term + beta * d;
   delta_dot       = (alpha / Iw) * (Omega_diag \ term2);
   Omega_dot       = ((1 - alpha) / Iw) * (B' * ((B * B') \ tau));
end
```

11) References

[1] K. S. Patankar, A HYBRID CMG-RW ACTUATOR FOR RAPID RETARGETING AND PRECISION, Gainesville: University of Florida, 2015.

[2] F. A. Leve, Novel Steering and Control Algorithms for Single-Gimbal Control Moment Gyroscopes, Gainesville: University of Florida, 2010.

[3] P. C. Hughes, Spacecraft Attitude Dynamics, New York: Dover Publications, 1940.

[4] P. Kelly, Control of Geostationary Satellite Orbits Using Solar Radiation Pressure, Gainesville: University of Florida, 2016.

[5] J. R. Wertz, Spacecraft Attitude Determination and Control, Boston: D. Reidel Publishing Company, 1978.

[6] H. F. Kennel, "Angular Momentum Desaturation for SKYLAB Using Gravity Gradient Torques," *NASA Technical Memorandum,* no. George C. Marshall Space Flight Center, p. 64, 1971.

[7] H. Sekine, S. Nomura, N. Nakamura and N. Venicius, "Angular momentum management strategies for deep space," *ELSEVIER,* no. 31 July 2025, p. 15, 2025.

[8] T. Ito, S. Ikari, R. Funase, S. Sakai and Y. Kawakatsu, "Active use of solar radiation pressure for angular momentum control of the PROCYON micro-spacecraft," *ScienceDirect,* vol. 152, no. November 2018, pp. 299-309, 2018.

[9] J. B. Rodríguez, N. H. Crisp, S. Livadiotti, P. C. Roberts and D. González, "VLEO EO Satellite Aerodynamic Control Tecnhiques and Mechanisms," Discoverer, 2019.

[10] M. J. Sidi, Spacecraft Dynamics and Control, New York: Cambridge University Press, 1997.

[11] K. Lemmer, "Propulsion for CubeSats," *Acta Astronautica,* vol. 134, no. May 2017, pp. 231-243, 2017.

[12] F. L. Markley and J. L. Crassidis, "Fundamentals of Spacecraft Attitude Determination and Control," New York, Springer, 2014.

[13] D. A. Vallado, Fundamentals of Astrodynamics and Applications, Microcosm Press/Springer, 2013.